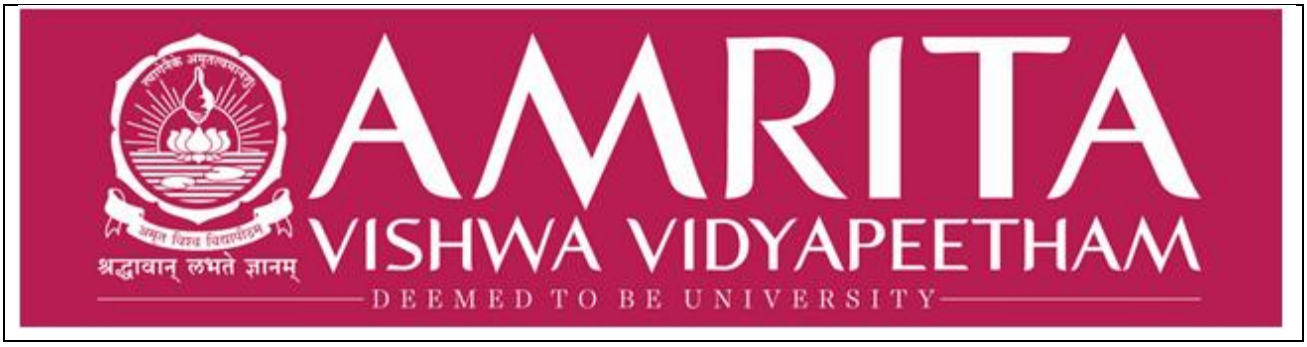


User Interface Design
23CSE113 LAB MANUAL

SUBMITTED BY	
NAME	DEEPAK E
ROLL NO	AV.SC.U4CSE25027
YEAR/SEM/SECTION	1st YEAR/SEM-2/CSE-A
CREDITS	03

SUBMITTED TO	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFESSOR(SI.GRADE)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

	home-appliances, books. Those should display on products.		
7	**WEEK 7**: To create the repository in GIT Hub and push or upload files of all the programs that are done to create different webpages and websites.	92-95	
8	**WEEK 8**:8a. Create a webpage that displays a square that continuously rotates 360° using CSS animation. **8b.** Create a ball that moves up and down continuously using CSS animation. **8c.** Create a circle that moves horizontally and rotes at the same time.	95-101	
9	Create an webpage using html, CSS, java script based on the given conditions: a)Create a button for each program, based on the button clicked the corresponding program has to be executed and should display the output. b)The logics for each button includes finding an even numbers, odd number, prime or not, factorial of a number, amstrong or not. c)Use appropriate designs to display the output		
10	a) Write JavaScript conditional statement to find the sign of product of three numbers using functions . b) Write a java script conditional statements to sort 3 numbers using functions display an alert box to show the result c) Write a java script conditional statement to find largest of find largest of five numbers using function and onclick event .display an alert box to show the result d) write a java script for loop that will iterate from 0 to 15. For each iteration ,it will check if the current number odd or even , and display a message on screen e) write a java script program to read 6 subjects marks from the student then computes the average marks of students		

	then , this average is used to determine the corresponding grade.		
11	a) A website requires users to enter a username. The system should display a message while typing if the username is less than 5 characters. b) A registration form should validate email format before submission. If invalid, the form should not be submitted c) A system should check password strength while typing and display whether it is weak or strong. d) A form should validate a mobile number when the user leaves the input field. The number must be exactly 10 digits e) Design an HTML page, such that the registration form must validate: Name (not empty) Email (valid format) Password (minimum 6 characters)		
12	**Aim:** To create and execute a basic Electron.js desktop application using Visual Studio Code, Node.js, and Electron.		

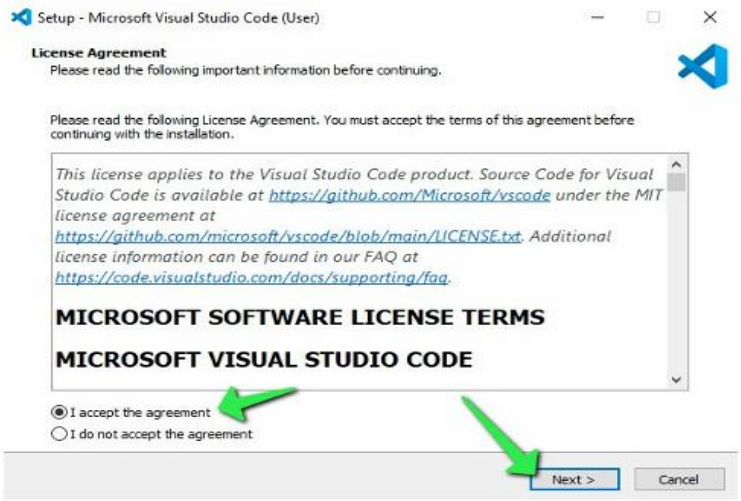
WEEK – 1

1.AIM: Steps to install VS Code.

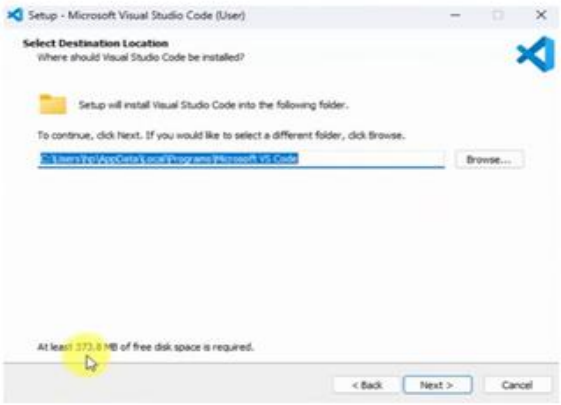
Procedure:

Step-1

Step-2



Step-3

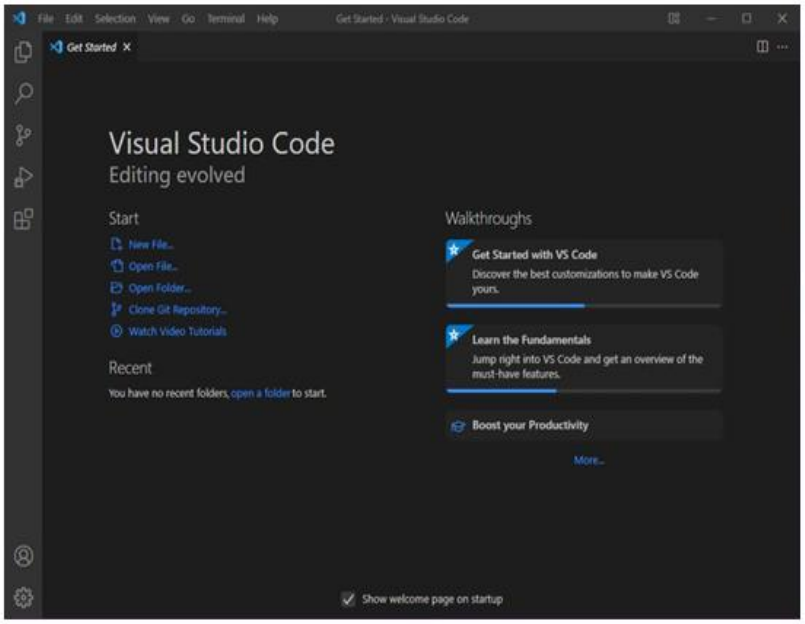


Step-4

Step-5

Step-6

Step-7



What is HTML?

- HTML stands for Hyper Text Markup Language
- HTML is the standard markup language for creating Web pages
- HTML describes the structure of a Web page
- HTML consists of a series of elements
- HTML elements tell the browser how to display the content

- HTML elements label pieces of content such as "this is a heading", "this is a paragraph", "this is a link", etc.

Definition of HTML Tags:

- The **<!DOCTYPE html>** declaration defines that this document is an HTML5 document
- The **<html>** element is the root element of an HTML page
- The **<head>** element contains meta information about the HTML page
- The **<title>** element specifies a title for the HTML page (which is shown in the browser's title bar or in the page's tab)
- The **<body>** element defines the document's body, and is a container for all the visible contents, such as headings, paragraphs, images, hyperlinks, tables, lists, etc.
- The **<h1>** element defines a large heading
- The **<p>** element defines a paragraph

HTML Page Structure:

```
<html>
    <head>
        <title>Page title</title>
    </head>
    <body>
        <h1>This is a heading</h1>
        <p>This is a paragraph.</p>
        <p>This is another paragraph.</p>
    </body>
</html>
```

SAMPLE CODES:

1. 1. **AIM:** To earn the basics of html(basics.html).

PROGRAM:

```
<!DOCTYPE html>
```

```
<html>
   <head>
      <title>First Web Page</title>
   </head>
   <body>
      <h1>Happy to learn HTML</h1>
      <p>This is my first sample code</p>
   </body>
</html>
```

OUTPUT:

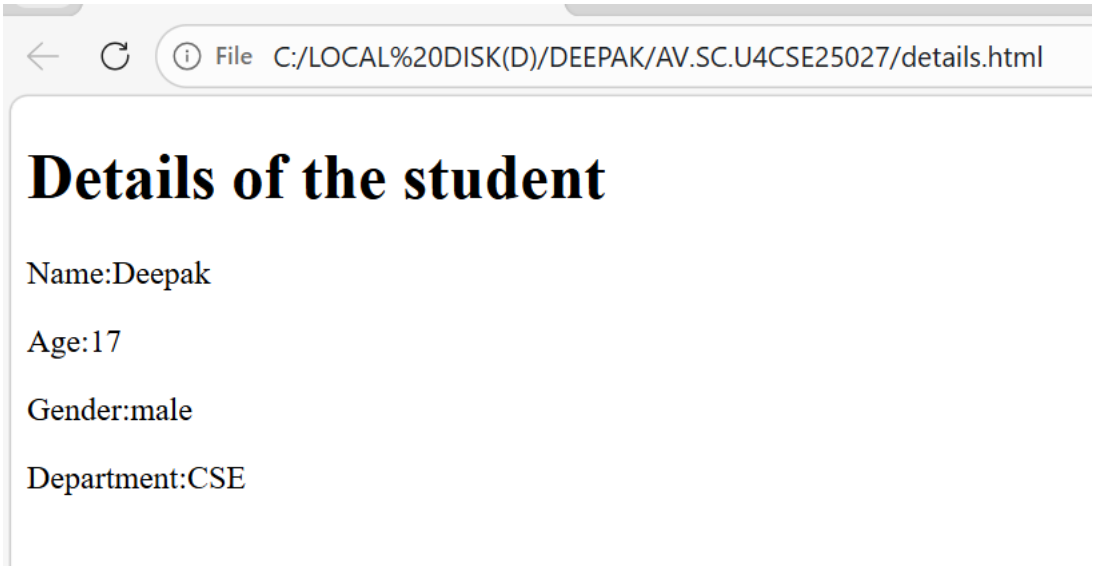
2.AIM: Understanding basic html tags**.**

Details of student:

```
<!DOCTYPE html>
<html>
   <head>
      <title>student Details Page</title>
   </head>
   <body>
      <h1>Details of the student</h1>
      <p>Name :Deepak</p>
      <p>Age :17</p>
      <p>Gender :male</p>
      <p>Department :CSE</p>
```

```
    </body>
</html>
```

OUTPUT:



Headings code(h1):

PROGRAM:

```
<!DOCTYPE html>
<html>
    <head>
        <title>Details Page</title>
    </head>
    <body>
        <h1>This is heading 1</h1>
        <h2>This is heading 2</h2>
        <h3>This is heading 3</h3>
        <h4>This is heading 4</h4>
        <h5>This is heading 5</h5>
        <h6>This is heading 6</h6>
    </body>
</html>
```

OUTPUT:

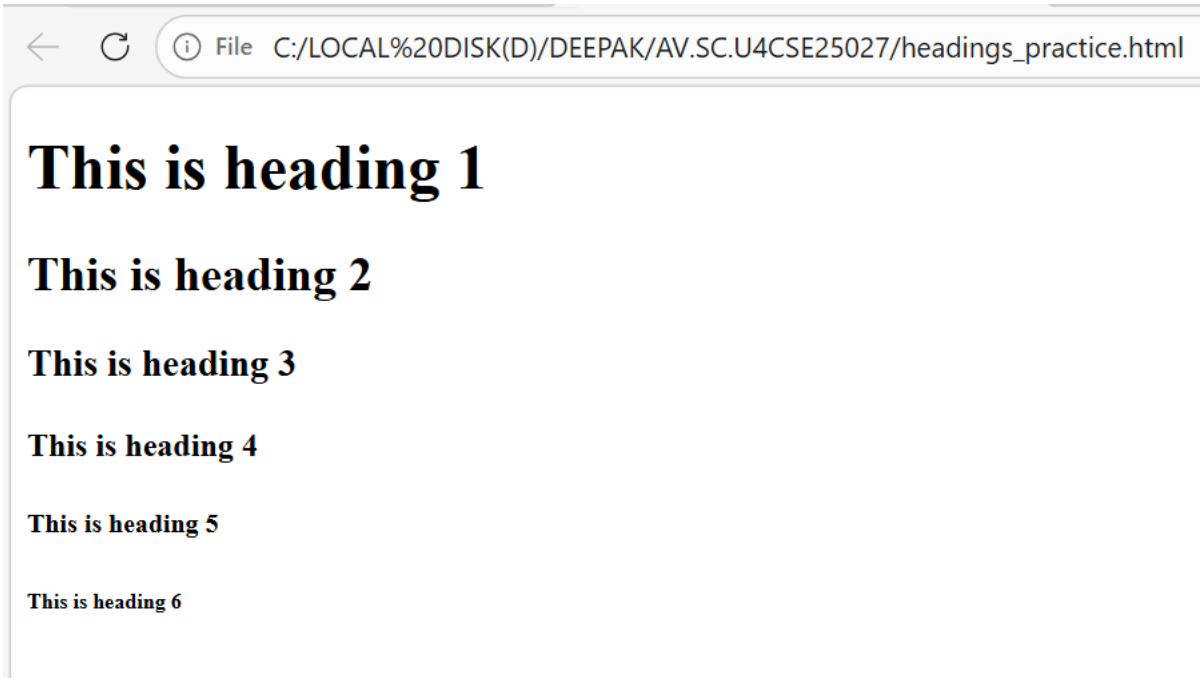


Image code(img scr):

PROGRAM:

```
<!DOCTYPE html>
<html>
   <head>
      <title>Html image</title>
   </head>
   <body>
      <h1>Image of amrita using HTML</h1>
      <p>This page gives image of amrita.</p>
      <img src="C:\Users\DEEBAK\Downloads\amrita-vishwa-vidyapeetham-university-logo-2024.jpg" alt="amrita.edu" width="225" height="225">
   </body>
</html>
```

OUTPUT:

Why alt is used here:

The alt (alternative text) describes the image **in words**. It is shown or read **when the image cannot be displayed** and is also important for **accessibility.**

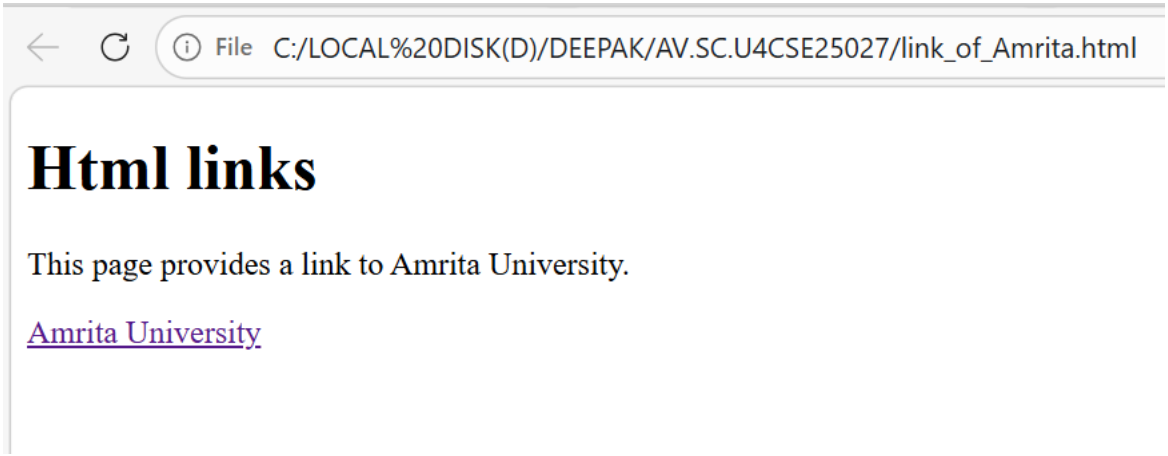
Links code(href):

1 Amrita code:

PROGRAM:

```
<!DOCTYPE html>
<html>
   <head>
      <title>Link of Amrita University</title>
   </head>
   <body>
      <h1>Html links</h1>
      <p>This page provides a link to Amrita University.</p>
      <a href=https://www.amrita.edu/>Amrita University</a><br>
   </body>
```

OUTPUT:



2 SRM University code:

PROGRAM:

```
<!DOCTYPE html>
<html>
   <head>
      <title>Html link for SRM University</title>
   </head>
   <body>
      <h1>link of SRM University</h1>
      <p>This page provides a link to SRM University.</p>
      <a href=https://www.srmist.edu.in/>SRM University</a><br>
   </body>
</html>
```

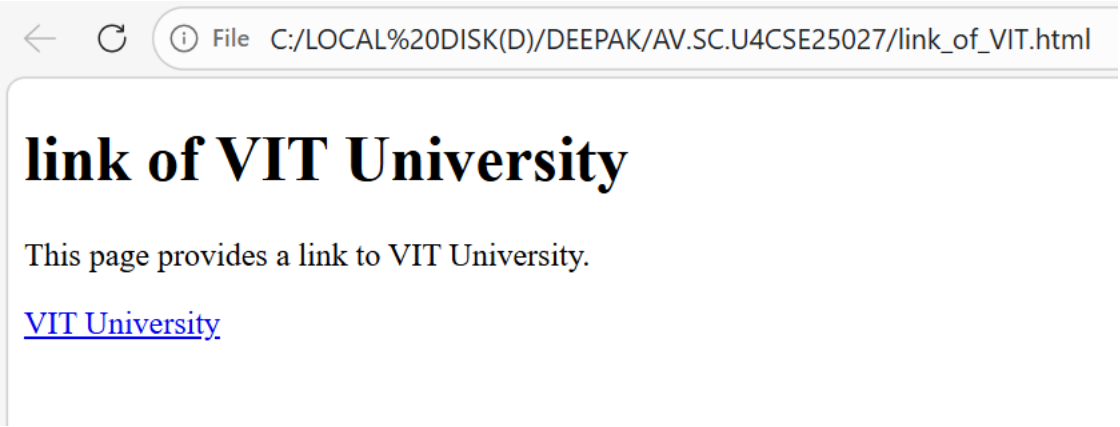
OUTPUT:

3 VIT University code:

OUPUT:

```
<!DOCTYPE html>
<html>
   <head>
      <title>Link of Amrita University</title>
   </head>
   <body>
      <h1>link of VIT University</h1>
      <p>This page provides a link to VIT University.</p>
      <a href=https://www.vit.edu/>VIT University</a><br>
   </body>
</html>
```

OUTPUT:



Why href is used here:

In HTML, the <a> (anchor) tag creates a **hyperlink**, but it does **nothing by itself** unless you tell it **where to go**.
The href (Hypertext Reference) attribute defines **the web address that opens when the link is clicked**.

Break using (br code):

PROGRAM:

```
<!DOCTYPE html>
<html>
   <head>
      <title>paragraph with line break</title>
   </head>
   <body>
      <h1>html using br tag</h1>
      <p>This is a <br> paragraph with a line break.</p>
   </body>
</html>
```

OUTPUT:

Why did we use br tag:

In HTML, text inside a <p> (paragraph) tag normally flows **continuously on the same line** until the browser automatically wraps it.
The
 tag forces the text to **move to a new line without starting a new paragraph**.

 is an **empty tag**, so it does not need a closing tag.

Using hr:

PROGRAM:

```
<!DOCTYPE html>
<html>
   <head>
      <title>paragraph with line break</title>
   </head>
   <body>
      <h1>html using hr tag</h1>
      <p>This is a <br> paragraph with a line break.</p>
   </body>
</html>
```

OUTPUT:

Why did I use hr tag:

In HTML, the <hr> tag represents a **thematic break** between different sections of content. It helps the reader understand that one topic or section has ended and another has begun.

<hr> is an **empty tag**, so it does not need a closing tag.

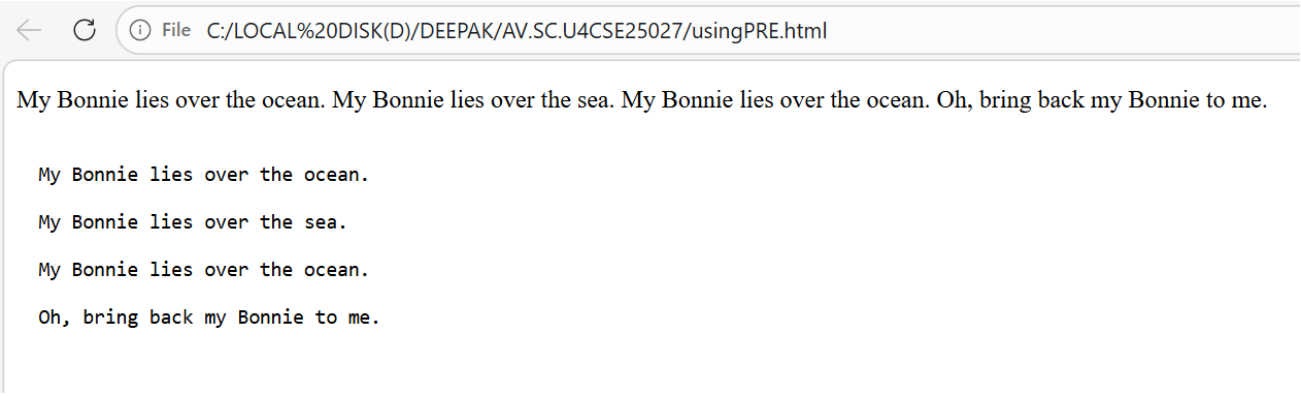
using pre code:

PROGRAM:

```
<!DOCTYPE html>
<html>
  <head>
    <title>horizontal rules using hr</title>
  </head>
  <body>
    <p>
 My Bonnie lies over the ocean.

 My Bonnie lies over the sea.

 My Bonnie lies over the ocean.
```

```
Oh, bring back my Bonnie to me.

    </p>

    <pre>
My Bonnie lies over the ocean.

My Bonnie lies over the sea.

My Bonnie lies over the ocean.

Oh, bring back my Bonnie to me.
  </pre>
  </body>
</html>
```

OUTPUT:



Why did we use pre tag:

In normal HTML paragraphs (<p>), the browser **collapses extra spaces and line breaks**.
That's why the formatting inside the first <p> tag does **not** appear the same as how you typed it in the code.

The <pre> tag (preformatted text) tells the browser:

"Show this text exactly as written."

WEEK-2

1. **AIM:** Understanding basic HTML Tags.

Using Style:

PROGRAM:

(a)<!DOCTYPE html>

<html>

<head>

<title>styles example</title>

<style>

.highlight {

background-color: yellow;

font-weight: bold;

}

.italic {

font-style: italic;

}

</style>

</head>

<body>

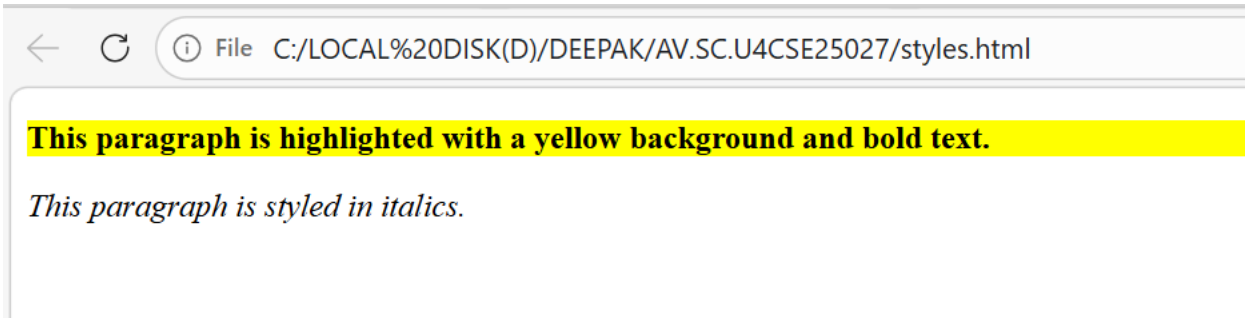
This paragraph is highlighted with a yellow background and bold text.

This paragraph is styled in italics.

</body>

</html>

OUTPUT:



(b)

PROGRAM:

```
<!DOCTYPE html>

<html>

<head>

<title>styles example</title>

</head>

<body style="background-color: lightpink;background-image: url('amma.webp') ;background-repeat: no-repeat;background-size: cover;">

<h1 style="text-align: center; color:brown; border: 3px solid red;">Amritānandamayī Amma</h1>

<pre style="font-size: medium;color:deeppink;font-family: cursive;font-style: italic;font-size: large;text-align: center;padding: 12px; border: 4px dashed oklab(44.731% -0.05508 -0.07556);;">

Sri Mātā Amritānandamayī Devi (born Sudhamani Idamannel; 27 September 1953), often known as Amma ("Mother"),is an Indian Hindu spiritual

leader, godwoman,[1] guru and humanitarian.[2][3] She is the chancellor of Amrita Vishwa Vidyapeetham, a multi-campus research university

.[4]In 2018, she was felicitated by Indian Prime Minister Narendra Modi for making the largest contribution to the Government of India's

Clean India Campaign Swachh Bharat Mission.[5] She was the first recipient of Vishwaratna Puraskar (Gem of the World Award) by Hindu

Parliament.[6]

<p class="highlight">This paragraph is highlighted with a yellow background and bold text.</p>
```

This paragraph is styled in italics.

</body>

</html>

OUTPUT:

Table(Using TR):

PROGRAM:

<!DOCTYPE html>

<html>

<head>

<title>styles example</title>

</head>

<body>

<h1>HTML </h1>

<table border="2" style="width:100%;text-align: center;color:burlywood;font-family: Arial, Helvetica, sans-serif;font-style: italic;background-color: aqua;width:80%;">

<caption style="font-family: cursive;font-style: italic;color:brown;font-size: x-large;background-color:aquamarine;">Company Contacts</caption>

<tr>

<th>serial no</th>

<th>company</th>

<th>contact</th>

<th>country</th>

</tr>

<tr>

<td>1</td>

<td>Alfreds Futterkiste</td>

<td>Maria Anders</td>

<td>Germany</td>

</tr>

<tr>

<td>2</td>

<td>Centro comercial Moctezuma</td>

<td>Francisco Chang</td>

<td>Mexico</td>

</tr>

<tr>

<td>3</td>

<td>Ernst Handel</td>

<td>Roland Mendel</td>

<td>Austria</td>

</tr>

<tr>

<td>4</td>

<td>Island Trading</td>

```
<td>Helen Bennett</td>
<td>UK</td>
</tr>
<tr>
<td>5</td>
<td>Laughing Bacchus Winecellars</td>
<td>Yoshi Tannamuri</td>
<td>Canada</td>
</tr>
<tr>
<td>6</td>
<td>Magazzini Alimentari Riuniti</td>
<td>Giovanni Rovelli</td>
<td>Italy</td>
</tr>
</table>
```

OUTPUT:

3.Creating Menu

PROGRAM:

```
<!DOCTYPE html>

<html>

<head>

<title style="background-color: aliceblue ;text-align: center; color: darkblue;">Le Bonquent Restaurent Menu</title>

</head>

<body style="background-color: lightyellow; background-image: url('restaurent1.jpeg'); background-repeat:repeat;color:rgba(226,43,180,0.545);"> </body>

<h1 style="font-family: 'cursive', sans-serif; text-align: center; color: rgba(226, 43, 180, 0.545); border: 3px solid red; background-color: aliceblue;">Le Bonquent Restaurent Menu</h1>

<table border="2" style="position: relative; right: 19%; margin: auto; text-align: center; color: darkblue; font-family: 'Trebuchet MS', sanswyhwg-serif; background-color:rgba(244, 255, 246, 0.95) ;">

<caption style="font-family: 'Comic Sans MS', cursive, sans-serif; font-style: italic; color: darkred; font-size: x-large; background-color: lightgoldenrodyellow;">Veg</caption>

<tr>

<th>Dish</th>

<th>Description</th>

<th>Price</th>
```

```
</tr>
<tr>
<td>Margherita Pizza</td>
<td>Classic pizza topped with fresh tomatoes, mozzarella cheese, and basil.</td>
<td>$10.99</td>
</tr>
<tr>
<td>Vegetable Stir Fry</td>
<td>Assorted vegetables sautéed in a savory soy sauce served over rice.</td>
<td>$9.49</td>
</tr>
<tr>
<td>Caesar Salad</td>
<td>Crisp romaine lettuce tossed with Caesar dressing, croutons, and Parmesan cheese.</td>
<td>$7.99</td>
</tr>
<tr>
<td>Paneer Tikka</td>
<td>Grilled paneer cubes marinated in spices and yogurt.</td>
<td>$11.49</td>
</tr>
<tr>
<td>Veggie Burger</td>
<td>Delicious veggie patty with lettuce, tomato, and cheese on a toasted bun.</td>
<td>$10.49</td>
</tr>
```

```
<tr>

<td>Chocolate Lava Cake</td>

<td>Warm chocolate cake with a gooey molten center.</td>

<td>$6.99</td>

</tr>

</table>

<table border="2" style="position: relative; left: 20%; margin: auto; text-align: center; color: darkblue; font-family: 'Trebuchet MS', sans-serif; background-color:rgba(255, 245, 243, 0.95) ;">

<caption style="font-family: 'Comic Sans MS', cursive, sans-serif; font-style: italic; color: darkred; font-size: x-large; background-color: lightgoldenrodyellow;">Non Veg</caption>

<tr>

<th>Dish</th>

<th>Description</th>

<th>Price</th>

</tr>

<tr>

<td>Chicken Alfredo</td>

<td>Creamy Alfredo sauce over fettuccine pasta with grilled chicken.</td>

<td>$13.99</td>

</tr>

<tr>

<td>Beef Burger</td>

<td>Juicy beef patty with lettuce, tomato, and cheese on a toasted bun.</td>

<td>$11.49</td>

</tr>

<tr>

<td>Fish and Chips</td>
```

<td>Crispy battered fish served with golden fries and tartar sauce.</td>

<td>$12.49</td>

</tr>

<tr>

<td>Chicken Tikka Masala</td>

<td>Grilled chicken pieces in a spiced tomato and cream sauce.</td>

<td>$14.99</td>

</tr>

<tr>

<td>Shrimp Scampi</td>

<td>Sautéed shrimp in garlic butter sauce served over linguine.</td>

<td>$15.99</td>

</tr>

<tr>

<td>New York Cheesecake</td>

<td>Rich and creamy cheesecake with a graham cracker crust.</td>

<td>$7.49</td>

</tr>

<table border="2" style="position: relative; right: 18%; margin: auto; text-align: center; color: darkblue; font-family: 'Trebuchet MS', sans-serif; background-color:rgba(255, 245, 243, 0.95) ;">

<caption style="font-family: 'Comic Sans MS', cursive, sans-serif; font-style: italic; color: darkred; font-size: x-large; background-color: lightgoldenrodyellow;">Deserts</caption>

<tr>

<th>Dish</th>

<th>Description</th>

<th>Price</th>

</tr>

<tr>
<td>Chocolate Brownie</td>
<td>Rich and fudgy chocolate brownie topped with nuts.</td>
<td>$5.99</td>
</tr>
<tr>
<td>Vanilla Ice Cream</td>
<td>Creamy vanilla ice cream made with real vanilla beans.</td>
<td>$4.49</td>
</tr>
<tr>
<td>Fruit Tart</td>
<td>Buttery tart crust filled with custard and topped with fresh fruits.</td>
<td>$6.49</td>
</tr>
<tr>
<td>Cheesecake</td>
<td>Classic New York-style cheesecake with a graham cracker crust.</td>
<td>$6.99</td>
</tr>
<tr>
<td>Tiramisu</td>
<td>Italian dessert made with layers of coffee-soaked ladyfingers and mascarpone cheese.</td>
<td>$7.49</td>
</tr>
<tr>

<td>Apple Pie</td>

<td>Traditional apple pie with a flaky crust and cinnamon-spiced filling.</td>

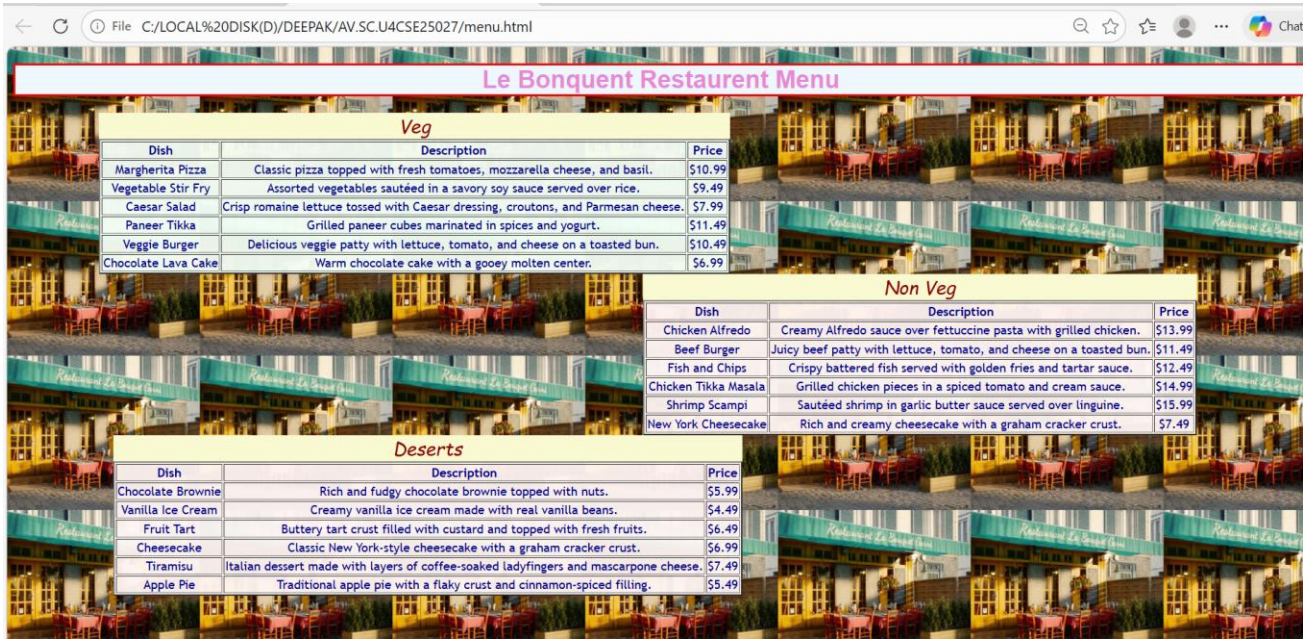
<td>$5.49</td>

</tr>

</table>

</html>

OUTPUT:



EXPLANATION OF TAGS:

The Core Table Structure;

1.<table>: The wrapper element that defines the start and end of the table.

2.<tr> (Table Row): Defines a single horizontal row of cells.

3.<th> (Table Header): Used for header cells (usually bold and centered by default). These represent the titles of columns or rows.

4.<td> (Table Data): Defines a standard data cell.

5.<caption>: Provides a title or summary for the table, usually displayed directly above the grid.

In HTML, style is an attribute used to change the visual appearance of an element.

Style Attribute:

In HTML, style is an attribute used to change the visual appearance of an element.

While HTML provides the structure (like text, rows, and columns), the style attribute provides the design (like colors, fonts, and spacing).

WEEK-3

3a. AIM: Construct a class time table using HTML Tags.

PROGRAM:

```
<!DOCTYPE html>

<html>

<head>

<title style="background-color:white ;text-align: center; color: white;">CSE-A Class Timetable</title>

</head>

<body style="background-color: white; "></body>

<h1 style="font-family: 'Arial Black', sans-serif; text-align: center; color:white; border: 3px solid; background-color: lightgrey;">CSE-A Class Timetable</h1>

<table border="1" style="text-align:center;margin: auto; color:black; font-family: Arial, Helvetica, sans-serif; background-color: white ;">

<tr></tr>

<th colspan="12">Master Time Table CSE II Semester A Section</th>

</tr>

<tr style="text-align: left;"></tr>

<th colspan="4" style="text-align: left;">Dept.CSE</th>

<th colspan="8" rowspan="3" style="text-align: left;">Class Room Venue: 510</th>

</tr>

<tr>

<th colspan="4" style="text-align: left;">Class: B. Tech. CSE-A</th>

</tr>
```

<tr>

<th colspan="4" style="text-align: left;">Semester-II</th>

</tr>

<tr>

<th colspan="4" style="text-align: left;">Class Advisors:

1. Name: Dr. K. Sindhu

Email : k_sindhu@av.amrita.edu

Mobile Number : +91-9959387057</th>

<th colspan="8" style="text-align: left;">2. Name: Dr. Sumanta Kumar Nanda

Email : n_sumantakumar@av.amrita.edu

Mobile Number :+91- 7978115993</th>

</tr>

<tr>

<th rowspan="2">Time/Day</th>

<th>Slot 1</th>

<th>Slot 2</th>

<th rowspan="2">10:35am-10:45am</th>

<th>Slot 3</th>

<th>Slot 4</th>

<th>Slot 5</th>

<th rowspan="2">01:15pm-02:10pm</th>

<th>Slot 6</th>

<th rowspan="2">3:00pm-3:10pm</th>

<th>Slot 7</th>

<th>Slot 8</th>

</tr>

```
<tr>
<td>8:55-9:45am</td>
<td>9:45-10:35am</td>
<td>10:45-11:35am</td>
<td>11:35-12:25pm</td>
<td>12:25-1:15pm</td>
<td>2:10-3:00pm</td>
<td>3:10-4:00pm</td>
<td>4:00-4:50pm</td>
</tr>
<tr>
<td>Monday</td>
<td>23CSE111 Object Oriented Programming</td>
<td>23CSE113 User Interface Design</td>
<td rowspan="5"style=" writing-mode:vertical-rl;
text-orientation:mixed;
transform: rotate(180deg);
text-align: center;
font-weight: bold;background-color:yellow;">Short Break</td>
<td colspan="2">23MAT116 Discrete Mathematics_lab_lab3</td>
<td>Library</td>
<td rowspan="5" style=" writing-mode: vertical-rl;
text-orientation: mixed;
transform: rotate(180deg);
text-align: center;
font-weight: bold;background-color:yellow;">Lunch Break</td>
```

<td>Library</td>

<td rowspan="5" style=" writing-mode: vertical-rl;

text-orientation: mixed;

transform: rotate(180deg);

text-align: center;

font-weight: bold;background-color: yellow;">Short Break</td>

<td colspan="2">Sports</td>

</tr>

<tr>

<td>Tuesday</td>

<td colspan="2">23CSE111 Object Oriented Programming_lab1</td>

<td>23MAT116 Discrete Mathematics</td>

<td colspan="2"> 23MAT117 Linear Algebra_LAB(BYOD)</td>

<td>Library</td>

<td colspan="2">22ADM111 Glimpses of Glorious India</td>

</tr>

<tr>

<td>Wednesday</td>

<td> 23PHY115Modern Physics</td>

<td>23MAT117 Linear Algebra</td>

<td> LIBRARY</td>

<td>23PHY115Modern Physics</td>

<td>23CSE111Object Oriented Programming</td>

<td>23MEE115Manufacturing Practice LAB</td>

<td colspan="2">23MEE115Manufacturing Practice LAB</td>

</tr>

<tr>

<td>Thursday</td>

<td> 23CSE113User Interface Design</td>

<td>23MAT117 Linear Algebra</td>

<td colspan="2">23CSE113User Interface Design Lab (BYOD)</td>

<td> 23MAT116Discrete Mathematics</td>

<td style="color: red;">COUNSELLING</td>

<td colspan="2">Library/Sports</td>

</tr>

<tr>

<td>Friday</td>

<td> 23MAT117 Linear Algebra</td>

<td>23PHY115Modern Physics(T)</td>

<td>23CSE111 Object Oriented Programming</td>

<td>23MAT116Discrete Mathematics</td>

<td style="color: red;">COUNSELLING</td>

<td> 22ADM111Glimpses of Glorious India(P)</td>

<td colspan="2">Library/Sports</td>

<tr>

<td>Course Code</td>

<td colspan="3">Course Name</td>

<td>L-T-P</td>

<td>Credits</td>

<td colspan="3">Faculty Name</td>

<td></td>

<td colspan="2">Assisting Faculty</td>

</tr>

<tr>

<td>23MAT116</td>

<td colspan="2">Discrete Mathematics</td>

<td></td>

<td>3-0-2</td>

<td>4</td>

<td colspan="3">Dr.Ram Pratap Chauhan</td>

<td></td>

<td colspan="2"></td>

</tr>

<tr>

<td>23MAT117</td>

<td colspan="2">Linear Algebra</td>

<td></td>

<td>3-0-2</td>

<td>4</td>

<td colspan="3">Dr.Rashmi Prasad</td>

<td></td>

<td colspan="2"></td>

<tr>

<td>23CSE111</td>

<td colspan="2">Object Oriented Programming</td>

<td></td>

<td>3-0-2</td>

<td>4</td>

<td colspan="3">Dr. Balaji TK</td>

<td></td>

<td colspan="2">Dr. Satya Rajendra Singh</td>

</tr>

<tr>

<td>23PHY115</td>

<td colspan="2">Modern Physics</td>

<td></td>

<td>2-1-0</td>

<td>3</td>

<td colspan="3"> Dr. C R Kesavalu</td>

<td></td>

<td colspan="2"></td>

<tr>

<td>23CSE113</td>

<td colspan="2">User Interface Design</td>

<td></td>

<td>2-0-2</td>

<td>3</td>

<td colspan="3">Dr.Rajkumar Batchu</td>

<td></td>

<td colspan="2">Mr. Barge Bharadwaj</td>

</tr>

<tr>

<td>23MEE115</td>

<td colspan="2">Manufacturing Practice</td>

<td></td>

<td>0-0-3</td>

<td>1</td>

<td colspan="3">Dr. Sathies S.</td>

<td></td>

<td colspan="2"></td>

</tr>

<tr>

<td>22ADM111</td>

<td colspan="2">Glimpses of Glorious India</td>

<td></td>

<td>2-0-1</td>

<td>2</td>

<td colspan="3">Dr. Siva Panuganti</td>

<td></td>

<td colspan="2"></td>

</tr>

<tr>

<td></td>

<td colspan="2">Total (15L + 1Tut +12P =28 hrs)</td>

<td></td>

<td>15-1-12</td>

<td>21</td>

<td colspan="3"></td>

<td></td>

<td colspan="2"></td>

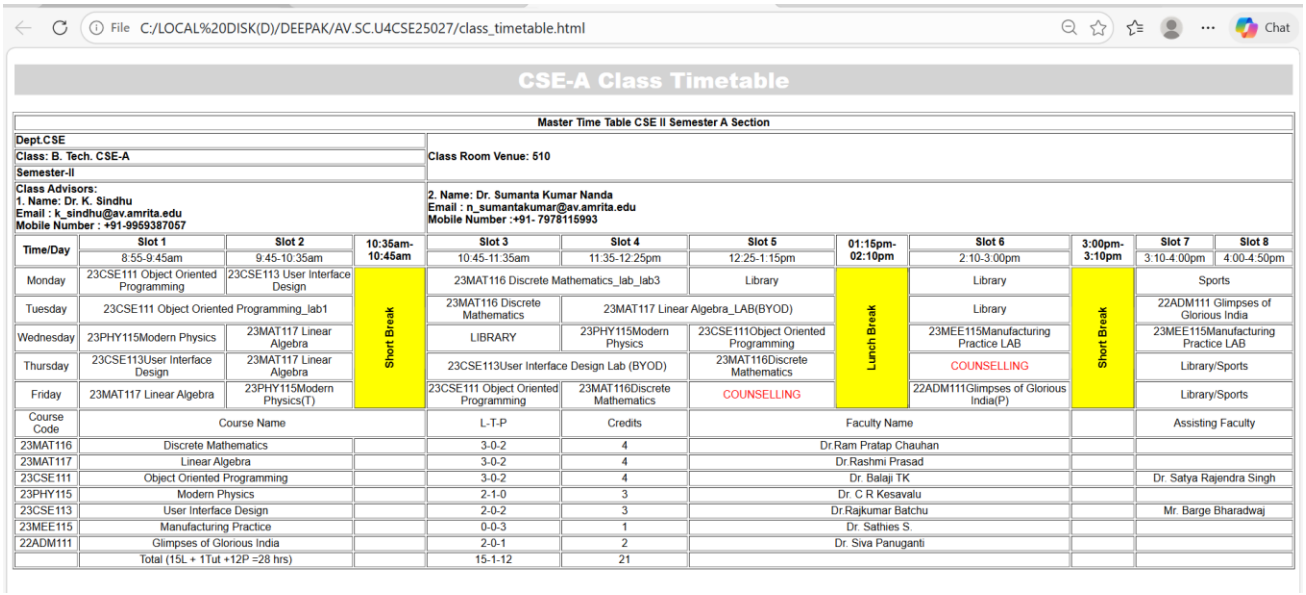
</tr>

</table>

</body>

</html>

OUTPUT:



File C:/LOCAL%20DISK(D)/DEEPAK/AV.SC.U4CSE25027/class_timetable.html

Chat

CSE-A Class Timetable

Master Time Table CSE II Semester A Section

Dept.CSE											
Class: B. Tech. CSE-A				**Class Room Venue: 510**							
Semester-II											
Class Advisors: **1. Name: Dr. K. Sindhu** **Email : k_sindhu@av.amrita.edu** **Mobile Number : +91-9959387057**				**2. Name: Dr. Sumanta Kumar Nanda** **Email : n_sumantakumar@av.amrita.edu** **Mobile Number :+91- 7978115993**							
Time/Day	**Slot 1**	**Slot 2**	**10:35am-10:45am**	**Slot 3**	**Slot 4**	**Slot 5**	**01:15pm-02:10pm**	**Slot 6**	**3:00pm-3:10pm**	**Slot 7**	**Slot 8**
	8:55-9:45am	9:45-10:35am		10:45-11:35am	11:35-12:25pm	12:25-1:15pm		2:10-3:00pm		3:10-4:00pm	4:00-4:50pm
Monday	23CSE111 Object Oriented Programming	23CSE113 User Interface Design	Short Break	23MAT116 Discrete Mathematics_lab_lab3		Library	Lunch Break	Library	Short Break	Sports	
Tuesday	23CSE111 Object Oriented Programming_lab1			23MAT116 Discrete Mathematics	23MAT117 Linear Algebra_LAB(BYOD)			Library		22ADM111 Glimpses of Glorious India	
Wednesday	23PHY115Modern Physics	23MAT117 Linear Algebra		LIBRARY	23PHY115Modern Physics	23CSE111Object Oriented Programming		23MEE115Manufacturing Practice LAB		23MEE115Manufacturing Practice LAB	
Thursday	23CSE113User Interface Design	23MAT117 Linear Algebra		23CSE113User Interface Design Lab (BYOD)		23MAT116Discrete Mathematics		COUNSELLING		Library/Sports	
Friday	23MAT117 Linear Algebra	23PHY115Modern Physics(T)		23CSE111 Object Oriented Programming	23MAT116Discrete Mathematics	COUNSELLING		22ADM111Glimpses of Glorious India(P)		Library/Sports	
Course Code	Course Name			L-T-P	Credits	Faculty Name				Assisting Faculty	
23MAT116	Discrete Mathematics			3-0-2	4	Dr.Ram Pratap Chauhan					
23MAT117	Linear Algebra			3-0-2	4	Dr.Rashmi Prasad					
23CSE111	Object Oriented Programming			3-0-2	4	Dr. Balaji TK				Dr. Satya Rajendra Singh	
23PHY115	Modern Physics			2-1-0	3	Dr. C R Kesavalu					
23CSE113	User Interface Design			2-0-2	3	Dr.Rajkumar Batchu				Mr. Barge Bharadwaj	
23MEE115	Manufacturing Practice			0-0-3	1	Dr. Sathies S					
22ADM111	Glimpses of Glorious India			2-0-1	2	Dr. Siva Panuganti					
	Total (15L + 1Tut +12P =28 hrs)			15-1-12	21						

3b. AIM: Construct a Webpage that should display the

Syllabus of each Subject in image formed, also display the current semester time table.

PROGRAM:

<!DOCTYPE html>

<html lang="en">

<head>

<body style="

background-image: url('BACKimage.png');

background-repeat: no-repeat;

```
background-size: cover;
background-position: center;">
<meta charset="UTF-8">
<title>Semester 2 Syllabus</title>
<style>
body {
font-family: Arial, sans-serif;
background-color:beige;
}
h1 {
text-align: center;
margin: 20px;
color: maroon;
}
.syllabus-grid {
display: grid;
grid-template-columns: repeat(2, 1fr);
gap: 20px;
width: 90%;
margin: 0 auto;
}
.subject-card {
background-color: white;
border: 2px solid maroon;
border-radius: 10px;
padding: 10px;
```

```
text-align: center;
}
.subject-card img {
width: 100%;
height: auto;
border-bottom: 1px solid maroon;
}
.subject-name {
margin-top: 10px;
font-size: 18px;
font-weight: bold;
color: navy;
}
</style>
</head>
<body>
<div class="container">
<h1>Semester 2 Syllabus</h1>
<div class="syllabus-grid">
<!-- First Row -->
<div class="subject-card">
<img src="DiscreteMATH.jpeg" alt="Subject 1 Syllabus">
<div class="subject-name">Discrete Mathematics</div>
</div>
<div class="subject-card">
<img src="LinearMATH.jpeg" alt="Subject 2 Syllabus">
```

```
<div class="subject-name">Linear Algebra</div>

</div>

<!-- Second Row -->

<div class="subject-card">

<img src="ModernPhysics.jpeg" alt="Subject 3 Syllabus">

<div class="subject-name">Modern Physics</div>

</div>

<div class="subject-card">

<img src="ManufacturingLB.jpeg" alt="Subject 4 Syllabus">

<div class="subject-name">manufacturing practices</div>

</div>

<!-- Third Row -->

<div class="subject-card">

<img src="OOPS.jpeg" style="background-image: url('OOPS.jpeg'); background-size: cover;"
alt="Subject 5 Syllabus">

<div class="subject-name">Object Oriented Programming</div>

</div>

<div class="subject-card">

<img src="UID.jpeg" style="background-size: cover;" alt="Subject 6 Syllabus">

<div class="subject-name">User Interface Design</div>

</div>

</div>

</div>

</body>

</html>

<html>
```

```
<head>
<body style="
background-image: url('mountain.png');
background-repeat: no-repeat;
background-size: cover;
background-position: center;
">
<meta charset="UTF-8">
<title>Master Time Table – CSE II Semester</title>
<style>
body {
font-family: Arial, sans-serif;
background-color:beige;
}
h2, h3 {
text-align: center;
margin: 5px;
}
.info-table, .timetable, .course-table {
width: 95%;
margin: 15px auto;
border-collapse: collapse;
background: white;
}
th, td {
border: 1px solid black;
```

```
padding: 8px;
text-align: center;
font-size: 14px;
}
th {
background-color: #e6e6e6;
}
.break {
background-color: yellow;
font-weight: bold;
writing-mode: vertical-rl;
transform: rotate(180deg);
}
.counselling {
color: red;
font-weight: bold;
}
.left {
text-align: left;
}
</style>
</head>
<body>
<!DOCTYPE html>
<html lang="en">
<head>
```

```
<body style="
background-image: url('mountain.png');
background-repeat: no-repeat;
background-size: cover;
background-position: center;
">
<meta charset="UTF-8">
<title>Semester 2 Syllabus</title>
<style>
body {
font-family: Arial, sans-serif;
background-color:beige;
}
h1 {
text-align: center;
margin: 20px;
color: maroon;
}
.syllabus-grid {
display: grid;
grid-template-columns: repeat(2, 1fr);
gap: 20px;
width: 90%;
margin: 0 auto;
}
.subject-card {
```

```
background-color: white;
border: 2px solid maroon;
border-radius: 10px;
padding: 10px;
text-align: center;
}
.subject-card img {
width: 100%;
height: auto;
border-bottom: 1px solid maroon;
}
.subject-name {
margin-top: 10px;
font-size: 18px;
font-weight: bold;
color: navy;
}
</style>
</head>
</body>
</html>
<html lang="en">
<head>
<body style="
background-image: url('mountain.png');
background-repeat: no-repeat;
```

```
background-size: cover;
background-position: center;
">
<meta charset="UTF-8">
<title>Master Time Table – CSE II Semester</title>
<style>
body {
font-family: Arial, sans-serif;
background-color:white;
}
h2, h3 {
text-align: center;
margin: 5px;
}
.info-table, .timetable, .course-table {
width: 95%;
margin: 15px auto;
border-collapse: collapse;
background: white;
}
th, td {
border: 1px solid black;
padding: 8px;
text-align: center;
font-size: 14px;
}
```

```
th {
background-color:white;
}
.break {
background-color: yellow;
font-weight: bold;
writing-mode: vertical-rl;
transform: rotate(180deg);
}
.counselling {
color: red;
font-weight: bold;
}
.left {
text-align: left;
}
</style>
</head>
<body>
<h2>Master Time Table – CSE II Semester (A Section)</h2>
<h3>Department of Computer Science & Engineering</h3>
<body style="background-color: white; "></body>
<h1 style="font-family: 'Arial Black', sans-serif; text-align: center; color:white; border: 3px solid; background-color: lightgrey;">CSE-A Class Timetable</h1>
<table border="1" style="text-align:center;margin: auto; color:black; font-family: Arial, Helvetica, sans-serif; background-color: white ;">
<tr></tr>
```

<th colspan="12">Master Time Table CSE II Semester A Section</th>

</tr>

<tr style="text-align: left;"></tr>

<th colspan="4" style="text-align: left;">Dept.CSE</th>

<th colspan="8" rowspan="3" style="text-align: left;">Class Room Venue: 510</th>

</tr>

<tr>

<th colspan="4" style="text-align: left;">Class: B. Tech. CSE-A</th>

</tr>

<tr>

<th colspan="4" style="text-align: left;">Semester-II</th>

</tr>

<tr>

<th colspan="4" style="text-align: left;">Class Advisors:

1. Name: Dr. K. Sindhu

Email : k_sindhu@av.amrita.edu

Mobile Number : +91-9959387057</th>

<th colspan="8" style="text-align: left;">2. Name: Dr. Sumanta Kumar Nanda

Email : n_sumantakumar@av.amrita.edu

Mobile Number :+91- 7978115993</th>

</tr>

<tr>

<th rowspan="2">Time/Day</th>

<th>Slot 1</th>

<th>Slot 2</th>

<th rowspan="2">10:35am-10:45am</th>

```
<th>Slot 3</th>
<th>Slot 4</th>
<th>Slot 5</th>
<th rowspan="2">01:15pm-02:10pm</th>
<th>Slot 6</th>
<th rowspan="2">3:00pm-3:10pm</th>
<th>Slot 7</th>
<th>Slot 8</th>
</tr>
<tr>
<td>8:55-9:45am</td>
<td>9:45-10:35am</td>
<td>10:45-11:35am</td>
<td>11:35-12:25pm</td>
<td>12:25-1:15pm</td>
<td>2:10-3:00pm</td>
<td>3:10-4:00pm</td>
<td>4:00-4:50pm</td>
</tr>
<tr>
<td>Monday</td>
<td>23CSE111 Object Oriented Programming</td>
<td>23CSE113 User Interface Design</td>
<td rowspan="5"style=" writing-mode:vertical-rl;
text-orientation:mixed;
transform: rotate(180deg);
```

```
text-align: center;
font-weight: bold;background-color:yellow;">Short Break</td>
<td colspan="2">23MAT116 Discrete Mathematics_lab_lab3</td>
<td>Library</td>
<td rowspan="5" style=" writing-mode: vertical-rl;
text-orientation: mixed;
transform: rotate(180deg);
text-align: center;
font-weight: bold;background-color:yellow;">Lunch Break</td>
<td>Library</td>
<td rowspan="5" style=" writing-mode: vertical-rl;
text-orientation: mixed;
transform: rotate(180deg);
text-align: center;
font-weight: bold;background-color: yellow;">Short Break</td>
<td colspan="2">Sports</td>
</tr>
<tr>
<td>Tuesday</td>
<td colspan="2">23CSE111 Object Oriented Programming_lab1</td>
<td>23MAT116 Discrete Mathematics</td>
<td colspan="2"> 23MAT117 Linear Algebra_LAB(BYOD)</td>
<td>Library</td>
<td colspan="2">22ADM111 Glimpses of Glorious India</td>
</tr>
<tr>
```

<td>Wednesday</td>

<td> 23PHY115Modern Physics</td>

<td>23MAT117 Linear Algebra</td>

<td> LIBRARY</td>

<td>23PHY115Modern Physics</td>

<td>23CSE111Object Oriented Programming</td>

<td>23MEE115Manufacturing Practice LAB</td>

<td colspan="2">23MEE115Manufacturing Practice LAB</td>

</tr>

<tr>

<td>Thursday</td>

<td> 23CSE113User Interface Design</td>

<td>23MAT117 Linear Algebra</td>

<td colspan="2">23CSE113User Interface Design Lab (BYOD)</td>

<td> 23MAT116Discrete Mathematics</td>

<td style="color: red;">COUNSELLING</td>

<td colspan="2">Library/Sports</td>

</tr>

<tr>

<td>Friday</td>

<td> 23MAT117 Linear Algebra</td>

<td>23PHY115Modern Physics(T)</td>

<td>23CSE111 Object Oriented Programming</td>

<td>23MAT116Discrete Mathematics</td>

<td style="color: red;">COUNSELLING</td>

<td> 22ADM111Glimpses of Glorious India(P)</td>

<td colspan="2">Library/Sports</td>

<tr>

<td>Course Code</td>

<td colspan="3">Course Name</td>

<td>L-T-P</td>

<td>Credits</td>

<td colspan="3">Faculty Name</td>

<td></td>

<td colspan="2">Assisting Faculty</td>

</tr>

<tr>

<td>23MAT116</td>

<td colspan="2">Discrete Mathematics</td>

<td></td>

<td>3-0-2</td>

<td>4</td>

<td colspan="3">Dr.Ram Pratap Chauhan</td>

<td></td>

<td colspan="2"></td>

</tr>

<tr>

<td>23MAT117</td>

<td colspan="2">Linear Algebra</td>

<td></td>

<td>3-0-2</td>

<td>4</td>

<td colspan="3">Dr.Rashmi Prasad</td>

<td></td>

<td colspan="2"></td>

<tr>

<td>23CSE111</td>

<td colspan="2">Object Oriented Programming</td>

<td></td>

<td>3-0-2</td>

<td>4</td>

<td colspan="3">Dr. Balaji TK</td>

<td></td>

<td colspan="2">Dr. Satya Rajendra Singh</td>

</tr>

<tr>

<td>23PHY115</td>

<td colspan="2">Modern Physics</td>

<td></td>

<td>2-1-0</td>

<td>3</td>

<td colspan="3"> Dr. C R Kesavalu</td>

<td></td>

<td colspan="2"></td>

<tr>

<td>23CSE113</td>

<td colspan="2">User Interface Design</td>

<td></td>

<td>2-0-2</td>

<td>3</td>

<td colspan="3">Dr.Rajkumar Batchu</td>

<td></td>

<td colspan="2">Mr. Barge Bharadwaj</td>

</tr>

<tr>

<td>23MEE115</td>

<td colspan="2">Manufacturing Practice</td>

<td></td>

<td>0-0-3</td>

<td>1</td>

<td colspan="3">Dr. Sathies S.</td>

<td></td>

<td colspan="2"></td>

</tr>

<tr>

<td>22ADM111</td>

<td colspan="2">Glimpses of Glorious India</td>

<td></td>

<td>2-0-1</td>

<td>2</td>

<td colspan="3">Dr. Siva Panuganti</td>

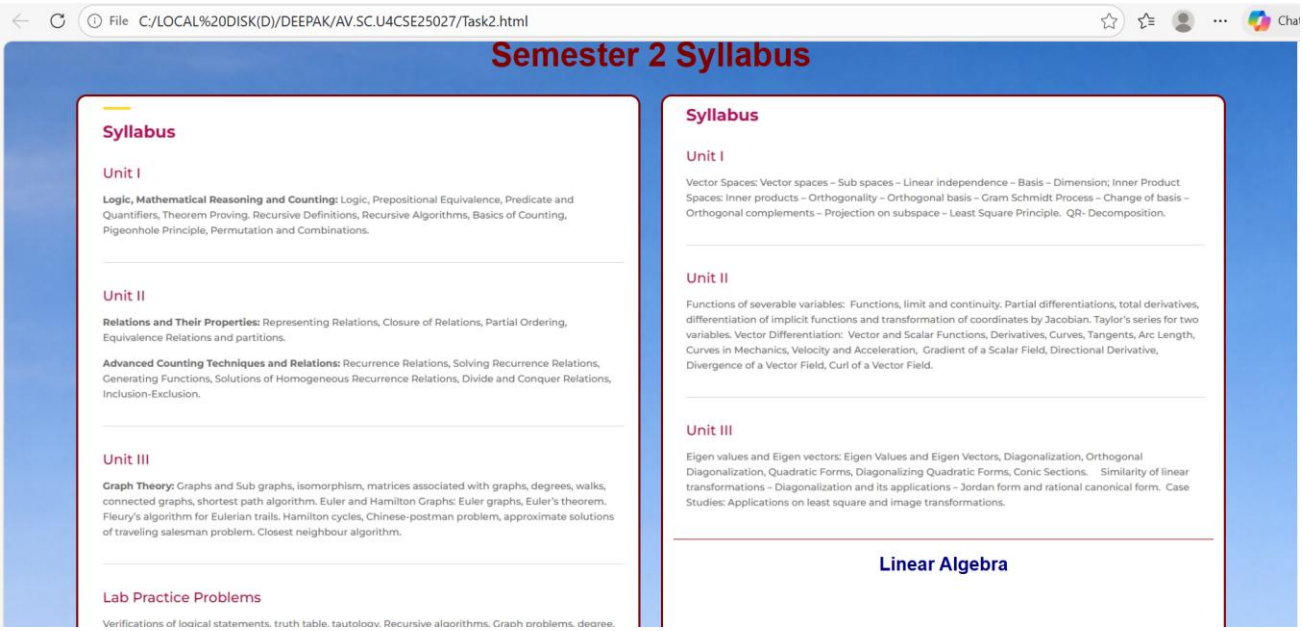
<td></td>

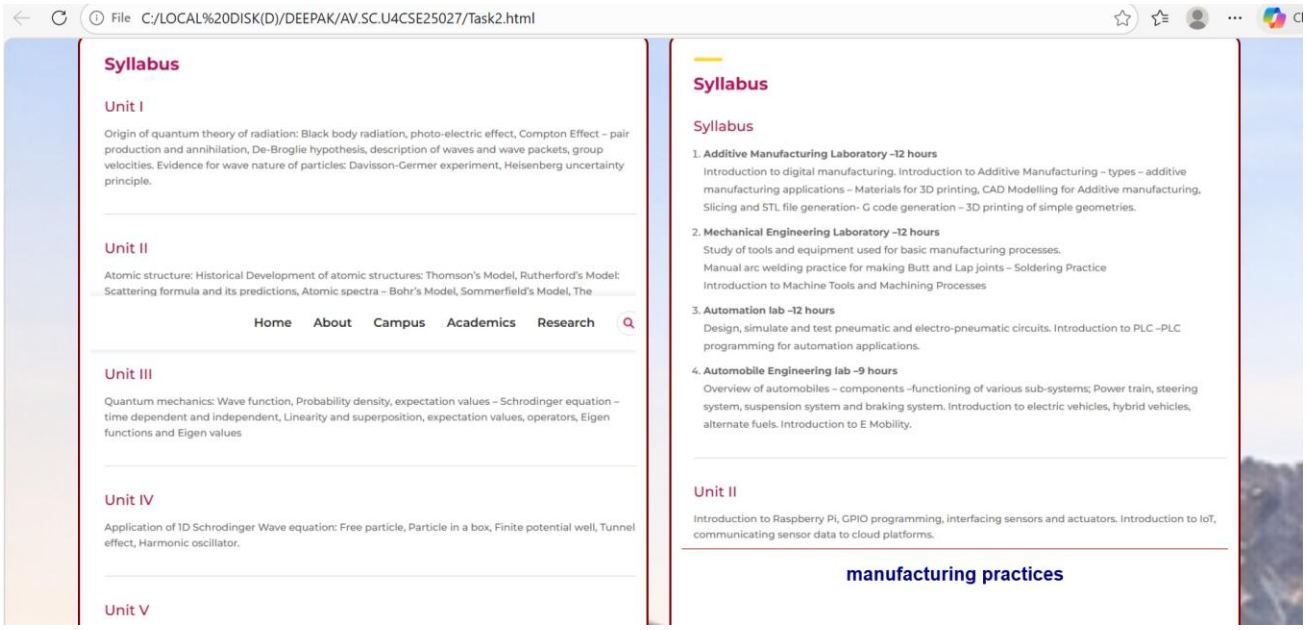
<td colspan="2"></td>

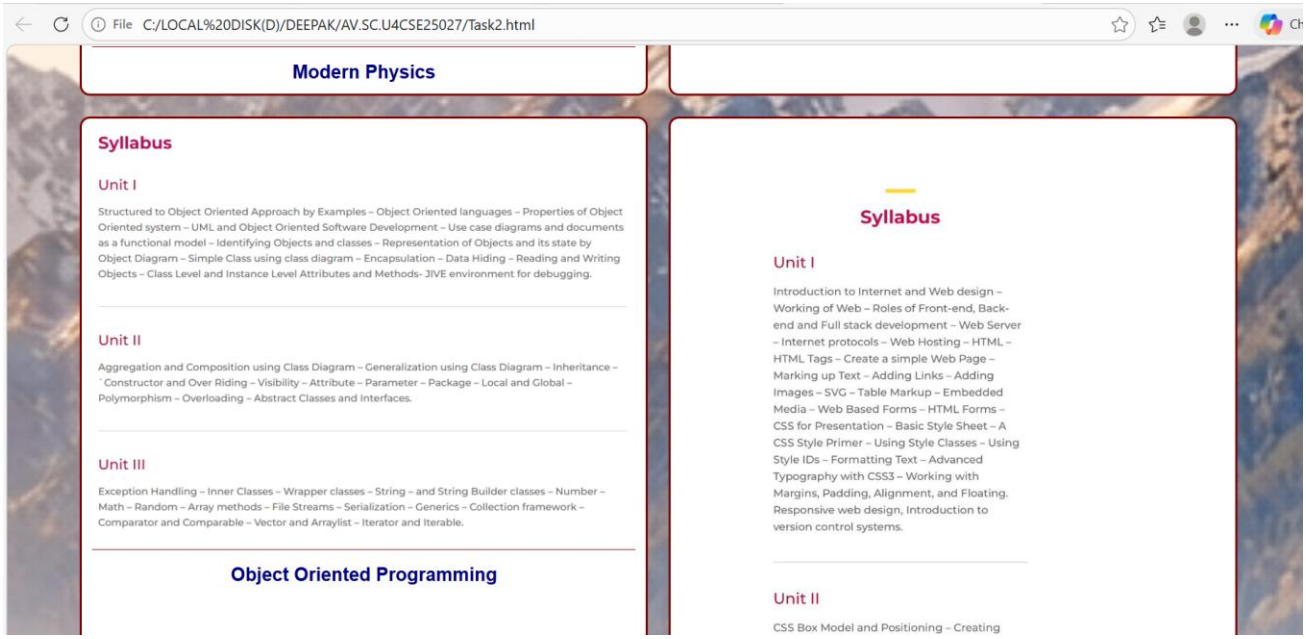
</tr>

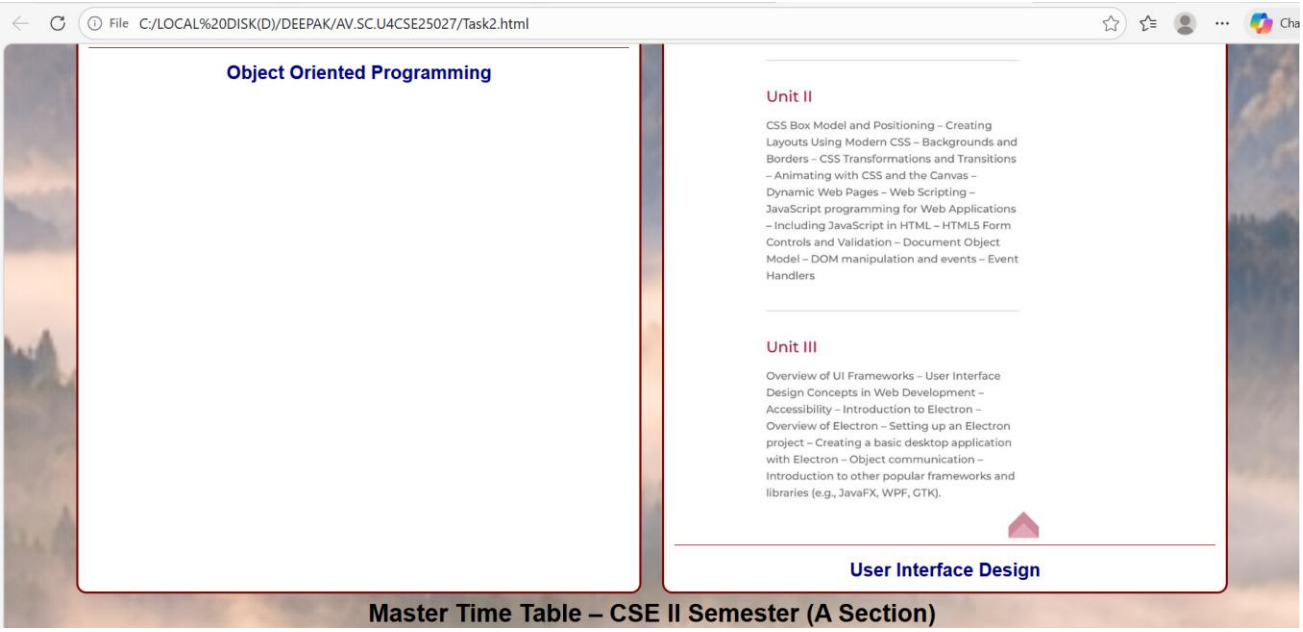
```
<tr>
<td></td>
<td colspan="2">Total (15L + 1Tut +12P =28 hrs)</td>
<td></td>
<td>15-1-12</td>
<td>21</td>
<td colspan="3"></td>
<td></td>
<td colspan="2"></td>
</tr>
</table>
</body>
</html>
```

OUTPUT:




File C:/LOCAL%20DISK(D)/DEEPAK/AV.SC.U4CSE25027/Task2.html
Syllabus
Unit I
Origin of quantum theory of radiation: Black body radiation, photo-electric effect, Compton Effect – pair production and annihilation, De-Broglie hypothesis, description of waves and wave packets, group velocities. Evidence for wave nature of particles: Davisson-Germer experiment, Heisenberg uncertainty principle.
Unit II
Atomic structure: Historical Development of atomic structures: Thomson's Model, Rutherford's Model: Scattering formula and its predictions, Atomic spectra – Bohr's Model, Sommerfield's Model, The
Home About Campus Academics Research
Unit III
Quantum mechanics: Wave function, Probability density, expectation values – Schrodinger equation – time dependent and independent, Linearity and superposition, expectation values, operators, Eigen functions and Eigen values
Unit IV
Application of 1D Schrodinger Wave equation: Free particle, Particle in a box, Finite potential well, Tunnel effect, Harmonic oscillator.
Unit V
Syllabus
Syllabus
1. Additive Manufacturing Laboratory –12 hours
Introduction to digital manufacturing. Introduction to Additive Manufacturing – types – additive manufacturing applications – Materials for 3D printing, CAD Modelling for Additive manufacturing, Slicing and STL file generation- G code generation – 3D printing of simple geometries.
2. Mechanical Engineering Laboratory –12 hours
Study of tools and equipment used for basic manufacturing processes.
Manual arc welding practice for making Butt and Lap joints – Soldering Practice
Introduction to Machine Tools and Machining Processes
3. Automation lab –12 hours
Design, simulate and test pneumatic and electro-pneumatic circuits. Introduction to PLC –PLC programming for automation applications.
4. Automobile Engineering lab –9 hours
Overview of automobiles – components –functioning of various sub-systems; Power train, steering system, suspension system and braking system. Introduction to electric vehicles, hybrid vehicles, alternate fuels. Introduction to E Mobility.
Unit II
Introduction to Raspberry Pi, GPIO programming, interfacing sensors and actuators. Introduction to IoT, communicating sensor data to cloud platforms.
manufacturing practices


File C:/LOCAL%20DISK(D)/DEEPAK/AV.SC.U4CSE25027/Task2.html
Modern Physics
Syllabus
Unit I
Structured to Object Oriented Approach by Examples – Object Oriented languages – Properties of Object Oriented system – UML and Object Oriented Software Development – Use case diagrams and documents as a functional model – Identifying Objects and classes – Representation of Objects and its state by Object Diagram – Simple Class using class diagram – Encapsulation – Data Hiding – Reading and Writing Objects – Class Level and Instance Level Attributes and Methods- JIVE environment for debugging.
Unit II
Aggregation and Composition using Class Diagram – Generalization using Class Diagram – Inheritance – `Constructor and Over Riding – Visibility – Attribute – Parameter – Package – Local and Global – Polymorphism – Overloading – Abstract Classes and Interfaces.
Unit III
Exception Handling – Inner Classes – Wrapper classes – String – and String Builder classes – Number – Math – Random – Array methods – File Streams – Serialization – Generics – Collection framework – Comparator and Comparable – Vector and Arraylist – Iterator and Iterable.
Object Oriented Programming
Syllabus
Unit I
Introduction to Internet and Web design – Working of Web – Roles of Front-end, Back-end and Full stack development – Web Server – Internet protocols – Web Hosting – HTML – HTML Tags – Create a simple Web Page – Marking up Text – Adding Links – Adding Images – SVG – Table Markup – Embedded Media – Web Based Forms – HTML Forms – CSS for Presentation – Basic Style Sheet – A CSS Style Primer – Using Style Classes – Using Style IDs – Formatting Text – Advanced Typography with CSS3 – Working with Margins, Padding, Alignment, and Floating. Responsive web design, Introduction to version control systems.
Unit II
CSS Box Model and Positioning – Creating



File C:/LOCAL%20DISK(D)/DEEPAK/AV.SC.U4CSE25027/class_timetable.html

CSE-A Class Timetable

Master Time Table CSE II Semester A Section

Dept.CSE
Class: B. Tech. CSE-A
Semester-II
Class Room Venue: 510

Class Advisors:
1. Name: Dr. K. Sindhu
Email : k_sindhu@av.amrita.edu
Mobile Number : +91-9969387057

2. Name: Dr. Sumanta Kumar Nanda
Email : n_sumantakumar@av.amrita.edu
Mobile Number :+91- 7978115993

Time/Day	Slot 1	Slot 2	10:35am-10:45am	Slot 3	Slot 4	Slot 5	01:15pm-02:10pm	Slot 6	3:00pm-3:10pm	Slot 7	Slot 8
	8:55-9:45am	9:45-10:35am		10:45-11:35am	11:35-12:25pm	12:25-1:15pm		2:10-3:00pm		3:10-4:00pm	4:00-4:50pm
Monday	23CSE111 Object Oriented Programming	23CSE113 User Interface Design	Short Break	23MAT116 Discrete Mathematics_lab_lab3		Library	Lunch Break	Library	Short Break	Sports	
Tuesday	23CSE111 Object Oriented Programming_lab1			23MAT116 Discrete Mathematics	23MAT117 Linear Algebra_LAB(BYOD)			Library		22ADM111 Glimpses of Glorious India	
Wednesday	23PHY115Modern Physics	23MAT117 Linear Algebra		LIBRARY	23PHY115Modern Physics	23CSE111Object Oriented Programming		23MEE115Manufacturing Practice LAB		23MEE115Manufacturing Practice LAB	
Thursday	23CSE113User Interface Design	23MAT117 Linear Algebra		23CSE113User Interface Design Lab (BYOD)		23MAT116Discrete Mathematics		COUNSELLING		Library/Sports	
Friday	23MAT117 Linear Algebra	23PHY115Modern Physics(T)		23CSE111 Object Oriented Programming	23MAT116Discrete Mathematics	COUNSELLING		22ADM111Glimpses of Glorious India(P)		Library/Sports	

Course Code	Course Name	L-T-P	Credits	Faculty Name	Assisting Faculty
23MAT116	Discrete Mathematics	3-0-2	4	Dr.Ram Pratap Chauhan	
23MAT117	Linear Algebra	3-0-2	4	Dr.Rashmi Prasad	
23CSE111	Object Oriented Programming	3-0-2	4	Dr. Balaji TK	Dr. Satya Rajendra Singh
23PHY115	Modern Physics	2-1-0	3	Dr. C R Kesavalu	
23CSE113	User Interface Design	2-0-2	3	Dr.Rajkumar Batchu	Mr. Barge Bharadwaj
23MEE115	Manufacturing Practice	0-0-3	1	Dr. Sathies S.	
22ADM111	Glimpses of Glorious India	2-0-1	2	Dr. Siva Panuganti	
	Total (15L + 1Tut +12P =28 hrs)	15-1-12	21		

WEEK-4

4a. AIM: Understanding some HTML Tags

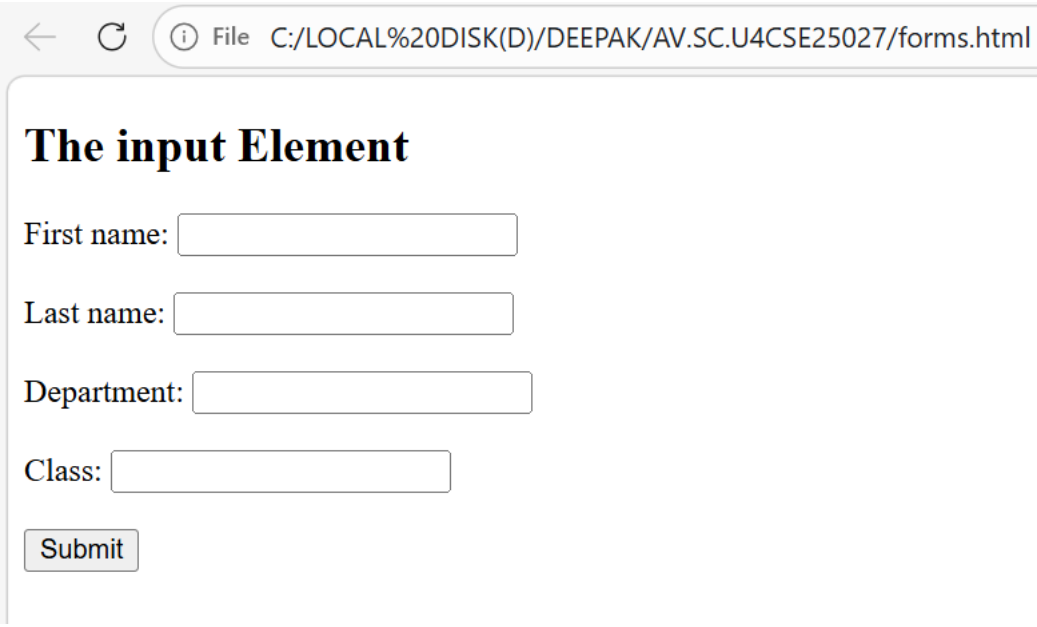
Using Forms:

PROGRAM:

```
<!DOCTYPE html>
```

```
<html>
<body>
<h2>The input Element</h2>
<form action="/action_page.php">
<label for="fname">First name:</label>
<input type="text" id="fname" name="fname"><br><br>
<label for="lname">Last name:</label>
<input type="text" id="lname" name="lname"><br><br>
<label for="department">Department:</label>
<input type="text" id="department" name="department"><br><br>
<label for="class">Class:</label>
<input type="text" id="class" name="class"><br><br>
<input type="submit" value="Submit">
</form>
</body>
</html>
```

OUTPUT:

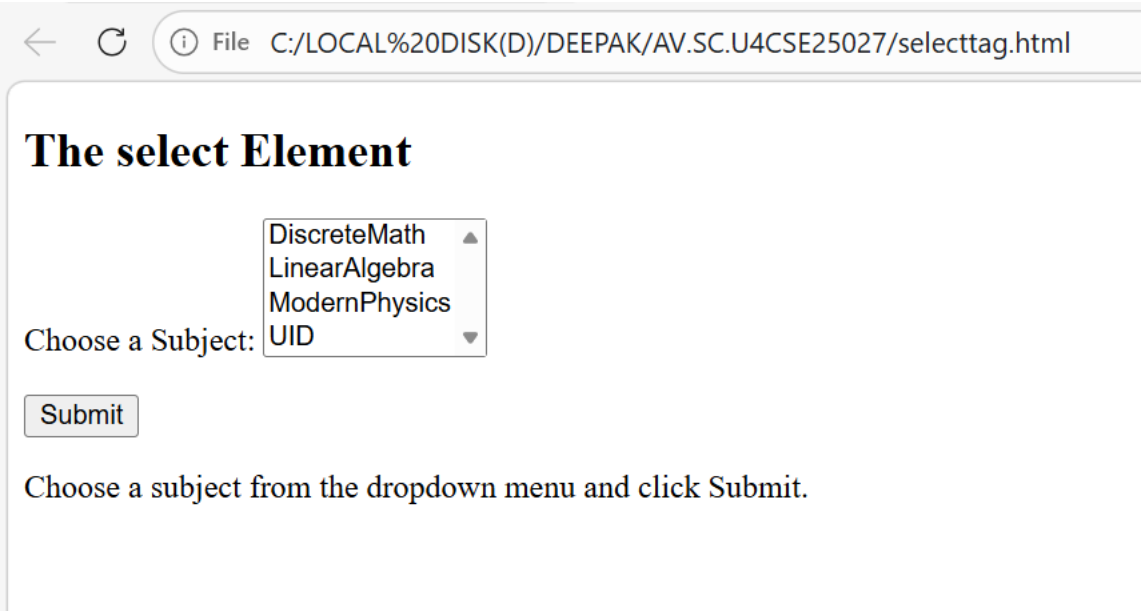


Using SelectTag:

PROGRAM:

```
<!DOCTYPE html>
<html>
<head>
<title>Select Tag Example</title>
</head>
<body>
<h2>The select Element</h2>
<form action="/action_page.php">
<label for="Subjects">Choose a Subject:</label>
<select id="Subjects" name="Subjects" size="4">
<option value="discrete">DiscreteMath</option>
<option value="linear">LinearAlgebra</option>
<option value="modern">ModernPhysics</option>
<option value="uid">UID</option>
</select><br><br>
<input type="submit" value="Submit">
</form>
<p>Choose a subject from the dropdown menu and click Submit.</p>
</html>
```

OUTPUT:



EXPLANATION OF TAGS:

1.Core Form Elements;

- **<form>**: The wrapper for all your input elements. It defines where the data should be sent (via the action attribute) and how it should be sent (usually GET or POST).
- **<label>**: Crucial for accessibility and user experience. It provides a text description for an input. Clicking a label also focuses the associated input field, making it easier to select small targets like checkboxes.

2.Text and Data Entry;

<input>: The most versatile element. By changing the type attribute, you can create various data fields.

- **type="date"**: Opens a native date picker, ensuring users provide a valid date format without manual typing.
- **type="file"**: Allows users to select one or more files from their device storage to upload.

<textarea>: Unlike a standard input, this allows for **multi-line text**. It's perfect for comments, bios, or feedback where a single line isn't enough.

3.Choices and Selections;

- **<input type="radio">**: Used when you want the user to select **exactly one** option from a set (e.g., choosing a shipping method). All buttons in a group must share the same name.
- **<input type="checkbox">**: Used for "yes/no" choices or when a user can select **multiple options** from a list (e.g., "Select your interests").

- **<select> and <option>**: Creates a **dropdown menu**. The <select> tag acts as the container, and each <option> defines a choice within that list. This saves space compared to listing out several radio buttons.

4.Submitting the Data;

- <button type="submit"> (or <input type="submit">): This is the "Go" button. It triggers the form's action and sends all the gathered data to the server.
- <button type="reset">: A "clear" button. It restores all fields in the form to their original default values. Use this sparingly, as users often click it by accident.

4b. AIM: Construct a web page that displays two different forms side by side using iframes.

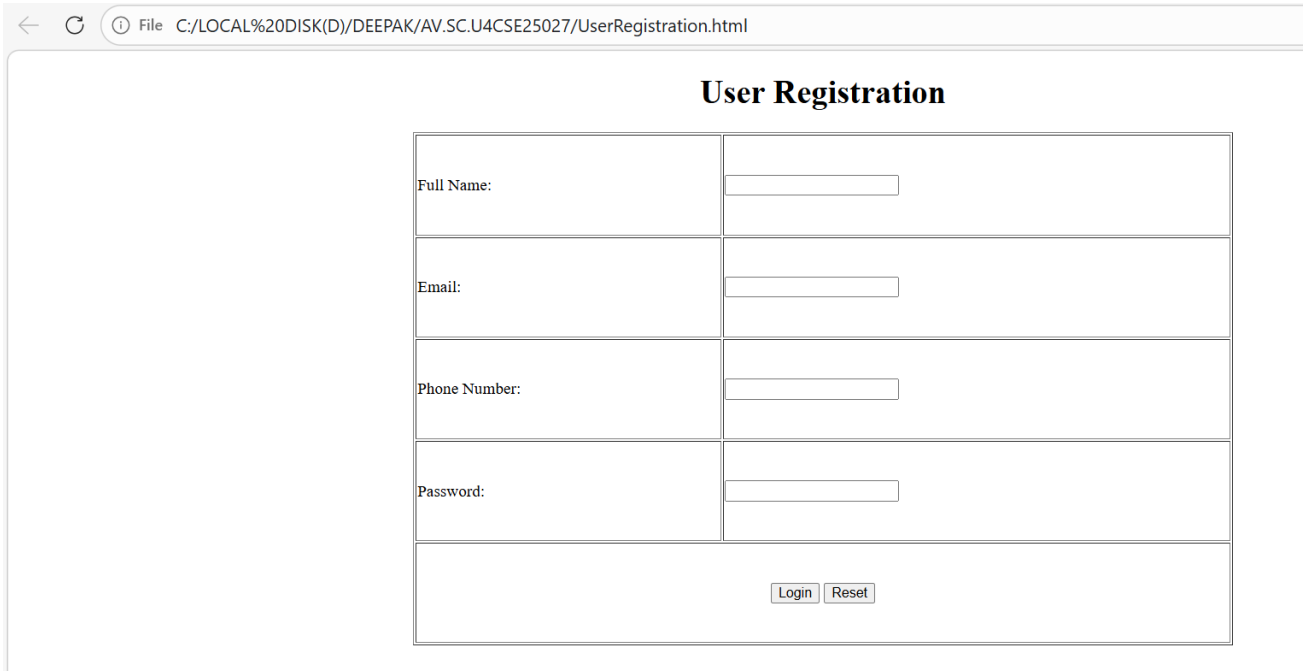
PROGRAM:

1) User Registration Form:

```
<!DOCTYPE html>
<html>
  <head>
    <title style="text-align: center;"> User Registration</title>
  </head>
  <body>
    <h1 style="text-align: center;">User Registration</h1>
    <form>
      <table border="1" style="margin: auto;" height="500px" width="800px">
        <tr>
          <td>Full Name:</td>
          <td><input type="text" name="fullname"></td>
        </tr>
        <tr>
          <td>Email:</td>
          <td><input type="email" name="email"></td>
        </tr>
        <tr>
```

```
    <td>Phone Number:</td>
    <td><input type="number" name="phonenumber"></td>
</tr>
<tr>
    <td>Password:</td>
    <td><input type="password" name="password"></td>
</tr>
<tr>
<td colspan="2" style="text-align: center;"><input type="submit" value="Login">
    <input type="reset" value="Reset"></td>
    </tr>
    </table>
    </form>
    </body>
    </html>
```

OUTPUT:

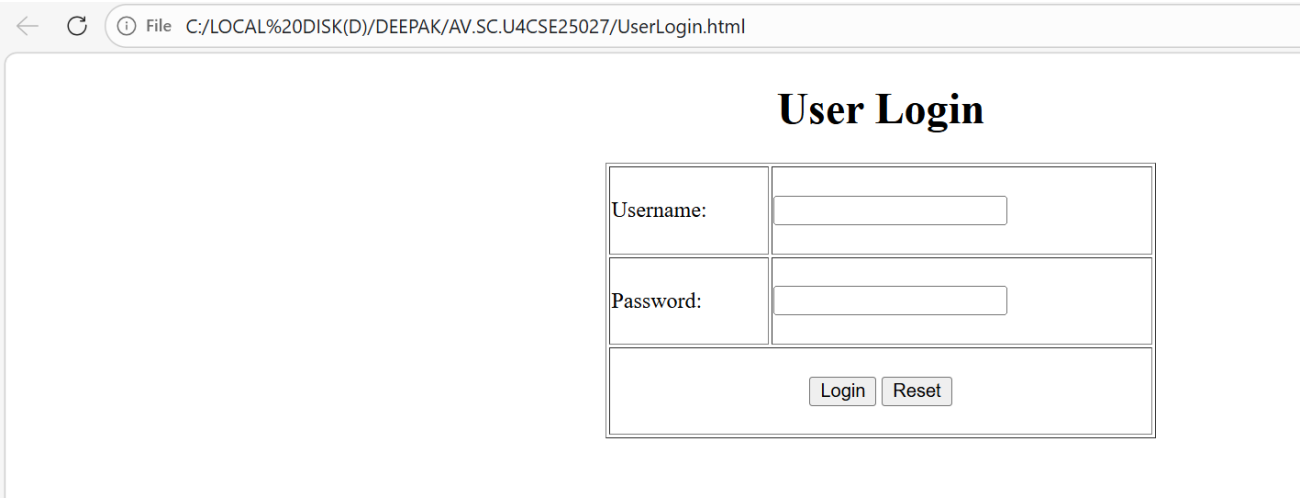


2)User Login Form:

PROGRAM:

```
<!DOCTYPE html>
<html>
  <head>
    <title style="text-align: center;"> User Login</title>
  </head>
  <body>
    <h1 style="text-align: center;">User Login</h1>
    <form>
      <table border="1" style="margin: auto;" height="200px" width="400px">
        <tr>
          <td>Username:</td>
          <td><input type="text" name="username"></td>
        </tr>
        <tr>
          <td>Password:</td>
          <td><input type="password" name="password"></td>
        </tr>
        <tr>
          <td colspan="2" style="text-align: center;"><input type="submit"
value="Login">
          <input type="reset" value="Reset"></td>
      </table>
    </form>
  </body>
```

OUTPUT:

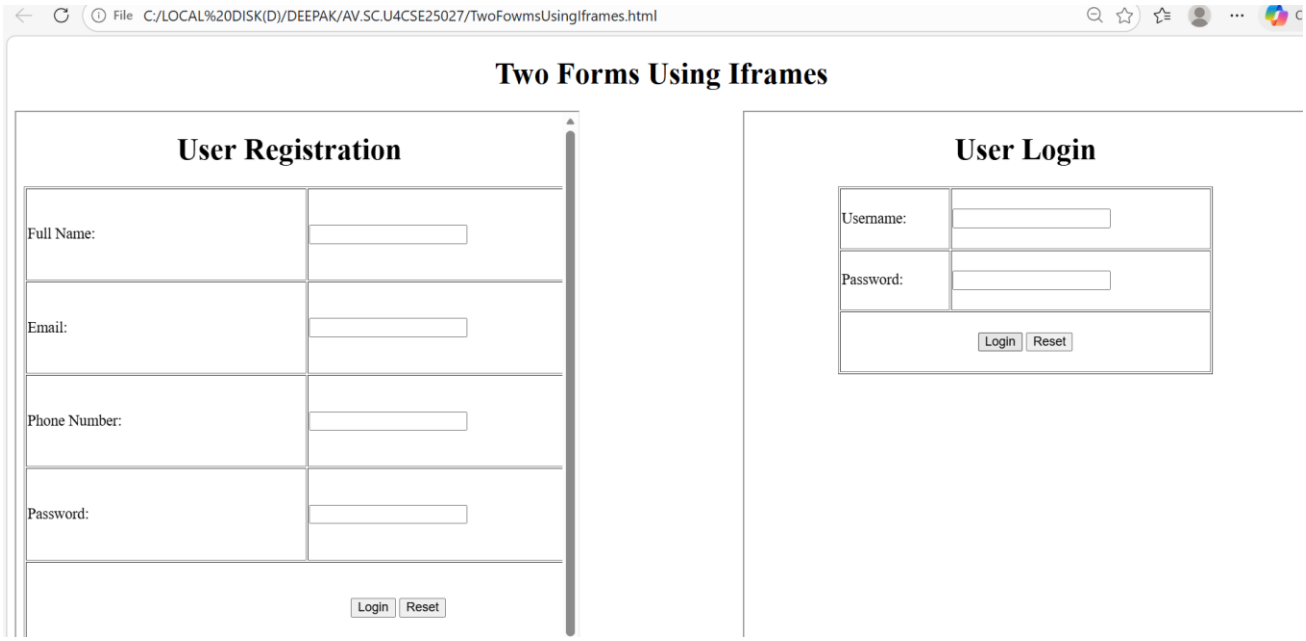


3)Two Forms(using iframes):

PROGRAM:

```
<!DOCTYPE html>
<html>
  <head>
    <title style="text-align: center;"> Two Forms Using Iframes</title>
  </head>
  <body>
    <h1 style="text-align: center;">Two Forms Using Iframes</h1>
    <iframe src="UserRegistration.html" width="600px" height="600px" style="float: left;"></iframe>
    <iframe src="UserLogin.html" width="600px" height="600px" style="float: right;"></iframe>
  </body>
```

OUTPUT:



EXPLANATION OF TAGS:

1. The (Division) Element;

Think of a as an **invisible box**. On its own, it doesn't do anything or change the look of the page. It is a "block-level" container used to group related elements together so you can style them with CSS or manipulate them with JavaScript.

- **In Forms:** You'll often wrap a <label> and an <input> inside a to keep them together and create spacing between different rows of the form.

- **Organization:** It helps break a webpage into logical sections like a header, sidebar, or gallery.

<div style = “display: flex; gap: 20px; flex-wrap; wrap;”>

Display: Activates flex box layout places child elements in a row make alignments and spacing easier.

Gap: Adds 20px space between the iframes, cleaner than using margins.

Wrap: Allows items to move to the next line important for responsiveness if screen is small-second iframes moves below first.

2.The <iframe> (Inline Frame) Element;

An <iframe> is essentially a window into another website. It allows you to embed an entire HTML document inside your current page.

- Common Uses:
    - Embedding a YouTube video.
    - Displaying a Google Map.
    - Showing an external advertisement or a social media feed.
- Security Note: Because iframes pull in content from other sources, browsers have strict security rules (like "Sandboxing") to prevent the embedded site from acting maliciously on your main page.

<iframe Style = “min width:300px; flex:1;”>

Min width: Prevents iframe from shrinking below 300px, makes layout responsive.

If screen becomes small, items wrap to next line.

Flex: Both iframe share space equally, Each takes equal width in the row.

WEEK-5

5(A) Aim: To create a responsive card layout using HTML and CSS with three cards, each containing an image, title, and button.

PROGRAM:

<!doctype html>

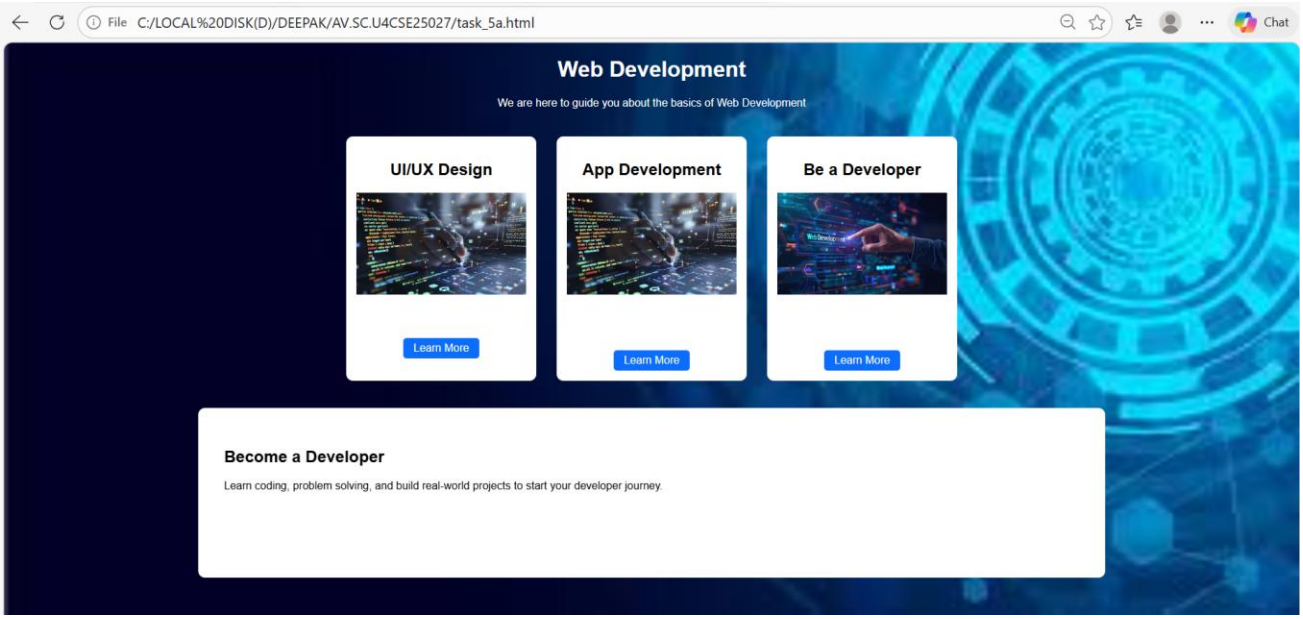
```
<html>
<head>
<title>Web Development</title>

<style>
body{
    margin:0;
    font-family:Arial;
    background-image: url('background_5a.jpeg');
    background-size: cover;
    background-repeat: no-repeat;
    background-position: center;
    text-align:center;
}

h1,p{
    color:white;
}

.container{
    display:flex;
    justify-content:center;
    gap:30px;
    margin-top:40px;
}

.card{
    background:white;
    width:250px;
    padding:15px;
```

```
    border-radius:10px;
}

.card img{
    width:100%;
    height:150px;
    object-fit:cover;
}

button{
    margin-top:10px;
    padding:6px 15px;
    background:#0d6efd;
    color:white;
    border:none;
    border-radius:5px;
    cursor:pointer;
    font-size:16px;
}

button:hover {
    background:#0b5ed7;
}

a {
    text-decoration: none;
}

iframe{
    margin-top:40px;
```

```
    width:70%;
    height:250px;
    border:none;
    border-radius:10px;
    background:white;
}
</style>

</head>

<body>

<h1>Web Development</h1>
<p>We are here to guide you about the basics of Web Development</p>

<div class="container">

    <div class="card">
        <h2>UI/UX Design</h2>
        <img src="card1.jpeg">
        <p>Explore UI/UX Design deeply.</p>
        <a href="uiux.html" target="infoFrame">
            <button>Learn More</button>
        </a>
    </div>

    <div class="card">
        <h2>App Development</h2>
        <img src="card2.jpeg">
        <p>Learn to create mobile applications.</p>
```

```
        <a href="app.html" target="infoFrame">
            <button>Learn More</button>
        </a>
    </div>

    <div class="card">
        <h2>Be a Developer</h2>
        <img src="card3.jpeg">
        <p>Start your development journey today.</p>
        <a href="dev.html" target="infoFrame">
            <button>Learn More</button>
        </a>
    </div>

</div>

<iframe name="infoFrame"></iframe>

</body>
</html>
```

OUTPUT:



5(B) Aim: To design a mini website using iframe with navigation links (Home, About, Contact), where clicking on each link displays the respective content within the same page using internal or external CSS.

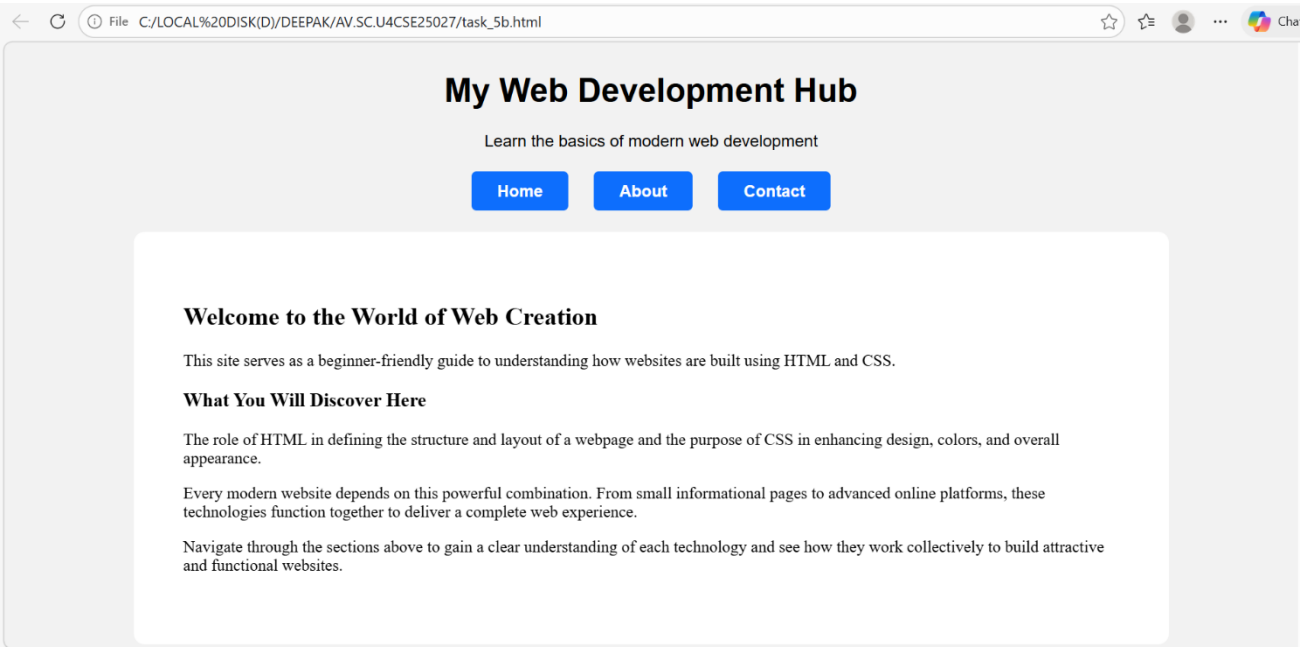
PROGRAM:

```
<!doctype html>
<html>
<head>
   <title>My Web Hub</title>
   <style>
      body{
         margin:0;
         font-family: Arial;
         text-align:center;
         background-color:#f2f2f2;
      }

      h1{
         margin-top:30px;
```

```
        }

        .buttons{
            margin:30px 0;
        }

        .buttons a{
            text-decoration:none;
            background-color:#0d6efd;
            color:white;
            padding:10px 25px;
            margin:10px;
            border-radius:5px;
        }

        .buttons a:hover{
            background-color:#084298;
        }

        iframe{
            width:80%;
            height:400px;
            border:none;
            background:white;
            border-radius:10px;
        }
    </style>
</head>

<body>
```

```
<h1>My Web Development Hub</h1>
<p>Learn the basics of modern web development</p>

<div class="buttons">
    <a href="task_5b_home.html" target="content"><b>Home</a>
    <a href="task_5b_about.html" target="content">About</a>
    <a href="task_5b_contact.html" target="content">Contact</a>
</div>

<iframe name="content"></iframe>

</body>
</html>
```

OUTPUT:



5(C) Aim: To design an Amrita login page using HTML and CSS that displays different campuses, where clicking on each campus shows its details on the same page.

PROGRAM:

```
<!doctype html>
<html>
<head>
<title>Amrita Campus Login</title>

<style>
body{
   margin:0;
   font-family:Arial;
   background-color:#a9445b;
}

.header{
   background:#7a1f33;
   color:white;
   padding:25px;
   text-align:center;
}

.campus-section{
   padding:40px;
   display:grid;
   grid-template-columns: repeat(3, 1fr);
   gap:25px;
}
```

```
.card{
    border-radius:10px;
    overflow:hidden;
}

.card img{
    width:100%;
    height:200px;
    object-fit:cover;
}

.card a{
    text-decoration:none;
    color:white;
}

.overlay{
    position:relative;
    margin-top:-40px;
    padding:10px;
    text-align:center;
    font-weight:bold;
}

iframe{
    width:80%;
    height:300px;
    border:none;
    border-radius:10px;
    background:white;
```

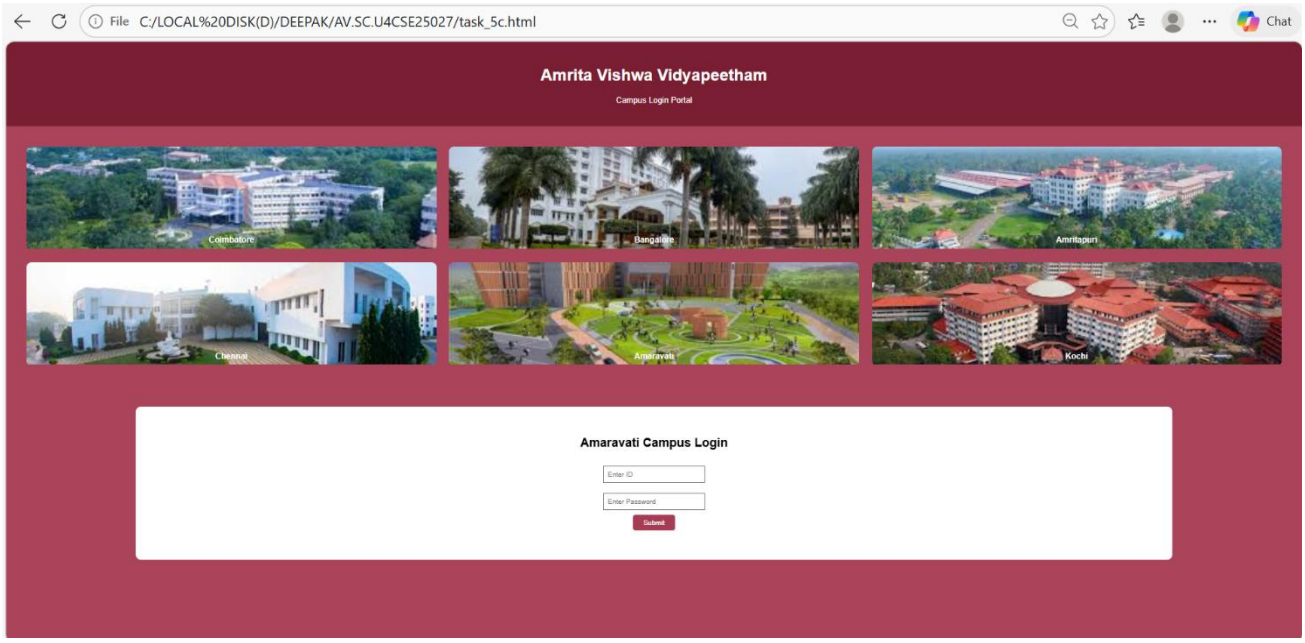
```
    margin:40px auto;
    display:block;
}
</style>
</head>

<body>

<div class="header">
    <h1>Amrita Vishwa Vidyapeetham</h1>
    <p>Campus Login Portal</p>
</div>

<div class="campus-section">

    <div class="card">
        <a href="coimbatore-login.html" target="loginFrame">
            <img src="coimbatore.jpeg">
            <div class="overlay">Coimbatore</div>
        </a>
    </div>

    <div class="card">
        <a href="bangalore-login.html" target="loginFrame">
            <img src="bangalore.jpeg">
            <div class="overlay">Bangalore</div>
        </a>
    </div>

    <div class="card">
```

```
        <a href="amritapuri-login.html" target="loginFrame">
            <img src="amritapuri.jpeg">
            <div class="overlay">Amritapuri</div>
        </a>
    </div>

    <div class="card">
        <a href="chennai-login.html" target="loginFrame">
            <img src="chennai.jpeg">
            <div class="overlay">Chennai</div>
        </a>
    </div>

    <div class="card">
        <a href="amaravati-login.html" target="loginFrame">
            <img src="amaravati.jpeg">
            <div class="overlay">Amaravati</div>
        </a>
    </div>

    <div class="card">
        <a href="kochi-login.html" target="loginFrame">
            <img src="kochi.jpeg">
            <div class="overlay">Kochi</div>
        </a>
    </div>

</div>

<iframe name="loginFrame"></iframe>
```

</body>

</html>

OUTPUT:



WEEK-6

AIM(6(A)):-To design a e-commerce application using the grid layout , this application should contain different categories of products such as electronics,clothes, home appliances , based on the category we select products should be displayed on the side bar?

PROGRAM:

<!DOCTYPE html>

<html>

<head>

<title>Amrita Sales</title>

<h1>Diwali Sale - Up to 50% Off!</h1>

<style>

body{

```
font-family: Arial;
margin:0;
}

header{
background:#c61d2d;
color:white;
padding:15px;
font-size:22px;
}

.container{
display:flex;
}

x
.sidebar{
width:220px;
background:#c61d2d;
color:white;
padding:20px;
height:500px;
}

.sale-banner{
background:#ffe5e5;
color:#c61d2d;
text-align:center;
padding:10px;
```

```
font-size:20px;
font-weight:bold;
}

.sidebar label{
display:block;
margin:10px 0;
cursor:pointer;
}

input{
display:none;
}

.products{
display:grid;
grid-template-columns:repeat(2,1fr);
gap:20px;
padding:20px;
flex:1;
}

.card{
border:1px solid #ddd;
padding:10px;
background:#f8f8f8;
```

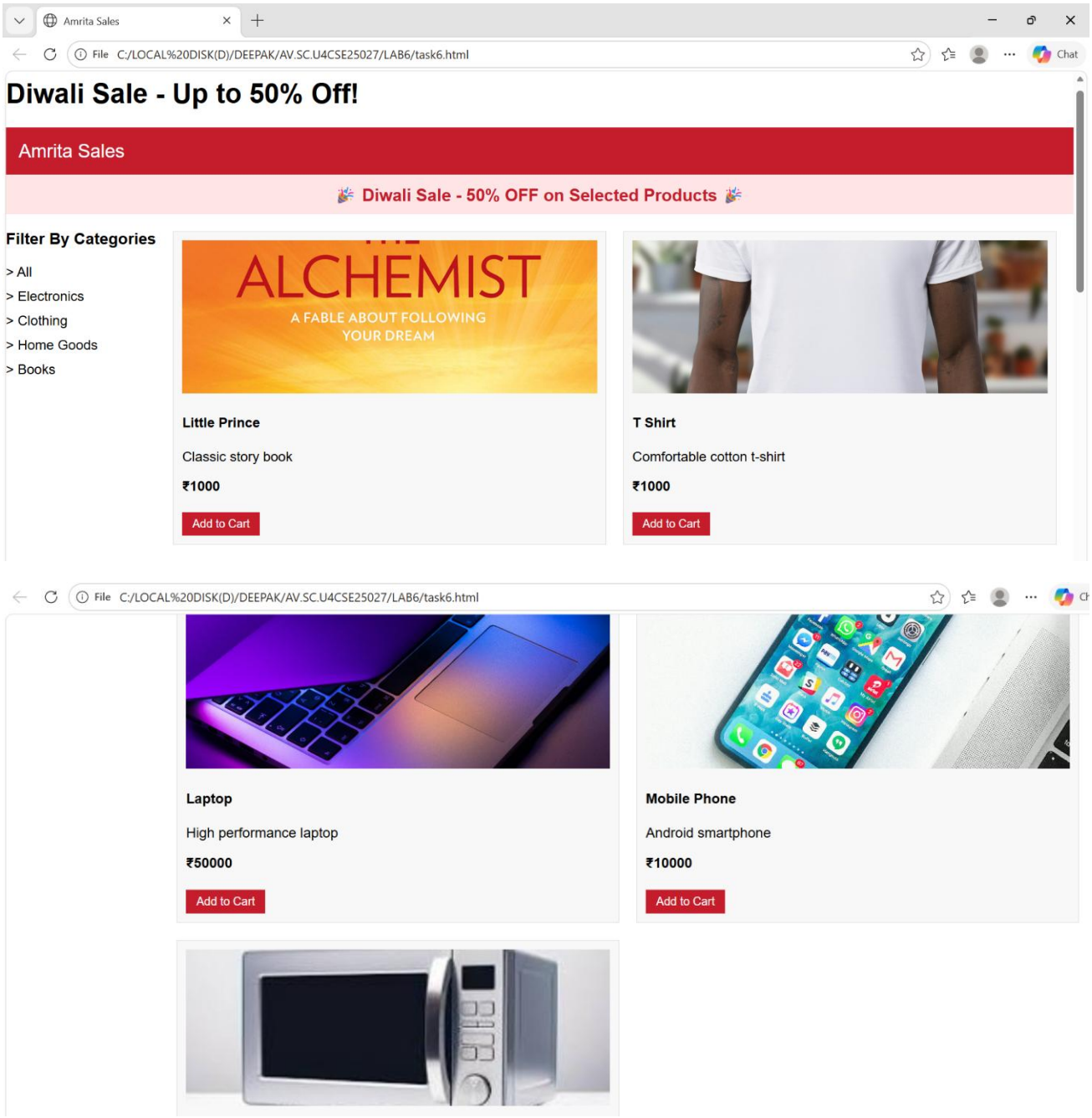
```
}

.card img{
width:100%;
height:180px;
object-fit:cover;
}

.price{
font-weight:bold;
margin-top:5px;
}

button{
background:#c61d2d;
color:white;
border:none;
padding:6px 12px;
margin-top:5px;
cursor:pointer;
}

/* Hide all products by default, show only the checked category */
.products .card {
  display: none;
}

#all:checked ~ .container .card,
#electronics:checked ~ .container .electronics,
```

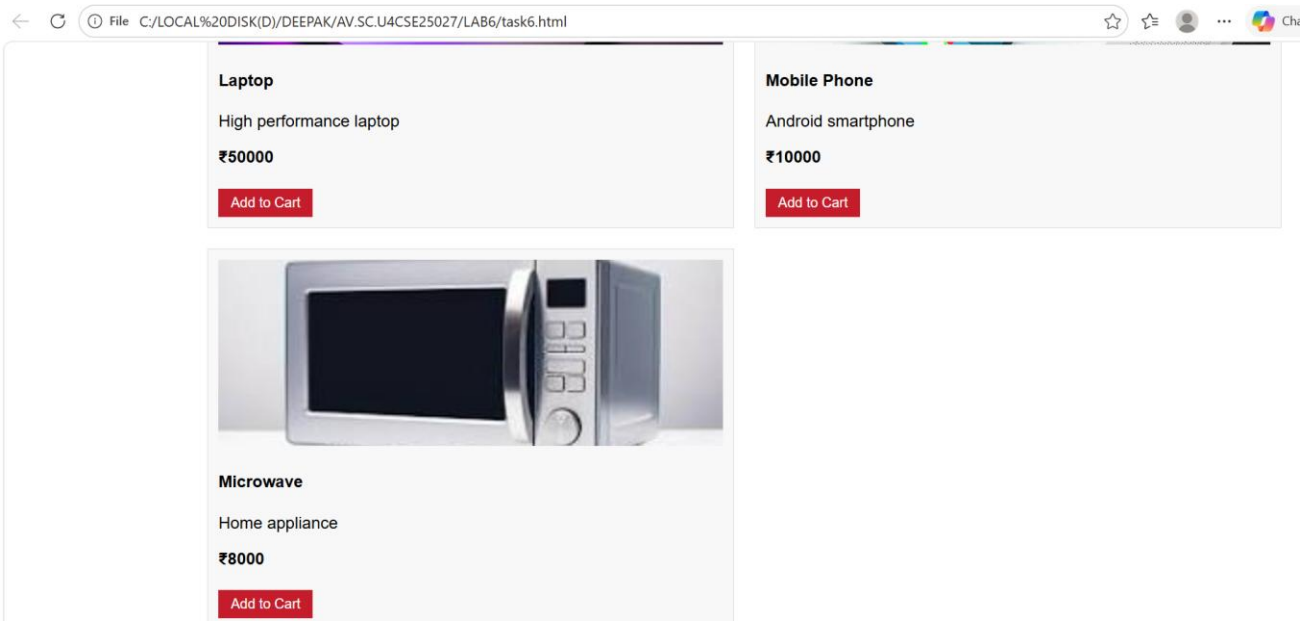
```
#clothing:checked ~ .container .clothing,
#books:checked ~ .container .books,
#home:checked ~ .container .home {
  display: block;
}

</style>

</head>

<body>

<header>Amrita Sales</header>

<input type="radio" name="cat" id="all" checked>
<input type="radio" name="cat" id="electronics">
<input type="radio" name="cat" id="clothing">
<input type="radio" name="cat" id="home">
<input type="radio" name="cat" id="books">
<div class="sale-banner">
🎉 Diwali Sale - 50% OFF on Selected Products 🎉
</div>
<div class="container">

<div class="sidebar">

<h3>Filter By Categories</h3>

<label for="all">> All</label>
```

```
<label for="electronics">> Electronics</label>
<label for="clothing">> Clothing</label>
<label for="home">> Home Goods</label>
<label for="books">> Books</label>

</div>

<div class="products">

<div class="card books">
<img src="https://m.media-amazon.com/images/I/71aFt4+OTOL.jpg">
<h4>Little Prince</h4>
<p>Classic story book</p>
<p class="price">₹1000</p>
<button>Add to Cart</button>
</div>

<div class="card clothing">
<img src="https://images.unsplash.com/photo-1521572163474-6864f9cf17ab">
<h4>T Shirt</h4>
<p>Comfortable cotton t-shirt</p>
<p class="price">₹1000</p>
<button>Add to Cart</button>
</div>

<div class="card electronics">
<img src="https://images.unsplash.com/photo-1517336714731-489689fd1ca8">
<h4>Laptop</h4>
<p>High performance laptop</p>
```

```
<p class="price">₹50000</p>
<button>Add to Cart</button>
</div>

<div class="card electronics">
<img src="https://images.unsplash.com/photo-1511707171634-5f897ff02aa9">
<h4>Mobile Phone</h4>
<p>Android smartphone</p>
<p class="price">₹10000</p>
<button>Add to Cart</button>
</div>

<div class="card home">
<img src="https://images.unsplash.com/photo-1586201375761-83865001e31c">
<h4>Microwave</h4>
<p>Home appliance</p>
<p class="price">₹8000</p>
<button>Add to Cart</button>
</div>

</div>

</div>

</body>
</html>
```

OUTPUT:


Amrita Sales
File C:/LOCAL%20DISK(D)/DEEPAK/AV.SC.U4CSE25027/LAB6/task6.html
Chat
Diwali Sale - Up to 50% Off!
Amrita Sales
Diwali Sale - 50% OFF on Selected Products
Filter By Categories
> All
> Electronics
> Clothing
> Home Goods
> Books
ALCHEMIST
A FABLE ABOUT FOLLOWING YOUR DREAM
Little Prince
Classic story book
₹1000
Add to Cart
T Shirt
Comfortable cotton t-shirt
₹1000
Add to Cart
File C:/LOCAL%20DISK(D)/DEEPAK/AV.SC.U4CSE25027/LAB6/task6.html
Laptop
High performance laptop
₹50000
Add to Cart
Mobile Phone
Android smartphone
₹10000
Add to Cart



AIM(6b). Design a responsive E-Commerce application using the grid layout. This application should different categories of products such as electronics, clothing, home-appliances, books. Those should display on products.

PROGRAM:

```
<!DOCTYPE html>
<html>
  <head>
    <title>E-Commerce Store</title>
    <style>
      body {
       margin: auto;
       font-family: 'Times New Roman', Times, serif;
       background-color: white;
      }
      .container {
        display: grid;
        grid-template-columns: auto auto 1fr auto;
        grid-template-rows: 80px 100px 1fr 100px;
        grid-template-areas:
```

```
    "header header header header"
    "sale sale sale sale"
    "sidebar products products products"
    "footer footer footer footer";
    gap: 20px;
    padding: 20px;
}
.header {
    grid-area: header;
    background-color: #a63d55;
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 24px;
}
.sale {
    grid-area: sale;
    background-color: #7a1f33;
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 20px;
}
.sidebar {
    grid-area: sidebar;
    background-color: #a9445b;
    color: white;
```

```
    padding: 15px;
}
.products {
    grid-area: products;
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 20px;
}

.product-card {
    background-color: white;
    border-radius: 10px;
    padding: 15px;
    text-align: center;
    transition: transform 0.2s ease, box-shadow 0.2s ease;
}
.product-card:hover {
    transform: translateY(-4px);
    box-shadow: 0 8px 18px rgba(0, 0, 0, 0.12);
}
.product-card img {
    width: 100%;
    height: 150px;
    object-fit: cover;
    border-radius: 10px;
}
.product-card h3 {
    margin: 10px 0;
}
```

```
.product-card p {
    color: #a63d55;
    font-weight: bold;
}
.category-input {
    display: none;
}
.category-button {
    display: block;
    width: 100%;
    padding: 10px 15px;
    margin: 10px 0;
    background-color: #7a1f33;
    color: white;
    border: none;
    border-radius: 5px;
    font-size: 14px;
    cursor: pointer;
    font-family: 'Times New Roman', Times, serif;
    text-align: center;
}
.category-button:hover {
    background-color: #5a1523;
}
#cat-all:checked ~ .sidebar label[for="cat-all"],
#cat-electronics:checked ~ .sidebar label[for="cat-electronics"],
#cat-fashion:checked ~ .sidebar label[for="cat-fashion"],
#cat-home-garden:checked ~ .sidebar label[for="cat-home-garden"],
#cat-sports-outdoors:checked ~ .sidebar label[for="cat-sports-outdoors"],
```

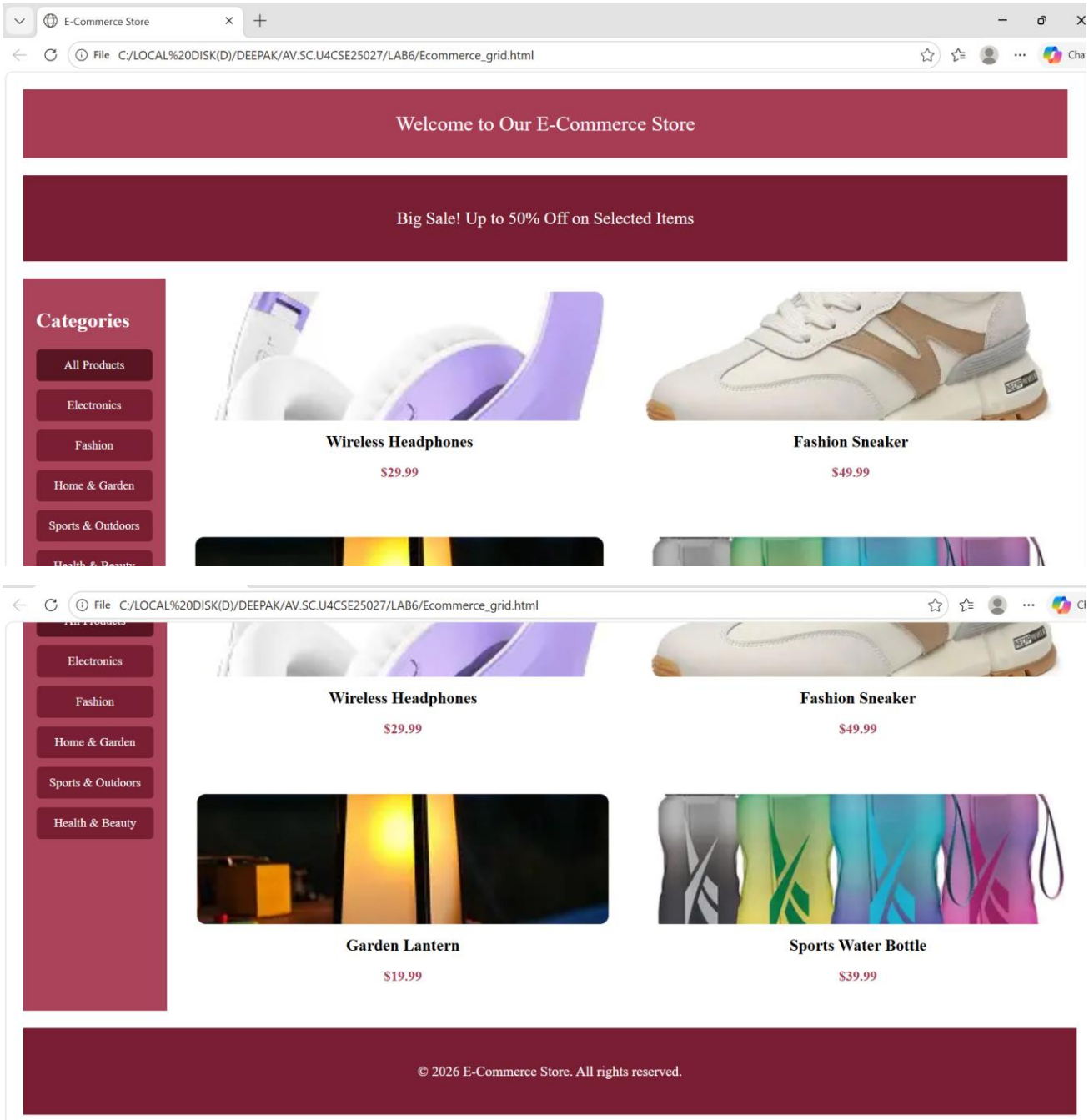
```
#cat-health-beauty:checked ~ .sidebar label[for="cat-health-beauty"] {
    background-color: #5a1523;
}
.products .product-card {
    display: none;
}
#cat-all:checked ~ .products .product-card {
    display: block;
}
#cat-electronics:checked ~ .products .product-card[data-category="electronics"],
#cat-fashion:checked ~ .products .product-card[data-category="fashion"],
#cat-home-garden:checked ~ .products .product-card[data-category="home-garden"],
#cat-sports-outdoors:checked ~ .products .product-card[data-category="sports-outdoors"],
#cat-health-beauty:checked ~ .products .product-card[data-category="health-beauty"] {
    display: block;
}
.sale::after {
    content: "Big Sale! Up to 50% Off on Selected Items";
}
#cat-electronics:checked ~ .sale::after {
    content: "Showing electronics products";
}
#cat-fashion:checked ~ .sale::after {
    content: "Showing fashion products";
}
#cat-home-garden:checked ~ .sale::after {
    content: "Showing home & garden products";
}
#cat-sports-outdoors:checked ~ .sale::after {
```

```
            content: "Showing sports & outdoors products";
        }
        #cat-health-beauty:checked ~ .sale::after {
            content: "Showing health & beauty products";
        }
        .footer {
            grid-area: footer;
            background-color: #7a1f33;
            color: white;
            display: flex;
            align-items: center;
            justify-content: center;
            font-size: 16px;
        }
    </style>
</head>
<body>
    <div class="container">
        <input class="category-input" type="radio" name="category" id="cat-all" checked>
        <input class="category-input" type="radio" name="category" id="cat-electronics">
        <input class="category-input" type="radio" name="category" id="cat-fashion">
        <input class="category-input" type="radio" name="category" id="cat-home-garden">
        <input class="category-input" type="radio" name="category" id="cat-sports-outdoors">
        <input class="category-input" type="radio" name="category" id="cat-health-beauty">
        <div class="header">Welcome to Our E-Commerce Store</div>
        <div class="sale"></div>
        <div class="sidebar">
            <h2>Categories</h2>
            <label for="cat-all" class="category-button">All Products</label>
```

```
            <label for="cat-electronics" class="category-button">Electronics</label>
            <label for="cat-fashion" class="category-button">Fashion</label>
            <label for="cat-home-garden" class="category-button">Home & Garden</label>
            <label for="cat-sports-outdoors" class="category-button">Sports & Outdoors</label>
            <label for="cat-health-beauty" class="category-button">Health & Beauty</label>
        </div>
        <div class="products">
            <div class="product-card" data-category="electronics">
                <img src="C:\LOCAL DISK(D)\DEEPAK\AV.SC.U4CSE25027\LAB6\wirelesshp.webp"
alt="Wireless Headphones">
                <h3>Wireless Headphones</h3>
                <p>$29.99</p>
            </div>
            <div class="product-card" data-category="fashion">
                <img src="C:\LOCAL DISK(D)\DEEPAK\AV.SC.U4CSE25027\LAB6\sneakers.webp"
alt="Fashion Sneaker">
                <h3>Fashion Sneaker</h3>
                <p>$49.99</p>
            </div>
            <div class="product-card" data-category="home-garden">
                <img src="C:\LOCAL DISK(D)\DEEPAK\AV.SC.U4CSE25027\LAB6\grandlantern.webp"
alt="Garden Lantern">
                <h3>Garden Lantern</h3>
                <p>$19.99</p>
            </div>
            <div class="product-card" data-category="sports-outdoors">
                <img src="C:\LOCAL DISK(D)\DEEPAK\AV.SC.U4CSE25027\LAB6\waterbottle.webp"
alt="Sports Water Bottle">
                <h3>Sports Water Bottle</h3>
                <p>$39.99</p>
            </div>
```

```
        </div>
        <div class="footer">© 2026 E-Commerce Store. All rights reserved.</div>
    </div>
</body>

</html>
```

OUTPUT:



Usage of grid layout on my code:-

Why Use CSS Grid?

- **Responsive design**: Easily adapts to different screen sizes.
- **Cleaner code**: No need for floats, positioning tricks, or nested tables.
- **Powerful alignment**: Items can overlap, span multiple rows/columns, or auto-fit.
- **Modern support**: Fully supported in all major browsers.

Main applications of grid layout:-

Browser support: While modern browsers support CSS Grid, older ones may not. Always check compatibility.

- **Learning curve**: More complex than Flexbox, but extremely powerful once mastered.
- **Best practice**: Use Grid for overall page structure and Flexbox for smaller components inside the grid.

How CSS transitions and transformations helped me in making my webpage look attractive and Magnificient ?

CSS Transitions

Transitions allow property changes (like color, size, or position) to occur gradually instead of instantly.

- **Key Properties:**
- **transition-property: Which CSS property to animate (e.g., width, background-color).**
- **transition-duration: How long the transition lasts (e.g., 2s).**
- **transition-timing-function: The speed curve (e.g., ease, linear, ease-in-out).**
- **transition-delay: When the transition should start.**

CSS Transformations

Transformations change the *shape, size, or position* of elements without affecting the document flow.

- **Common Functions:**
- **rotate(angle): Rotates an element (e.g., rotate(45deg)).**
- **scale(x, y): Resizes an element (e.g., scale(1.5)).**
- **translate(x, y): Moves an element (e.g., translate(50px, 100px)).**
- **skew(x, y): Slants an element (e.g., skew(20deg, 10deg)).**

WEEK-7

7 AIM: To create the repository in GIT Hub and push or upload files of all the programs that are done to create different webpages and websites

Procedure:-

01

Create a GitHub accountGo to github.com, click Sign Up, and register using your email, username, and password.

02

Install Git locallyDownload Git from git-scm.com and install it on your computer. Verify installation with 'git --version' in terminal.

03

Configure Git user detailsSet your username and email with 'git config --global user.name "YourName"' and 'git config --global user.email "you@example.com"'.

04

Create a new repositoryOn GitHub, click New Repository, give it a name (e.g., Lab7-Webpages), choose public/private, and click Create.

05

Initialize Git in project folderOpen terminal in your project folder and run 'git init' to start version control.

06

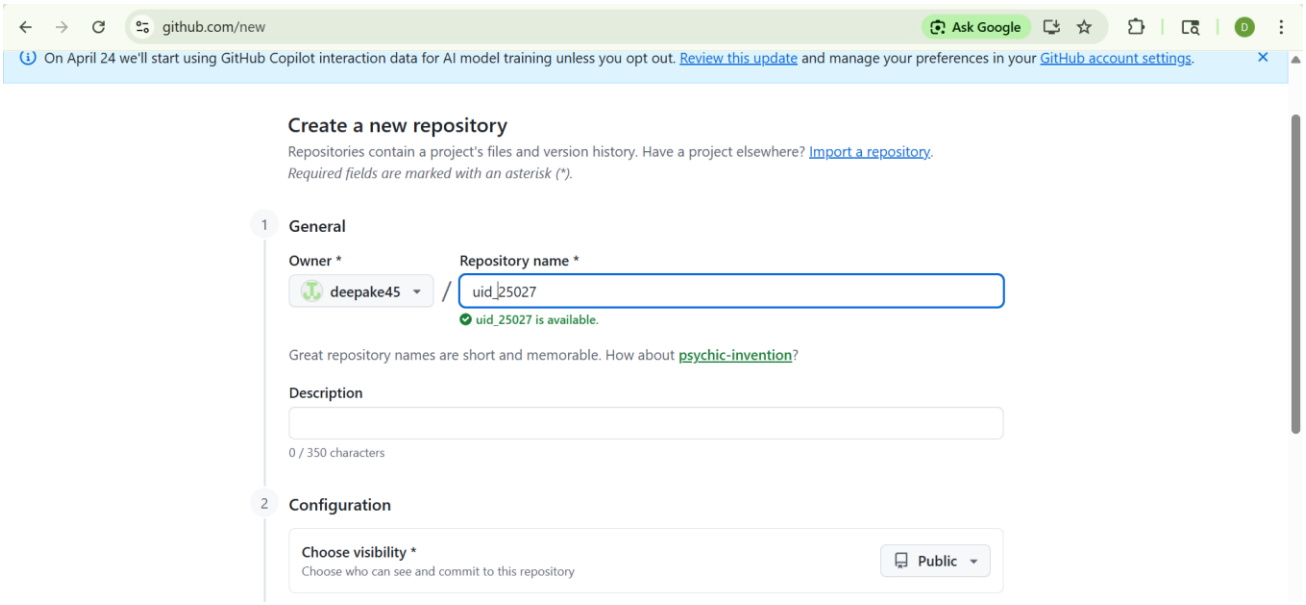
Add and commit filesUse 'git add .' to stage all files, then 'git commit -m "Initial commit"' to save changes.

07

Connect to GitHub repositoryCopy the repository URL from GitHub and run 'git remote add origin <URL>' in your terminal.

08

Push files to GitHubRun 'git push -u origin main' (or master if default) to upload your files to the repository.



Commands that I used to push my files are:-

1. git init Initializes a new Git repository in your current folder. It creates a hidden .git directory where Git stores all version history.

2. git add . Stages all files in the current directory (the . means “everything here”). This tells Git which files you want to include in the next commit.

3. git commit -m "First commit" Saves a snapshot of the staged files into the repository with a message ("First commit"). The message should describe what you changed or added.

4. git branch -M main Renames the current branch to main. GitHub uses main as the default branch name now (instead of master).

5. git remote add origin <PASTE_YOUR_URL_HERE> Links your local repository to a remote repository on GitHub. Replace <PASTE_YOUR_URL_HERE> with the HTTPS or SSH URL of your GitHub repo.

6. git push -u origin main Pushes your local main branch to the remote repository named origin (GitHub). The -u sets up tracking so future git push commands know where to send changes.

Updating Files Later

- git add . Stage new or modified files again.
- git commit -m "RANDOM" Commit those changes with a message (here "RANDOM" is just a placeholder — use something meaningful like "Updated CSS").
- git push origin main Pushes the new commit(s) to GitHub.

```
PS C:\LOCAL DISK(D)\DEEPAK\AV.SC.U4CSE25027> git remote add origin https://github.com/deepake45/uid-25027.git
>> git branch -M main
>> git push -u origin main
error: remote origin already exists.
branch 'main' set up to track 'origin/main'.
Everything up-to-date
PS C:\LOCAL DISK(D)\DEEPAK\AV.SC.U4CSE25027> git pull
Already up to date.
PS C:\LOCAL DISK(D)\DEEPAK\AV.SC.U4CSE25027> git push
Everything up-to-date
PS C:\LOCAL DISK(D)\DEEPAK\AV.SC.U4CSE25027> git add
Nothing specified, nothing added.
hint: Maybe you wanted to say 'git add .'?
hint: Disable this message with "git config set advice.addEmptyPathspec false"
PS C:\LOCAL DISK(D)\DEEPAK\AV.SC.U4CSE25027> git add .
PS C:\LOCAL DISK(D)\DEEPAK\AV.SC.U4CSE25027> git pull .
From .
 * branch            HEAD       -> FETCH_HEAD
Already up to date.
PS C:\LOCAL DISK(D)\DEEPAK\AV.SC.U4CSE25027> git push .
Everything up-to-date
PS C:\LOCAL DISK(D)\DEEPAK\AV.SC.U4CSE25027> git commit -m "Added new files"
```

After following all the commands my files are pushed into Git hub and hence are displayed as follows:-

WEEK-8

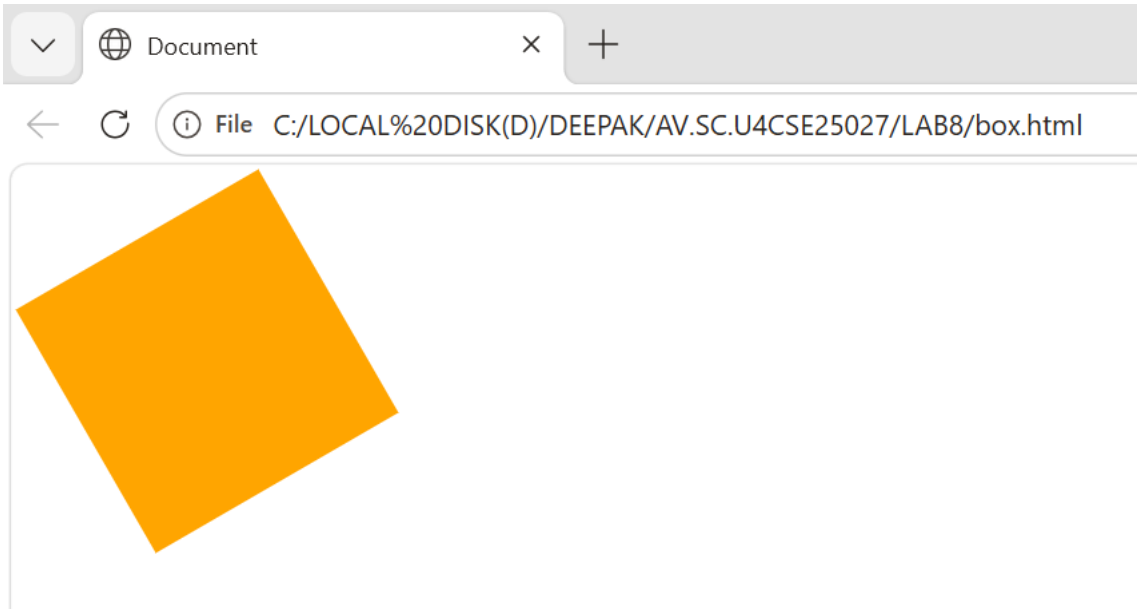
8(a):-Aim:-Create a webpage that displays a square that continuously rotates 360° using CSS animation.

PROGRAM:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <style>
        body {
            padding: 20px;
        }
        .square {
            width: 140px;
            height: 140px;
```

```
        background-color:orange;
        animation: rotateSquare 2s linear infinite;
      }

      @keyframes rotateSquare {
        from {
          transform: rotate(0deg);
        }
        to {
          transform: rotate(360deg);
        }
      }
    </style>
</head>
<body>
    <div class="square"></div>
</body>
</html>
```

OUTPUT:



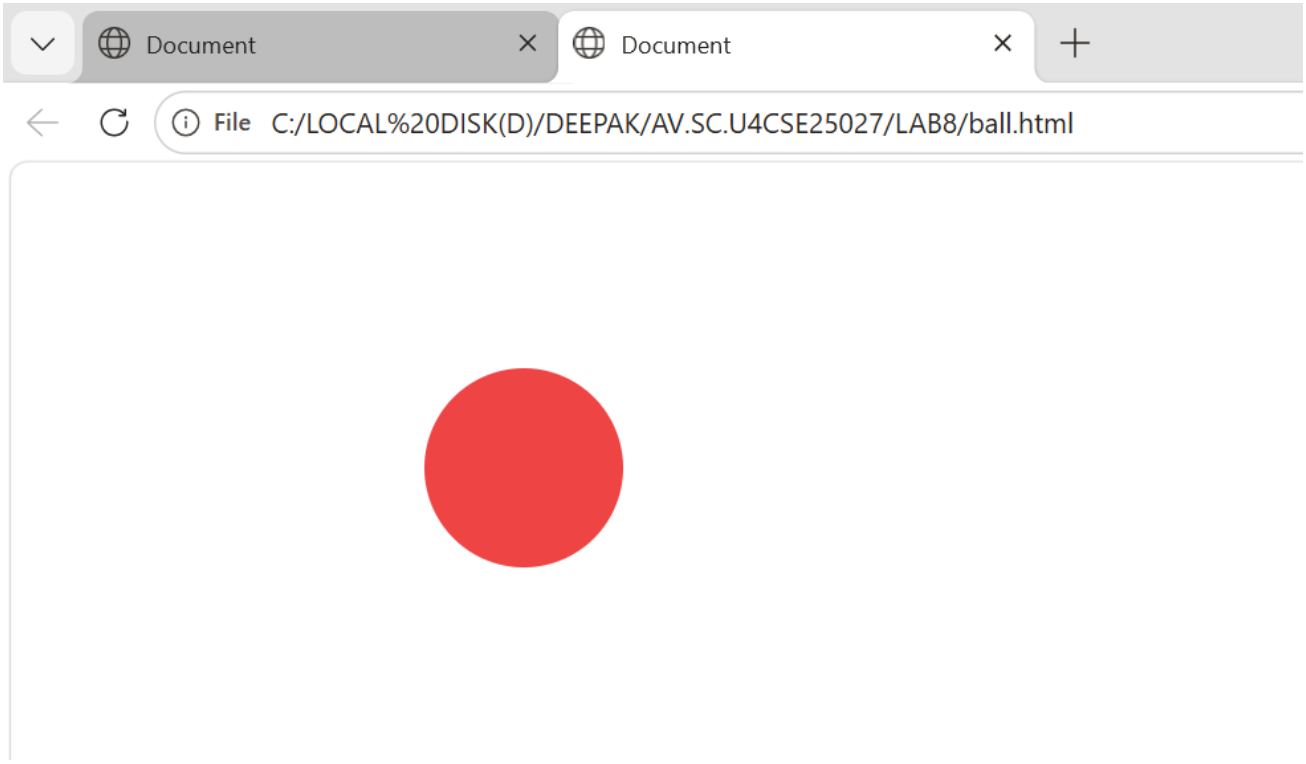
8(a):-AIM:-Create a ball that moves up and down continuously using CSS animation.

PROGRAM:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Document</title>
   <style>
      body {
         padding: 200px;
      }

      .ball {
         width: 100px;
         height: 100px;
         border-radius: 50%;
         background: #ef4444;
         animation: bounce 1.2s ease-in-out infinite;
      }

      @keyframes bounce {
         0%, 100% {
            transform: translateY(0);
         }
         50% {
            transform: translateY(-120px);
         }
      }
```

```
    </style>
</head>
<body>
    <div class="ball"></div>
</body>
</html>
```

OUTPUT:



8(c):-AIM:- Create a circle that moves horizontally and rotes at the same time.

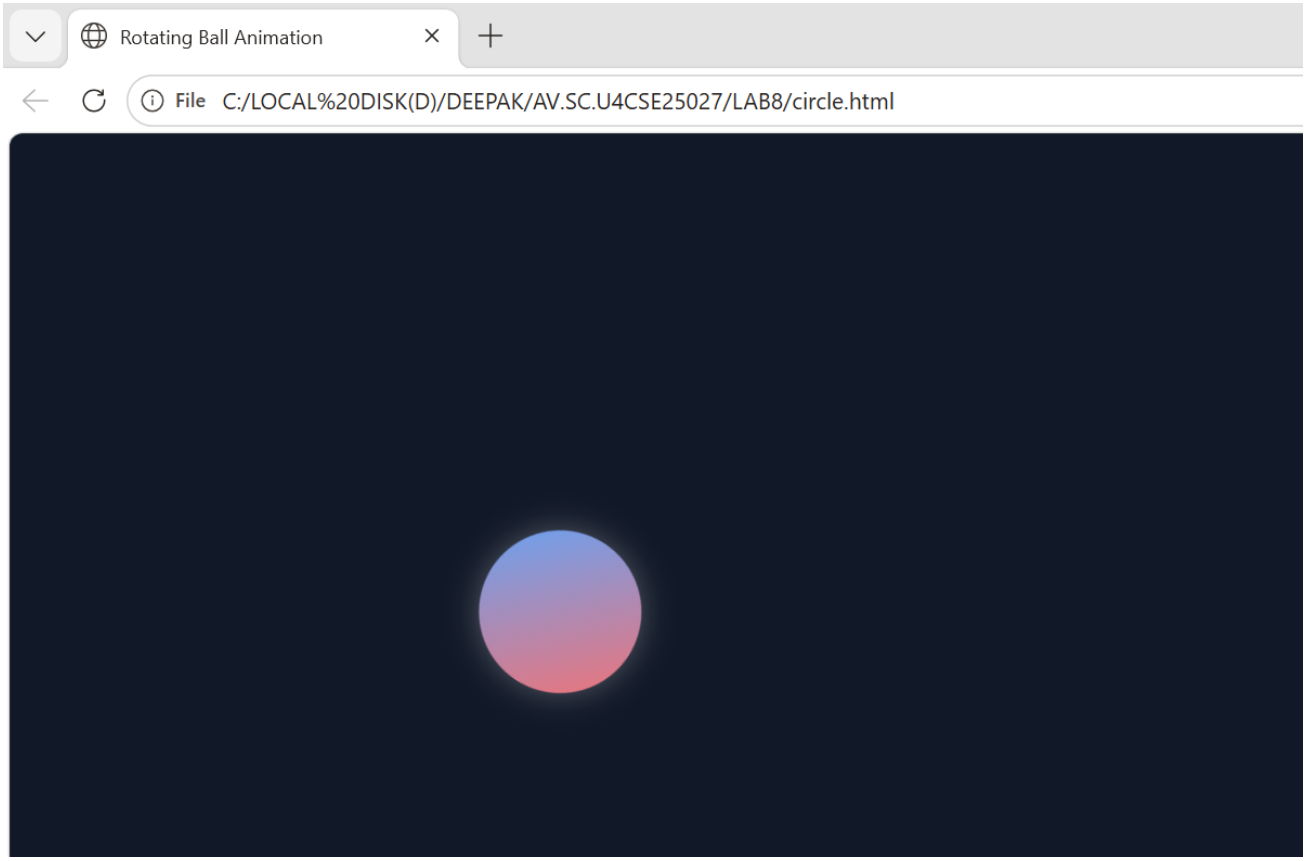
PROGRAM:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Rotating Ball Animation</title>
```

```
<style>
    body {
        margin: 0;
        height: 100vh;
        display: flex;
        align-items: center;
        justify-content: flex-start; /* Start from left */
        background: #111827; /* Dark background for contrast */
        overflow: hidden;
        padding: 0 20px;
    }

    .ball {
        width: 100px;
        height: 100px;
        border-radius: 50%;
        background: linear-gradient(45deg, #f87171, #60a5fa);
        box-shadow: 0 0 20px rgba(255, 255, 255, 0.3);
        animation: moveRotate 3s linear infinite alternate;
    }

    @keyframes moveRotate {
        from {
            transform: translateX(0) rotate(0deg);
        }
        to {
            transform: translateX(320px) rotate(360deg);
        }
    }
```

```
    </style>
</head>
<body>
    <div class="ball"></div>
</body>
</html>
```

OUTPUT:



8a) Rotating Square

Tags / properties mainly used:

- → to create square
- width, height
- background-color
- animation
- @keyframes
- transform: rotate()

Purpose:

To continuously rotate a square **360°**.

8b) Ball Moving Up and Down

Tags / properties mainly used:

- → to create ball
- width, height
- background-color
- border-radius: 50%
- animation
- @keyframes
- transform: translateY()

Purpose:

To make the ball move **up and down continuously**.

8c) Circle Moving Horizontally and Rotating

Tags / properties mainly used:

- → to create circle
- width, height
- background-color
- border-radius: 50%
- animation
- @keyframes
- transform: translateX() rotate()

Purpose:

Briefly:-

To move the circle **left to right** while also **rotating at the same time**.

CSS properties like width, height, background-color, border-radius, animation, transform, and @keyframes are used to design shapes and apply animations such as rotation and movement. These animations make the webpage more interactive and visually attractive.

Week-9

Create an webpage using html, CSS, java script based on the given conditions:

a)Create a button for each program, based on the button clicked the corresponding program has to be executed and should display the output.

b)The logics for each button includes finding an even numbers, odd number, prime or not, factorial of a number, armstrong or not.

c)Use appropriate designs to display the output.

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Number Programs</title>
  <style>
    body {
      font-family: Arial;
      text-align: center;
      background: #f4f4f4;
    }

    .container {
      margin-top: 50px;
      padding: 20px;
      background: white;
      width: 300px;
      margin-left: auto;
      margin-right: auto;
      border-radius: 10px;
      box-shadow: 0px 0px 10px gray;
    }
```

```
        input {
            padding: 10px;
            width: 80%;
            margin-bottom: 10px;
        }

        button {
            padding: 10px;
            margin: 5px;
            border: none;
            border-radius: 5px;
            background: #007BFF;
            color: white;
            cursor: pointer;
        }

        button:hover {
            background: #0056b3;
        }

        #output {
            margin-top: 15px;
            padding: 10px;
            background: #e9ecef;
            border-radius: 5px;
        }
    </style>
</head>
```

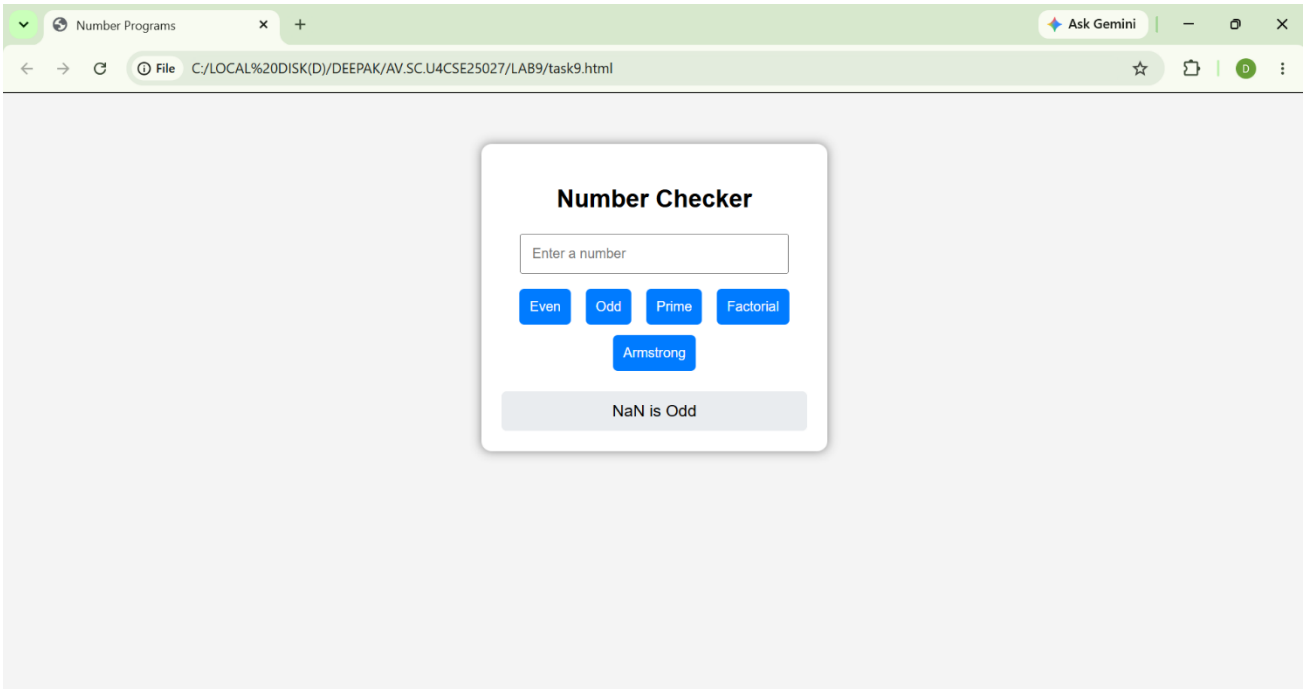
```
<body>

<div class="container">
  <h2>Number Checker</h2>

  <input type="number" id="num" placeholder="Enter a number">

  <br>

  <button onclick="checkEven()">Even</button>
  <button onclick="checkOdd()">Odd</button>
  <button onclick="checkPrime()">Prime</button>
  <button onclick="factorial()">Factorial</button>
  <button onclick="armstrong()">Armstrong</button>

  <div id="output">Output will appear here</div>
</div>

<script>
  function getNumber() {
    return parseInt(document.getElementById("num").value);
  }

  function display(msg) {
    document.getElementById("output").innerHTML = msg;
  }

  function checkEven() {
    let n = getNumber();
```

```
    if (n % 2 === 0)
        display(n + " is Even");
    else
        display(n + " is Not Even");
}

function checkOdd() {
    let n = getNumber();
    if (n % 2 !== 0)
        display(n + " is Odd");
    else
        display(n + " is Not Odd");
}

function checkPrime() {
    let n = getNumber();
    let isPrime = true;

    if (n <= 1) isPrime = false;

    for (let i = 2; i <= Math.sqrt(n); i++) {
        if (n % i === 0) {
            isPrime = false;
            break;
        }
    }

    if (isPrime)
        display(n + " is Prime");
```

```
    else
        display(n + " is Not Prime");
}

function factorial() {
    let n = getNumber();
    let fact = 1;

    for (let i = 1; i <= n; i++) {
        fact *= i;
    }

    display("Factorial of " + n + " is " + fact);
}

function armstrong() {
    let n = getNumber();
    let sum = 0;
    let temp = n;

    while (temp > 0) {
        let digit = temp % 10;
        sum += digit * digit * digit;
        temp = Math.floor(temp / 10);
    }

    if (sum === n)
        display(n + " is an Armstrong Number");
    else
```

```
        display(n + " is Not an Armstrong Number");
    }
</script>

</body>
</html>
```

Output:



Explanation:

Explanation:

<input type="number"> → Lets the user enter a number.

<button onclick="function()"> → Creates buttons that call JavaScript functions when clicked.

input{...} → Adds padding and font size to the input box.

button{...} → Styles buttons with padding, color, rounded corners.

button:hover{...} → Changes button color when mouse hovers.

result{...} → Styles the result text (green, larger font).

JavaScript (Logic)

- document.getElementById("id") → Connects JavaScript to HTML elements.

- .value → Reads the number entered in the input box.
- .innerHTML → Updates the result paragraph with new text.
- **Even/Odd functions** → Use % (modulo) to check divisibility by 2.
- **Prime function** → Loops from 2 to N-1 to check if divisible.
- **Factorial function** → Uses a for loop to multiply numbers up to N.
- **Armstrong function** →
    - Converts number to digits (toString().length).
    - Extracts digits with % and Math.floor().
    - Raises each digit to the power of total digits (**).
    - Compares sum with original number.

Week-10

Aim: write JavaScript conditional statement to find the sign of product of three numbers using functions .

Codes:

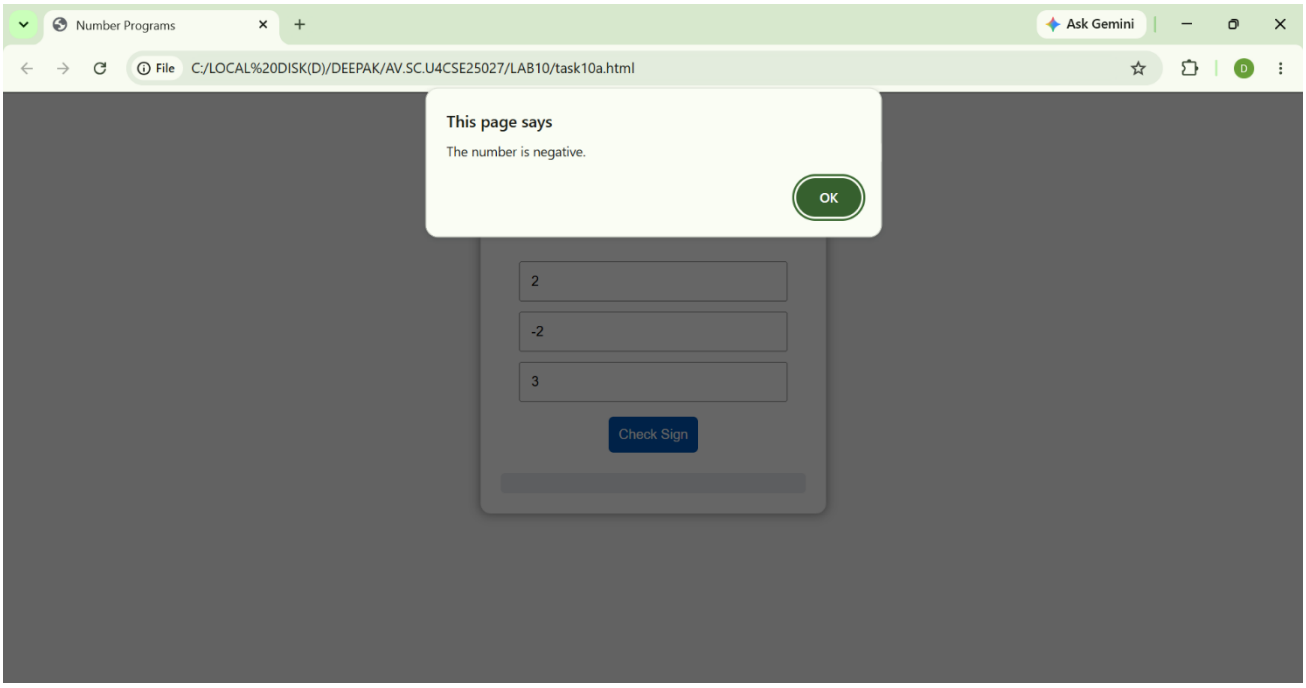
```
<!DOCTYPE html>
<html>
<head>
  <title>Number Programs</title>
  <style>
    body {
      font-family: Arial;
      text-align: center;
      background: #f4f4f4;
    }
```

```
.container {
    margin-top: 50px;
    padding: 20px;
    background: white;
    width: 300px;
    margin-left: auto;
    margin-right: auto;
    border-radius: 10px;
    box-shadow: 0px 0px 10px gray;
}

input {
    padding: 10px;
    width: 80%;
    margin-bottom: 10px;
}

button {
    padding: 10px;
    margin: 5px;
    border: none;
    border-radius: 5px;
    background: #007BFF;
    color: white;
    cursor: pointer;
}

button:hover {
```

```
            background: #0056b3;
        }

        #output {
            margin-top: 15px;
            padding: 10px;
            background: #e9ecef;
            border-radius: 5px;
        }
    </style>
</head>
<body>
    <div class="container">
        <h2>Sign of Product of 3 numbers</h2>
        <input type="number" id="num1" placeholder="Enter 1st number">
        <input type="number" id="num2" placeholder="Enter 2nd number">
        <input type="number" id="num3" placeholder="Enter 3rd number">
        <br>
        <button onclick="checkNumber()">Check Sign</button>
        <div id="output"></div>
    </div>

    <script>
        function checkNumber() {
            const num1 = document.getElementById('num1').value;
            const num2 = document.getElementById('num2').value;
            const num3 = document.getElementById('num3').value;
            const output = document.getElementById('output');
```

```
        if (num1 === '' || num2 === '' || num3 === '') {
            output.textContent = 'Please enter all three numbers.';
            return;
        }

        const number = parseFloat(num1) * parseFloat(num2) * parseFloat(num3);

        if (isNaN(number)) {
            window.alert('Invalid input. Please enter valid numbers.');
            return;
        }

        if (number > 0) {
            window.alert('The number is positive.');
        } else if (number < 0) {
            window.alert('The number is negative.');
        } else {
            window.alert('The number is zero.');
        }
    }
  </script>
</body>
</html>
```

Output:



Explanation:

The program determines the sign of the product of three numbers without focusing on the exact value.

1. **The inputs are taken from three fields and converted into numeric values using Number().**
2. **The product of the three numbers is calculated:**
   let product = a * b * c;
3. **The sign of the result is determined using conditional checks:**
   - **If product > 0 → the result is positive (+)**
   - **If product < 0 → the result is negative (-)**
   - **If product == 0 → the result is zero (0)**
4. **The output is displayed using alert().**

Logical Insight:
Instead of checking the exact product value, the program mainly focuses on identifying the sign (positive, negative, or zero). Direct multiplication is used here because it makes the logic simple, clear, and easy to understand.

B)

Aim: write a java script conditional statements to sort 3 numbers using functions display an alert box to show the result

Codes:

<!DOCTYPE html>

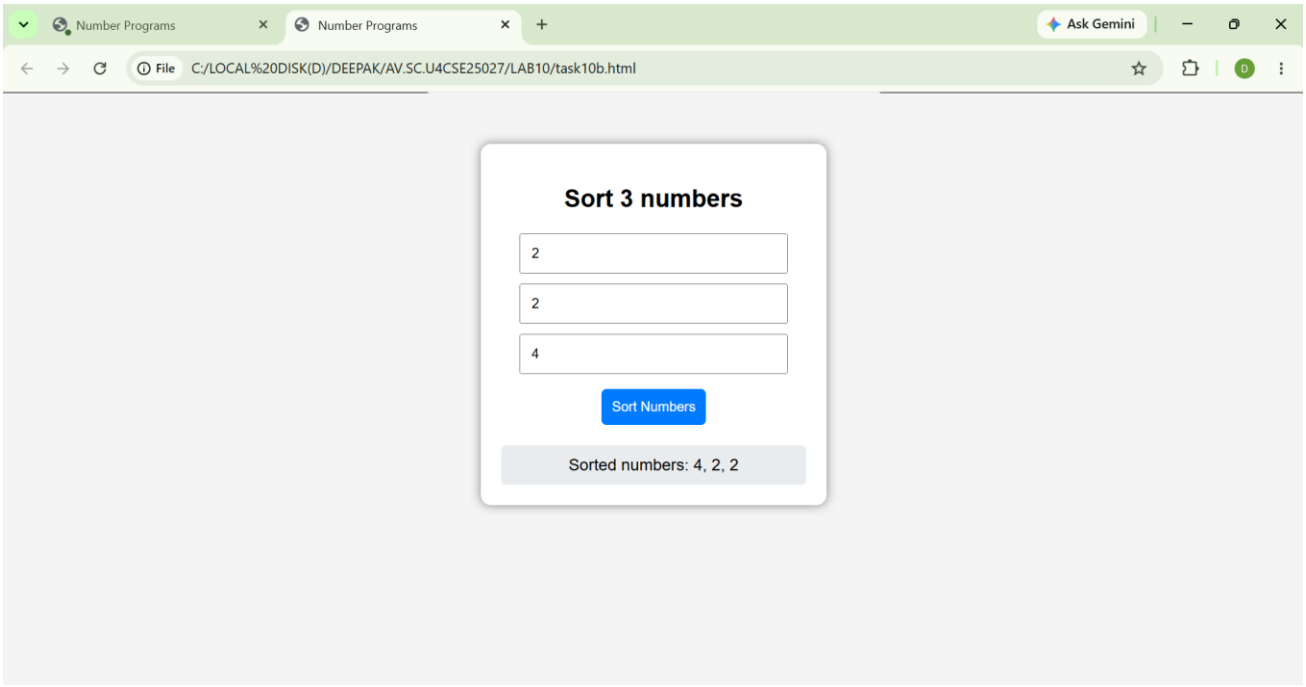
```
<html>
<head>
  <title>Number Programs</title>
  <style>
    body {
      font-family: Arial;
      text-align: center;
      background: #f4f4f4;
    }

    .container {
      margin-top: 50px;
      padding: 20px;
      background: white;
      width: 300px;
      margin-left: auto;
      margin-right: auto;
      border-radius: 10px;
      box-shadow: 0px 0px 10px gray;
    }

    input {
      padding: 10px;
      width: 80%;
      margin-bottom: 10px;
    }

    button {
      padding: 10px;
```

```
            margin: 5px;
            border: none;
            border-radius: 5px;
            background: #007BFF;
            color: white;
            cursor: pointer;
        }

        button:hover {
            background: #0056b3;
        }

        #output {
            margin-top: 15px;
            padding: 10px;
            background: #e9ecef;
            border-radius: 5px;
        }
    </style>
</head>
<body>
    <div class="container">
        <h2>Sort 3 numbers</h2>
        <input type="number" id="num1" placeholder="Enter 1st number">
        <input type="number" id="num2" placeholder="Enter 2nd number">
        <input type="number" id="num3" placeholder="Enter 3rd number">
        <br>
        <button onclick="sortNumbers()">Sort Numbers</button>
        <div id="output"></div>
```

```
</div>

<script>
    function getNumber(id) {
        return parseFloat(document.getElementById(id).value);
    }

    function validateNumbers(numbers) {
        return numbers.every(num => !isNaN(num));
    }

    function sortDescending(numbers) {
        return numbers.slice().sort((a, b) => b - a);
    }

    function showOutput(message) {
        document.getElementById('output').textContent = message;
    }

    function sortNumbers() {
        const numbers = [
            getNumber('num1'),
            getNumber('num2'),
            getNumber('num3')
        ];

        if (!validateNumbers(numbers)) {
            showOutput('Please enter valid numbers.');
            return;
```

```
        }

        const sortedNumbers = sortDescending(numbers);
        showOutput(`Sorted numbers: ${sortedNumbers.join(', ')}`);
      }
  </script>
</body>
</html>
```

Output:



Explanation:

The program sorts three numbers in ascending order without using built-in sorting methods.

1. The inputs are taken from three fields and converted into numeric values using Number().
2. A temporary variable is used for swapping values when required:
   let temp;
3. The numbers are sorted using conditional comparisons and swapping:
   - If a > b → swap a and b
   - If a > c → swap a and c
   - If b > c → swap b and c
4. After these comparisons:
   - a stores the smallest number

- b stores the middle number
- c stores the largest number

5. The sorted result is displayed using alert().

Logical Insight:
Instead of using a built-in sorting function, the program compares and swaps numbers step by step. This method makes the sorting process simple, clear, and easy to understand for small sets of numbers.

C) Aim:

Write a java script conditional statement to find largest of find largest of five numbers using function and onclick event .display an alert box to show the result

Codes:

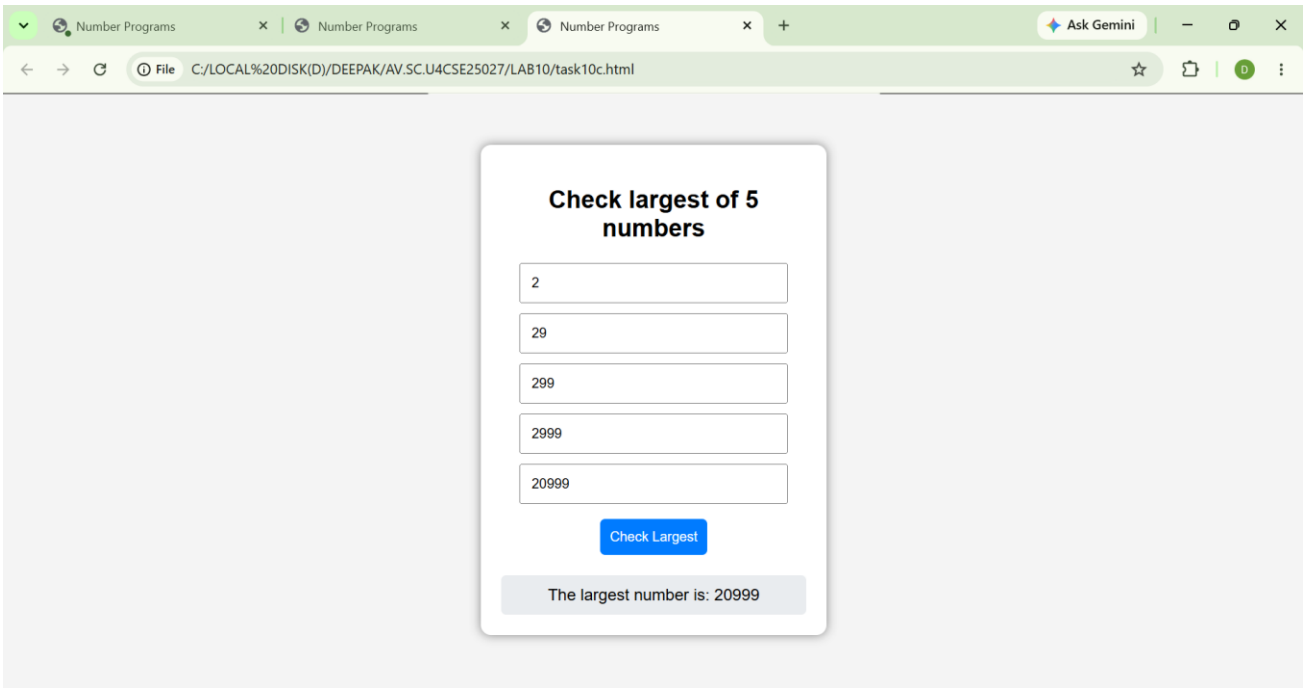
```
<!DOCTYPE html>
<html>
<head>
  <title>Number Programs</title>
  <style>
    body {
      font-family: Arial;
      text-align: center;
      background: #f4f4f4;
    }

    .container {
      margin-top: 50px;
      padding: 20px;
      background: white;
      width: 300px;
      margin-left: auto;
      margin-right: auto;
      border-radius: 10px;
      box-shadow: 0px 0px 10px gray;
```

```
    }

    input {
        padding: 10px;
        width: 80%;
        margin-bottom: 10px;
    }

    button {
        padding: 10px;
        margin: 5px;
        border: none;
        border-radius: 5px;
        background: #007BFF;
        color: white;
        cursor: pointer;
    }

    button:hover {
        background: #0056b3;
    }

    #output {
        margin-top: 15px;
        padding: 10px;
        background: #e9ecef;
        border-radius: 5px;
    }
</style>
```

```
</head>
<body>
  <div class="container">
    <h2>Check largest of 5 numbers </h2>
    <input type="number" id="num1" placeholder="Enter 1st number">
    <input type="number" id="num2" placeholder="Enter 2nd number">
    <input type="number" id="num3" placeholder="Enter 3rd number">
    <input type="number" id="num4" placeholder="Enter 4th number">
    <input type="number" id="num5" placeholder="Enter 5th number">
    <br>
    <button onclick="checkLargest()">Check Largest</button>
    <div id="output"></div>
  </div>
  <script>
    function checkLargest() {
      let num1 = parseFloat(document.getElementById("num1").value);
      let num2 = parseFloat(document.getElementById("num2").value);
      let num3 = parseFloat(document.getElementById("num3").value);
      let num4 = parseFloat(document.getElementById("num4").value);
      let num5 = parseFloat(document.getElementById("num5").value);

      let largest = num1;

      if (num2 > largest) {
        largest = num2;
      }
      if (num3 > largest) {
        largest = num3;
      }
```

```
        if (num4 > largest) {
            largest = num4;
        }
        if (num5 > largest) {
            largest = num5;
        }

        document.getElementById("output").innerText = "The largest number is: " + largest;
    }
  </script>
</body>
</html>
```

Output:



Explanation:

The program determines the largest number among five unique inputs without using complex conditions.

1. The inputs are taken from five fields and converted into numeric values using Number().
2. Input validation is performed to ensure all fields are filled.
   • If any input field is empty → the program stops execution.

3. A duplicate check is performed to ensure all numbers are unique:
   • If any two numbers are equal → the program stops execution
   • This ensures only unique values are processed
4. A temporary variable is initialized with the first number:
   let temp = a;
5. The remaining numbers are compared one by one with temp:
   • If b > temp → temp = b
   • If c > temp → temp = c
   • If d > temp → temp = d
   • If e > temp → temp = e
6. After all comparisons:
   • temp stores the largest number
7. The final result is displayed using alert().

Logical Insight:
Instead of using multiple complex conditions, the program compares numbers step by step using incremental comparison. This approach makes the logic simple, efficient, and easy to maintain.

D)

Aim: write a java script for loop that will iterate from 0 to 15. For each iteration ,it will check if the current number odd or even , and display a message on screen

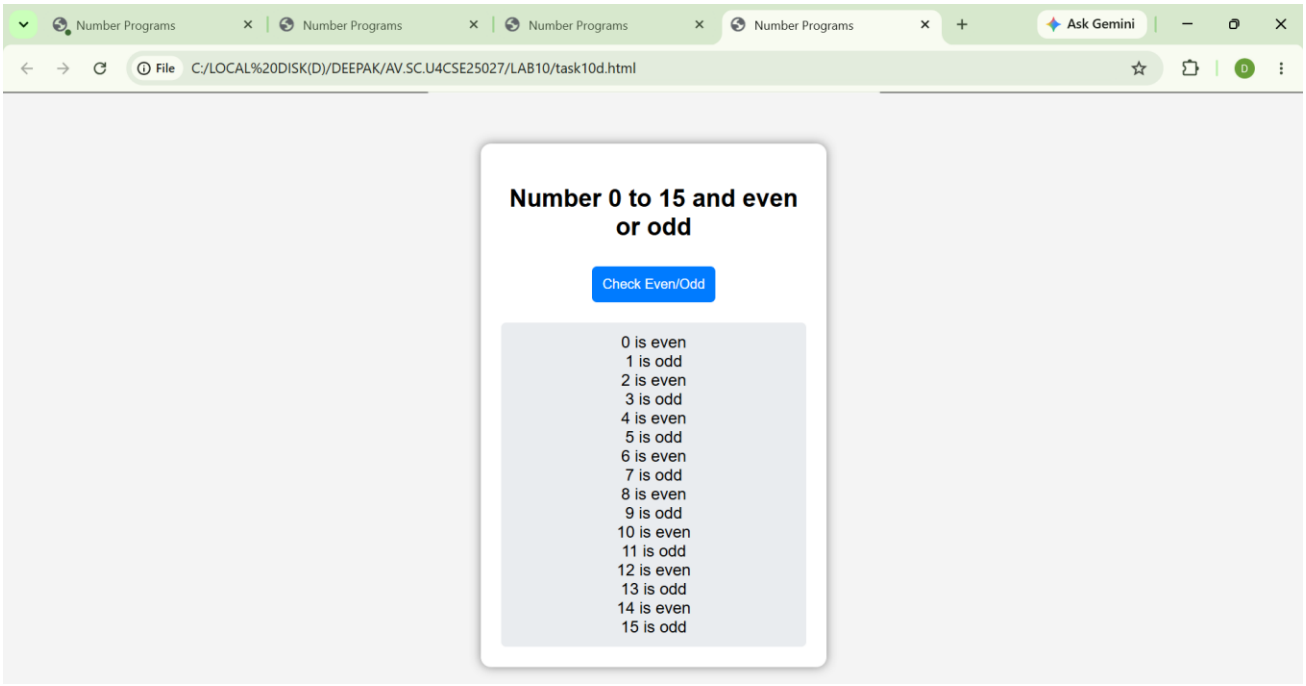
Codes:

```
<!DOCTYPE html>

<html>

<head>

  <title>Number Programs</title>

  <style>

    body {

      font-family: Arial;

      text-align: center;

      background: #f4f4f4;

    }


    .container {

      margin-top: 50px;
```

```
    padding: 20px;
    background: white;
    width: 300px;
    margin-left: auto;
    margin-right: auto;
    border-radius: 10px;
    box-shadow: 0px 0px 10px gray;
}

input {
    padding: 10px;
    width: 80%;
    margin-bottom: 10px;
}

button {
    padding: 10px;
    margin: 5px;
    border: none;
    border-radius: 5px;
    background: #007BFF;
    color: white;
    cursor: pointer;
}

button:hover {
    background: #0056b3;
}
```

```
        #output {
            margin-top: 15px;
            padding: 10px;
            background: #e9ecef;
            border-radius: 5px;
        }
    </style>
</head>
<body>
    <div class="container">
        <h2>Number 0 to 15 and even or odd</h2>
        <button onclick="checkEvenOdd()">Check Even/Odd</button>
        <div id="output"></div>
    </div>

    <script>
        function checkEvenOdd() {
            let output = document.getElementById('output');
            output.innerHTML = ''; // Clear previous output

            for (let i = 0; i <= 15; i++) {
                let type = (i % 2 === 0) ? 'even' : 'odd';
                output.innerHTML += `${i} is ${type}<br>`;
            }
        }
    </script>
</body>
</html>
```

Output:



Explanation:

The program identifies whether numbers are even or odd from 0 to a given limit.

1. The input value is taken from the input field using document.getElementById().
2. Input validation is performed to ensure the field is not empty.
   • If the input field is empty → the program stops execution.
3. The input value is converted into a numeric value using Number():
   n = Number(n);
4. A loop is used to iterate through numbers from 0 to n:
   for (let i = 0; i <= n; i++)
5. Each number is checked using the modulus operator:
   i % 2 === 0
   • If true → the number is Even
   • Otherwise → the number is Odd
6. The result is stored in a string and displayed inside the output .

Logical Insight:
Instead of checking numbers manually, the program uses a loop and modulus operator to identify even and odd numbers efficiently. This approach makes the logic simple, clear, and easy to understand for any range of numbers.

E)

Aim: write a java script program to read 6 subjects marks from the student then computes the average marks of students then , this average is used to determine the corresponding grade.

Codes:

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Grade Calculator</title>
</head>
<body>

<script>
  let total = 0;

  for(let i = 1; i <= 6; i++) {
    let mark = parseFloat(prompt("Enter marks for Subject " + i + ":"));
    total += mark;
  }

  let average = total / 6;
  let grade;

  if(average >= 90)
    grade = "A+";
  else if(average >= 80)
    grade = "A";
  else if(average >= 70)
    grade = "B";
  else if(average >= 60)
    grade = "C";
  else if(average >= 50)
```

```
        grade = "D";
    else
        grade = "F";

    document.write("<h2>Student Result</h2>");
    document.write("Average Marks: " + average.toFixed(2) + "<br>");
    document.write("Grade: " + grade);
</script>

</body>
</html>
```

Output:

Student Result

Average Marks: 75.00
Grade: B

Explanation:

The program calculates the average marks of six subjects and assigns a grade based on performance.

1. The marks of six subjects are taken from input fields and converted into numeric values using Number().
2. Input validation is performed to ensure all fields are filled.
   • If any input field is empty → the program stops execution.

3. The total marks are calculated by adding all subject marks:
   sum = a + b + c + d + e + f;
4. The average marks are calculated using:
   avg = sum / 6;
5. The grade is assigned using conditional checks:
   • If avg >= 90 → Grade = A
   • If avg >= 75 → Grade = B
   • If avg >= 60 → Grade = C
   • Otherwise → Grade = D
6. After calculation:
   • avg stores the average marks
   • grade stores the assigned grade
7. The final result (average and grade) is displayed on the screen.

Logical Insight:

Instead of checking marks individually, the program first calculates the average and then assigns a grade using conditional logic. This approach makes the grading process simple, organized, and easy to understand.

Week 11

A) Aim: A website requires users to enter a username. The system should display a message while typing if the username is less than 5 characters.

Codes:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Username Validation</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
        }
        .error {
            color: red;
            margin-top: 5px;
        }
    </style>
</head>
<body>
    <h1>Enter Username</h1>
    <form>
        <label for="username">Username:</label>
        <input type="text" id="username" name="username" oninput="validateUsername()">
```

```
    <div id="message" class="error" style="display: none;"></div>
  </form>

  <script>
    function validateUsername() {
      const username = document.getElementById('username').value;
      const message = document.getElementById('message');

      if (username.length < 5) {
        message.textContent = 'Username must be at least 5 characters.';
        message.style.display = 'block';
      } else {
        message.style.display = 'none';
      }
    }
  </script>
</body>
</html>
```

Output:

Explanation:

The program validates a username based on the minimum length requirement.

1. The input value is taken from the text field using document.getElementById().
2. Input validation is performed to ensure the field is not empty.
   • If the input field is empty → a message is displayed asking the user to enter a username.
3. The length of the username is checked using:
   if (a.length < 5)
4. Based on the username length:
   • Less than 5 characters → Invalid Username
   • 5 or more characters → Valid Username
5. The result is displayed dynamically below the input field.

Logical Insight:
Instead of checking complex conditions, the program validates the username by checking its length. This approach makes the validation process simple, clear, and easy to understand.

B)

Aim: A registration form should validate email format before submission. If invalid, the form should not be submitted

Codes:

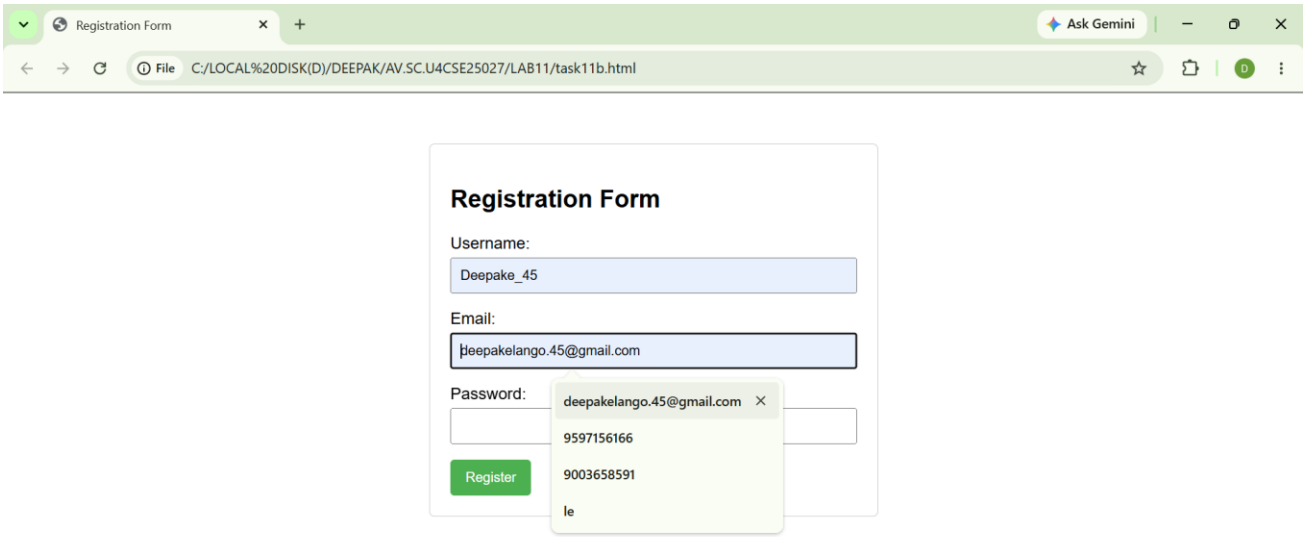
<!DOCTYPE html>

```
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Registration Form</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            max-width: 400px;
            margin: 50px auto;
            padding: 20px;
            border: 1px solid #ccc;
            border-radius: 5px;
        }
        .form-group {
            margin-bottom: 15px;
        }
        label {
            display: block;
            margin-bottom: 5px;
        }
        input[type="text"], input[type="email"], input[type="password"] {
            width: 100%;
            padding: 8px;
            box-sizing: border-box;
        }
        button {
            padding: 10px 15px;
            background-color: #4CAF50;
```

```
            color: white;
            border: none;
            border-radius: 4px;
            cursor: pointer;
        }
        button:hover {
            background-color: #45a049;
        }
        .error {
            color: red;
            font-size: 14px;
            margin-top: 5px;
        }
    </style>
</head>
<body>
    <h2>Registration Form</h2>
    <form id="registrationForm">
        <div class="form-group">
            <label for="username">Username:</label>
            <input type="text" id="username" name="username" required>
        </div>
        <div class="form-group">
            <label for="email">Email:</label>
            <input type="email" id="email" name="email" required>
            <div id="emailError" class="error"></div>
        </div>
        <div class="form-group">
            <label for="password">Password:</label>
```

```
        <input type="password" id="password" name="password" required>
    </div>
    <button type="submit">Register</button>
</form>

<script>
    document.getElementById('registrationForm').addEventListener('submit', function(event) {
        const email = document.getElementById('email').value;
        const emailError = document.getElementById('emailError');

        // Basic email validation regex
        const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

        if (!emailRegex.test(email)) {
            event.preventDefault(); // Prevent form submission
            emailError.textContent = 'Please enter a valid email address.';
        } else {
            emailError.textContent = ''; // Clear error message
        }
    });
</script>
</body>
</html>
```

Output:



Explanation:

The program validates an email address based on a proper email format.

1. The input value is taken from the email field using document.getElementById().
2. Input validation is performed to ensure the field is not empty.
   • If the input field is empty → the program stops execution or asks the user to enter an email.
3. A pattern (regular expression) is defined to check the email format:
   let pattern = /^[^]+@[^]+\.[a-z]{2,3}$/;
4. The entered email is checked against the pattern using:
   pattern.test(a)
5. Based on the result:
   • If the email matches the pattern → **Email is Valid**
   • If the email does not match the pattern → **Email is Invalid**
6. The result is displayed using alert().

Logical Insight:
Instead of checking the email manually, the program uses a regular expression pattern to validate the format. This approach makes the validation process simple, accurate, and easy to understand.

C)

Aim: A system should check password strength while typing and display whether it is weak or strong.

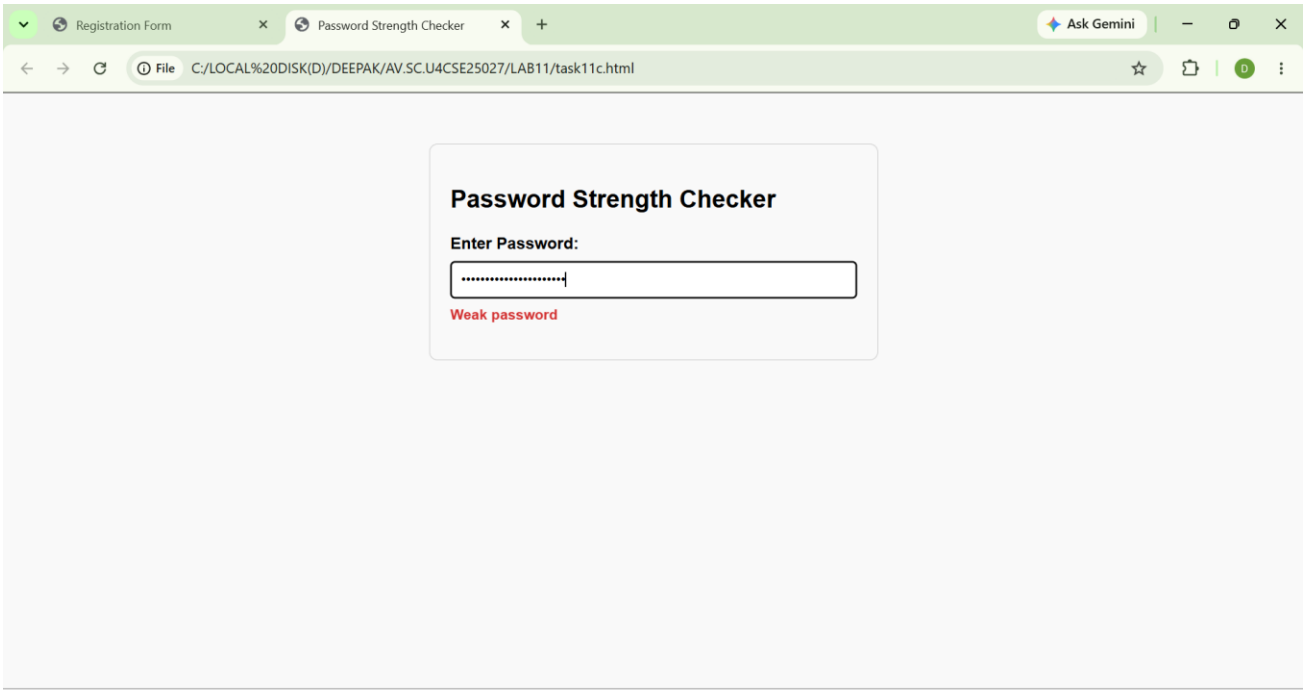
Codes:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Password Strength Checker</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            max-width: 400px;
            margin: 50px auto;
            padding: 20px;
            border: 1px solid #ccc;
            border-radius: 8px;
            background-color: #f9f9f9;
        }
        .form-group {
            margin-bottom: 16px;
        }
        label {
            display: block;
            margin-bottom: 8px;
            font-weight: bold;
        }
        input[type="password"] {
            width: 100%;
            padding: 10px;
            border: 1px solid #ccc;
```

```
            border-radius: 4px;
            box-sizing: border-box;
        }
        .strength-message {
            margin-top: 8px;
            font-size: 14px;
            font-weight: bold;
        }
        .weak {
            color: #d32f2f;
        }
        .strong {
            color: #388e3c;
        }
    </style>
</head>
<body>
    <h2>Password Strength Checker</h2>
    <div class="form-group">
        <label for="password">Enter Password:</label>
        <input type="password" id="password" placeholder="Type a password...">
        <div id="strengthMessage" class="strength-message"></div>
    </div>
    <script>
        const passwordInput = document.getElementById('password');
        const strengthMessage = document.getElementById('strengthMessage');

        passwordInput.addEventListener('input', function() {
            const value = passwordInput.value;
```

```
        const strength = getPasswordStrength(value);

        if (!value) {
            strengthMessage.textContent = '';
            strengthMessage.className = 'strength-message';
            return;
        }

        strengthMessage.textContent = strength.message;
        strengthMessage.className = 'strength-message ' + strength.className;
    });

    function getPasswordStrength(password) {
        const minLength = 8;
        const hasLowercase = /[a-z]/.test(password);
        const hasUppercase = /[A-Z]/.test(password);
        const hasNumber = /[0-9]/.test(password);
        const hasSymbol = /[^A-Za-z0-9]/.test(password);

        const score = [hasLowercase, hasUppercase, hasNumber, hasSymbol].filter(Boolean).length;

        if (password.length >= minLength && score >= 3) {
            return { message: 'Strong password', className: 'strong' };
        }
        return { message: 'Weak password', className: 'weak' };
    }
  </script>
</body>
</html>
```

Output:



Explanation:

The program checks the strength of a password based on predefined conditions.

1. The password is taken from the input field using document.getElementById().
2. Input validation is performed to ensure the field is not empty.
   • If the input field is empty → a message is displayed asking the user to enter a password.
3. Two patterns (regular expressions) are defined to check password strength:
   • **Strong Password** → contains uppercase letters, special symbols, and a minimum of 6 characters
   • **Medium Password** → contains uppercase letters and a minimum of 6 characters
4. The entered password is checked against the patterns using:
   strong.test(a)
5. Based on matching:
   • If it matches the **strong pattern** → **Strong Password**
   • If it matches the **medium pattern** → **Medium Password**
   • Otherwise → **Weak Password**
6. The result is displayed dynamically with color on the screen.

Logical Insight:

Instead of checking password conditions manually, the program uses regular expression patterns to classify password strength. This approach makes the validation process simple, organized, and easy to understand.

D)

Aim: A form should validate a mobile number when the user leaves the input field. The number must be exactly 10 digits

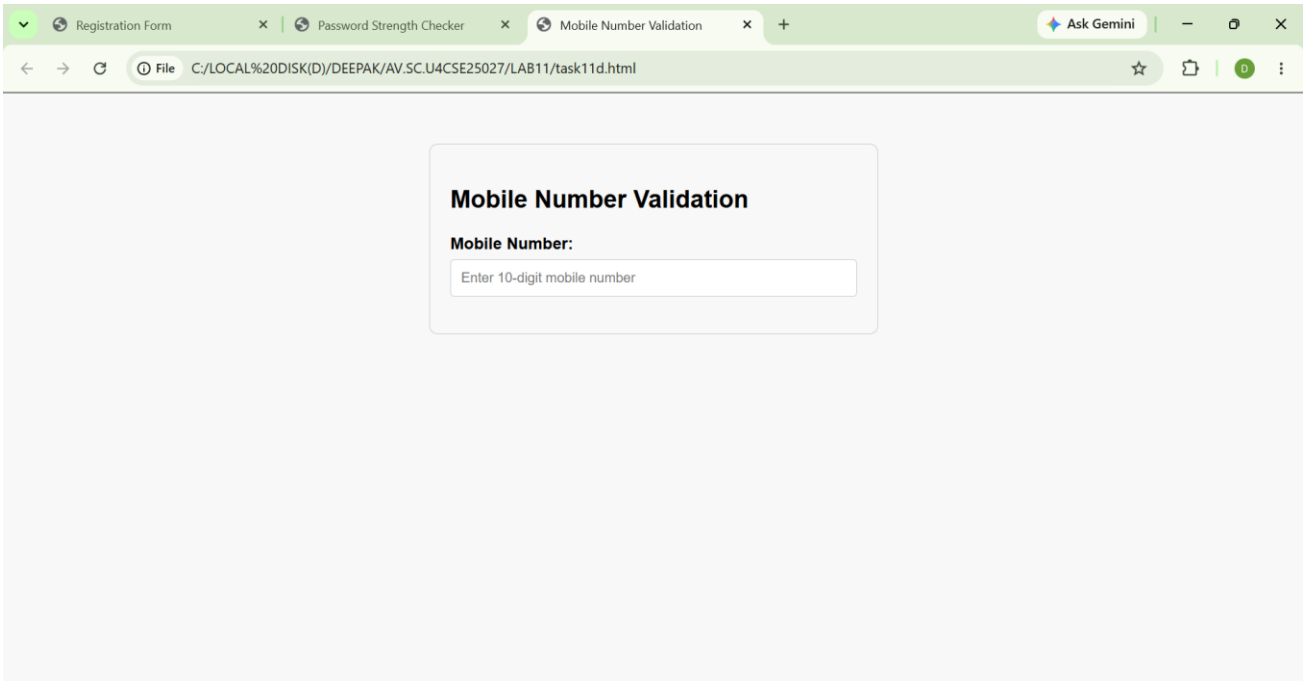
Codes:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mobile Number Validation</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 400px;
      margin: 50px auto;
      padding: 20px;
      border: 1px solid #ccc;
      border-radius: 8px;
      background-color: #f8f8f8;
    }
    .form-group {
      margin-bottom: 16px;
    }
    label {
      display: block;
      margin-bottom: 6px;
      font-weight: bold;
    }
    input[type="text"] {
```

```
            width: 100%;
            padding: 10px;
            border: 1px solid #ccc;
            border-radius: 4px;
            box-sizing: border-box;
        }
        .error {
            color: #d32f2f;
            margin-top: 8px;
            font-size: 14px;
        }
    </style>
</head>
<body>
    <h2>Mobile Number Validation</h2>
    <div class="form-group">
        <label for="mobile">Mobile Number:</label>
        <input type="text" id="mobile" name="mobile" placeholder="Enter 10-digit mobile number">
        <div id="mobileError" class="error"></div>
    </div>
    <script>
        const mobileInput = document.getElementById('mobile');
        const mobileError = document.getElementById('mobileError');

        mobileInput.addEventListener('blur', () => {
            const value = mobileInput.value.trim();
            const digitsOnly = /^[0-9]{10}$/;

            if (!value) {
```

```
            mobileError.textContent = 'Mobile number is required.';
            return;
        }

        if (!digitsOnly.test(value)) {
            mobileError.textContent = 'Enter exactly 10 digits.';
        } else {
            mobileError.textContent = '';
        }
    });
  </script>
</body>
</html>
```

Output:



Explanation:

Logic Explanation – Mobile Validation

The program validates a mobile number based on a proper 10-digit format.

1. The input value is taken from the text field using document.getElementById().
2. Input validation is performed to ensure the field is not empty.
   • If the input field is empty → a message is displayed asking the user to enter a mobile number.
3. A pattern (regular expression) is defined to check the mobile number format:
   /^[0-9]{10}$/
4. The entered mobile number is checked against the pattern.
5. Based on the result:
   • If the number contains exactly 10 digits → Valid Mobile Number
   • Otherwise → Invalid Mobile Number
6. The result is displayed dynamically on the screen.

◆ Logical Insight:

Instead of checking each digit manually, the program uses a regular expression pattern to validate the mobile number format. This approach makes the validation process simple, accurate, and easy to understand.

E)

Aim: Design an HTML page, such that the registration form must validate: Name (not empty) Email (valid format) Password (minimum 6 characters)

Codes:

```
<!DOCTYPE html>

<html>

<head>

  <title>Registration Form</title>

</head>

<body>


<h2>Student Registration Form</h2>


<form onsubmit="return validateForm()">
```

```
    Name:
    <input type="text" id="name"><br><br>

    Email:
    <input type="email" id="email"><br><br>

    Password:
    <input type="password" id="password"><br><br>

    <input type="submit" value="Register">

</form>

<script>
function validateForm() {

    let name = document.getElementById("name").value;
    let email = document.getElementById("email").value;
    let password = document.getElementById("password").value;

    if(name.trim() == "") {
        alert("Name cannot be empty");
        return false;
    }

    let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

    if(!emailPattern.test(email)) {
        alert("Enter a valid email address");
```

```
        return false;
    }

    if(password.length < 6) {
        alert("Password must contain at least 6 characters");
        return false;
    }

    alert("Registration Successful");
    return true;
}
</script>

</body>
</html>
```

Output:

Explanation:

Logic Explanation – Registration Form Validation

The program validates user inputs for name, email, and password before allowing registration.

1. The input values for name, email, and password are taken from input fields using document.getElementById().
2. Name validation is performed to ensure the field is not empty:
   if (a === "")
   • If the name field is empty → an error message is displayed.
3. Email validation is performed using a pattern (regular expression):
   emailPattern.test(b)
   • The entered email is checked for a valid email format
   • If invalid → an error message is displayed.
4. Password validation is performed using a pattern:
   strong.test(c)
   The password is checked for:
   • Minimum 6 characters
   • At least one uppercase letter
   • At least one special symbol
5. Based on validation:
   • If all conditions are satisfied → Registration Successful message is displayed
   • Otherwise → an appropriate error message is shown.
6. The final result is displayed dynamically on the screen.

Logical Insight:
Instead of checking inputs manually, the program uses validation conditions and regular expression patterns to verify name, email, and password fields. This approach makes the registration process simple, secure, and easy to understand.

WEEK-12

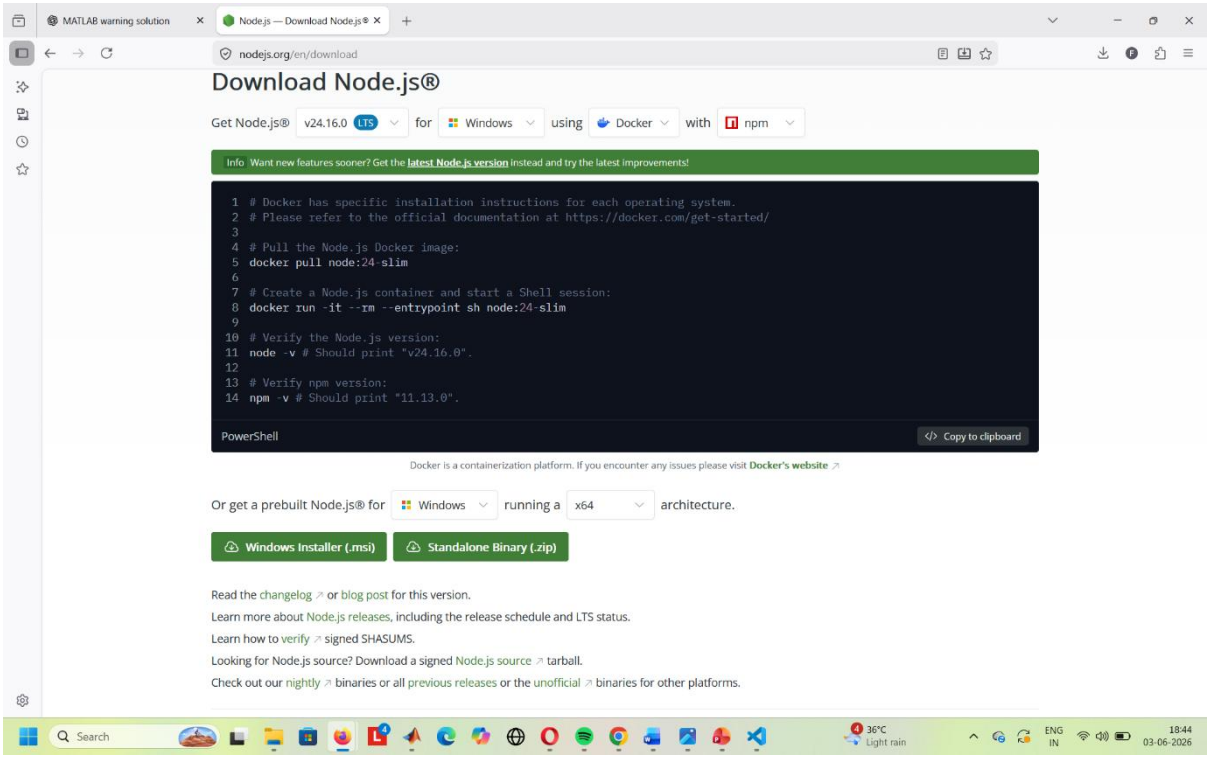
Aim: To create and execute a basic Electron.js desktop application using Visual Studio Code, Node.js, and Electron.

Introduction:

Electron.js is an open-source framework that enables developers to build cross-platform desktop applications using web technologies such as HTML, CSS, and JavaScript. Electron combines Chromium and Node.js into a single runtime environment, allowing web applications to function as desktop applications on Windows, macOS, and Linux.

Software Requirements:

- Visual Studio Code
- Node.js
- Electron.js
- Windows Command Prompt / VS Code Terminal



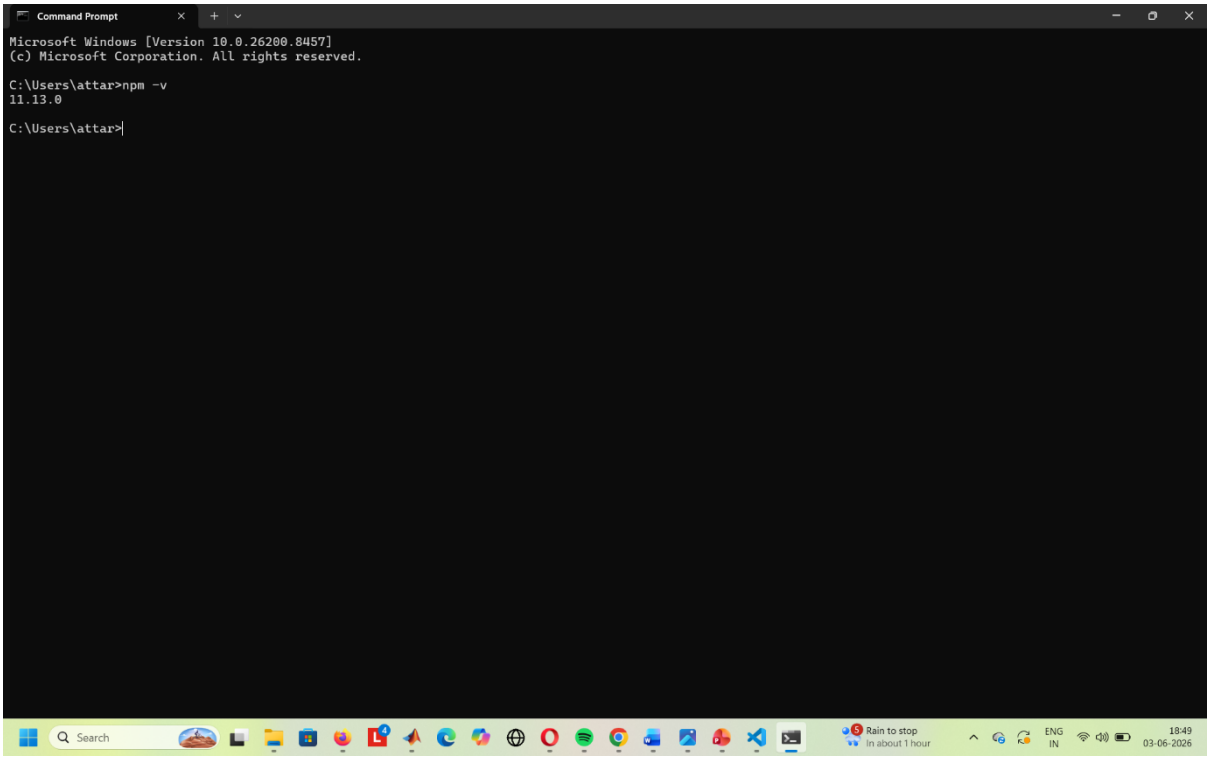
Prerequisite :

Electron requires Node.js to manage dependencies and run scripts.

• Download the latest LTS version of Node.js from nodejs.org

• Install it on your system (Windows, macOS, or Linux)

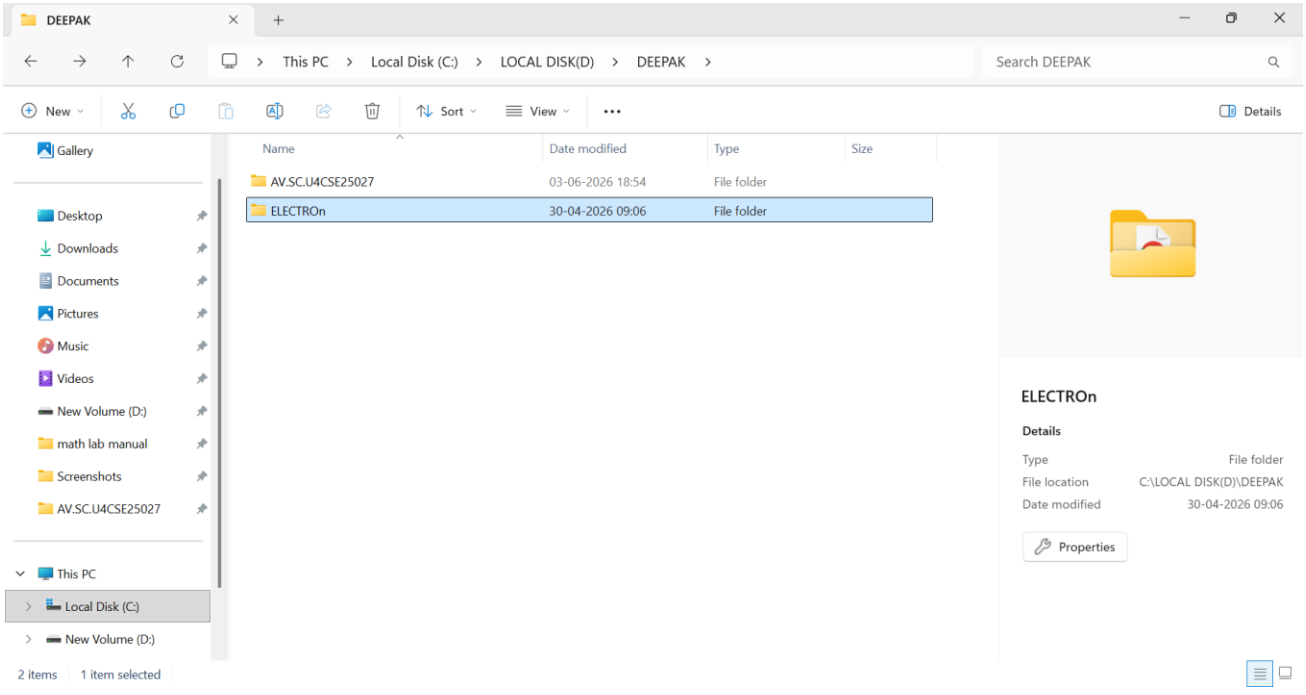
• Verify installation with node -v and npm -v



Procedure

Step 1: Create a New Project Folder

Create a new empty folder and open it in Visual Studio Code**.**



Step 2: Open Terminal

Open the integrated terminal from:

Terminal → New Terminal

Step 3: Initialize Node.js Project

Execute the following command:

npm init -y

This creates a package.json file with default settings.

Step 4: Install Electron

Run the following command:

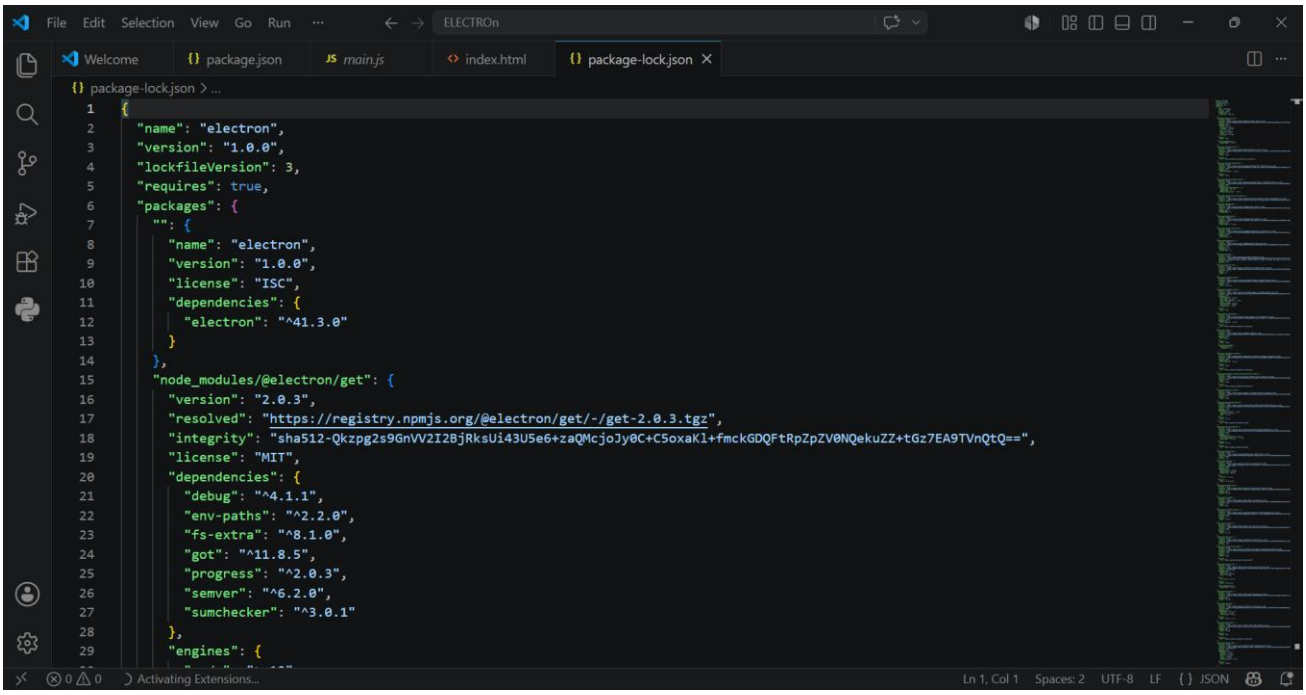
npm install electron --save-dev

This installs Electron as a development dependency**.**

Step 5: Configure package.json

Modify the package.json file and ensure it contains:

```
{
  "main": "main.js",
  "scripts": {
    "start": "electron ."
  }
}
```



Step 6: Create Project Files

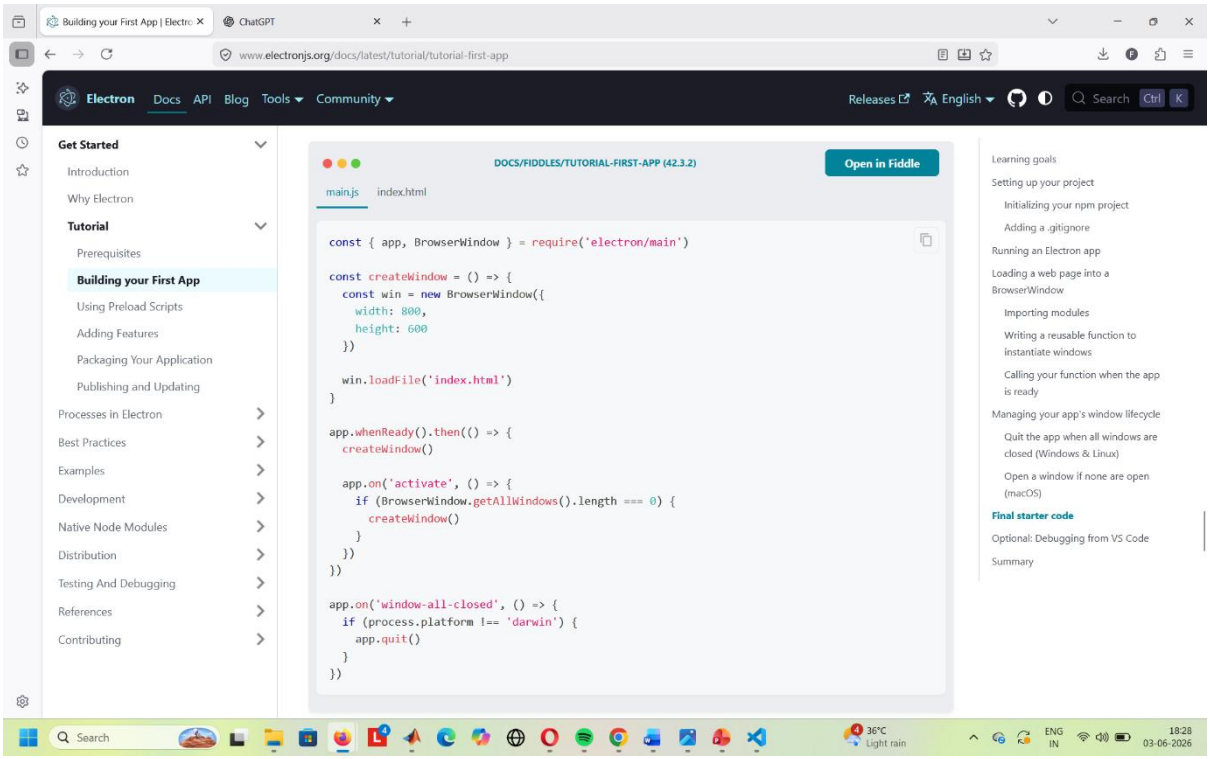
Open the official electron.js website and copy the code with the instructions given below.

Create the following files:

main.js

Copy the Final Starter Code from the Electron documentation:

Docs → Tutorial → Building Your First App → Final Starter Code



Step 7: Verify File Names

Ensure that the HTML file name specified in main.js matches the actual HTML file stored in the project folder.

Example:

mainWindow.loadFile('index.html');

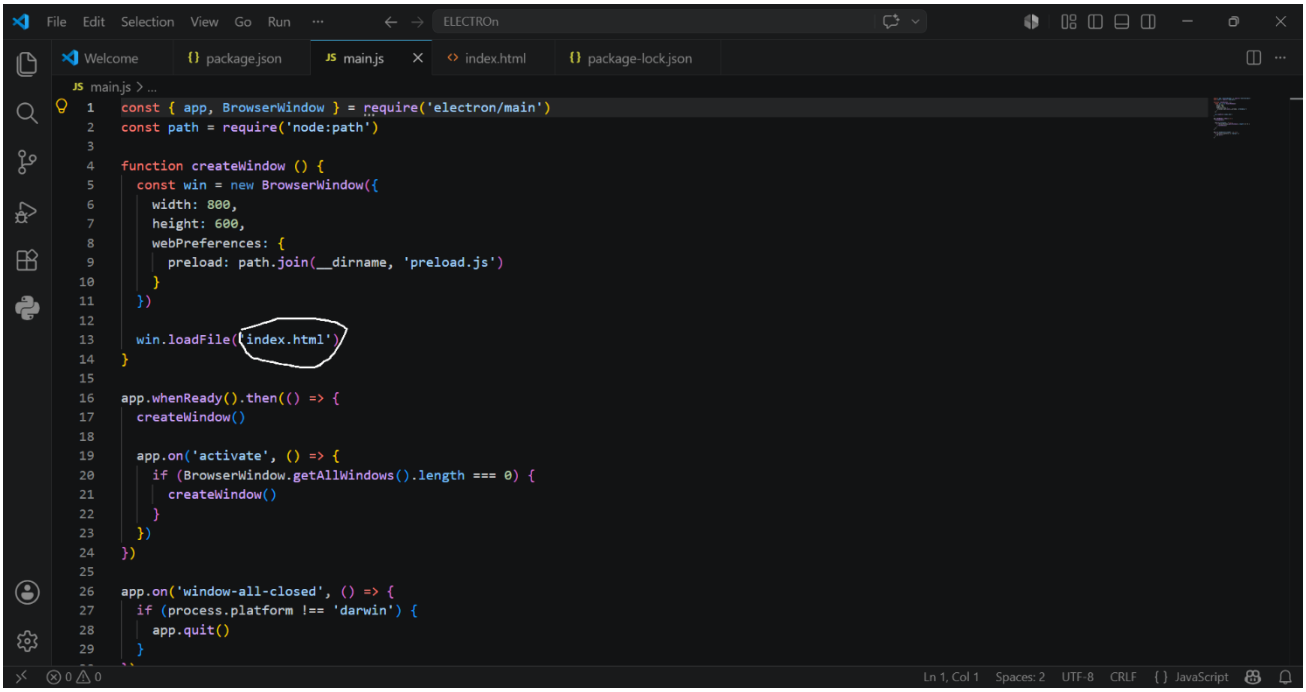
If the code loads index.html, the file in the folder must also be named index.html

Step 8: Run the Application

Execute the following command:

npm start

The Electron desktop application window will launch successfully.


File Edit Selection View Go Run ···
ELECTROn
Welcome
package.json
main.js
index.html
package-lock.json
main.js > ...
1 const { app, BrowserWindow } = require('electron/main')
2 const path = require('node:path')
3
4 function createWindow () {
5 const win = new BrowserWindow({
6 width: 800,
7 height: 600,
8 webPreferences: {
9 preload: path.join(__dirname, 'preload.js')
10 }
11 })
12
13 win.loadFile('index.html')
14 }
15
16 app.whenReady().then(() => {
17 createWindow()
18
19 app.on('activate', () => {
20 if (BrowserWindow.getAllWindows().length === 0) {
21 createWindow()
22 }
23 })
24 })
25
26 app.on('window-all-closed', () => {
27 if (process.platform !== 'darwin') {
28 app.quit()
29 }
Ln 1, Col 1
Spaces: 2
UTF-8
CRLF
{ } JavaScript